Yukio Technical Application and Feasibility Analysis

English edition · Evidence snapshot: 18 October 2026

Decision purpose: assess what the research and accessible implementation support, compare feasible routes, and identify the conditions required for later engineering decisions.

Repository reviewed: leozhang8654/yukio-desktop-pet, commit 460fb1935966ef2c3472a0bfa661fe90836e5855.

Review boundary: source inspection plus six narrowly scoped Python executions on constructed inputs. Native desktop applications, real client sessions, Qwen accounts and proposed integrations were not tested.

Reading route: §§1-3, 7-8 and 11-13 support decisions; §§4-6 explain mechanisms and alternatives; §9 defines falsifiable checks; §10 describes conditional architecture implications; §14 provides source scope and continuation context.

Contents

Page numbers and section titles link to the report.

1. Executive Summary 5

1.1 Scope, method and evidence 5

1.2 Findings that change the engineering decision 5

1.3 Recommended decision direction 6

2. Project Goals and Current State 7

2.1 Decision scope and the evidence available 7

2.2 Evidence status and feasibility are separate axes 8

2.3 Current software structure 9

2.4 Specific differences from the earlier proposal and documentation 10

2.5 Actual bounded execution results 11

2.6 What is presently unknown 11

3. Research-to-Requirement Mapping 13

4. Available Data and Observation Boundaries 16

4.1 Evidence status and execution boundary 16

4.2 Current official capability ledger 17

4.3 Per-surface capability matrix 21

4.4 Source semantics that cannot be unified by name 22

4.5 Version and coverage contract to validate 23

5. Candidate Technical Methods 24

5.1 Activity collection and evidence boundaries 24

5.2 Event processing, durable history and state reconstruction 26

5.3 Multi-agent relationships within the execution history 32

5.4 Evidence-grounded explanation 33

5.5 Qwen provider and interface feasibility 39

5.6 Human understanding and visualization 42

5.7 Human intervention: requests, issues and useful actions . . . 44
5.8 Evaluation: measure the observer . . . 46
5.9 Privacy and security across the observation path . . . 48
6. Comparative Analysis . . . 50
6.1 Observation routes: existing work versus cooperating execution - DEC-02/03 . . . 50
6.2 Identity and multi-agent relationships - DEC-01/04/11 . . . 50
6.3 Durable history and recovery - DEC-05/06 . . . 51
6.4 State and progress - DEC-07 . . . 51
6.5 Evidence selection and explanation - DEC-08 . . . 52
6.6 Qwen service and output protocol - DEC-09 . . . 52
6.7 Presentation and user participation - DEC-10/12/13 . . . 53
6.8 Evaluation, data policy and infrastructure - DEC-14/15/16 . . . 53
7. Feasibility Assessment . . . 55
8. Critical Unknowns and Technical Risks . . . 57
9. Validation and Experimental Plan . . . 59
9.1 Status, purpose and common test contract . . . 59
9.2 Staged gates and stopping rules . . . 59
9.3 Client interfaces and observation contracts . . . 60
9.4 Durable history, state and relationship invariants . . . 63
9.5 Explanation contracts and model operation . . . 64
9.6 Human understanding, safe participation and system boundaries . . . 66
9.7 Interpretation, sample limits and completion criteria . . . 68
10. Architecture Implications . . . 69
10.1 What the evidence changes . . . 69
10.2 Candidate responsibility boundaries . . . 69
10.3 One application process or a separate local collector? . . . 71
10.4 Data decisions that become expensive to reverse . . . 72

10.5 Complexity that can be deferred . . . 72

10.6 Failure behavior is part of the proposed architecture . . . 73

11. Recommended Priorities . . . 74

11.1 Current decision gate: establish what a supported observation means . . . 74

11.2 Next gate: demonstrate a reviewable, reproducible record . . . 74

11.3 Value gate: justify language generation and presentation . . . 74

11.4 Later gate: demonstrate useful participation . . . 74

12. Research and Development Dependencies . . . 75

13. Decision Matrix . . . 76

14. Supporting Evidence and References . . . 79

14.1 How to interpret and continue this evidence record . . . 79

14.2 Research and educational navigation . . . 80

14.3 Project inputs, pinned implementation and actual probes . . . 80

14.4 Current client and provider documentation . . . 87

14.5 Independently inspected engineering and research sources . . . 97

14.6 Inherited KB foundations and limitations . . . 108

14.7 Companion evidence and continuation files . . . 117

1. Executive Summary

Yukio’s core goal appears achievable as a bounded observer: preserve the activity its supported clients expose, reconstruct an explicitly incomplete history, and explain what that evidence supports. The foundational observation-and-history capability is B - Likely Feasible, Requires Validation. Broader narrative usefulness and intervention require separate assessment. The available evidence does not establish comprehensive observation of arbitrary desktop sessions, reliable general error detection, or an implemented Qwen explanation service. Those are different claims with different dependencies.

1.1 Scope, method and evidence

This analysis applies the Professional Research Knowledge Base’s six domains to the supplied project goals and the 13 October 2026 proposal. It cross-checks their assumptions against current official interfaces, the public repository pinned to commit `460fb1935966ef2c3472a0bfa661fe90836e5855`, and six recorded executions of selected, unmodified Python functions. The README advertises 0.4.6; the user’s installed binary, revision, operating-system build and client versions remain unknown. The probes used constructed inputs on Python 3.12.14, Linux x86_64. No desktop application, actual agent session, Qwen account or native reply route was exercised. PROJECT-GOALS; PROJECT-PLAN; KB; CODE-BASE; RUN-RECORD

Three classifications remain independent throughout. **Evidence status** describes what was inspected or tested: Verified Implementation, Code-Observed, Documented Capability, Research-Supported Method, Proposed Solution, or Unknown. **Feasibility A-E** describes applicability under stated conditions. **Decision status** describes whether to adopt a principle, validate an integration, investigate a question, defer a capability, or reject a proposed use. Adopting an evidence-preservation requirement does not certify its implementation.

1.2 Findings that change the engineering decision

The existing native applications provide useful parsers, source discovery, session routing, pet presentation and specific question-handling paths. Their `PetEvent` representation serves display behavior. It drops information needed for detailed evidence review, and some synthetic or inactivity-derived events express presentation choices rather than observed execution facts. A new history cannot recover content already discarded. The appropriate direction is incremental preservation and projection, with reuse of current presentation components where their behavior fits. CODE-EVENTS; CODE-SOURCES; CODE-PARSERS; CODE-ROUTER; CODE-UI

The bounded probes expose two concrete robustness priorities. Two constructed interleavings attach or clear failure information incorrectly; an equal-length file replacement is missed on the tested filesystem. Other fixtures confirm partial-object buffering and shrinking-file recovery. These results justify call-scoped association and file-generation validation. They do not estimate how often current clients produce those interleavings or establish native Windows/macOS behavior. RUN-01-RUN-06; RUN-RECORD

The candidate interfaces are richer than a logs-only assessment: current documents include Codex hooks and Claude SDK historical readers, alongside structured managed execution and

telemetry. Their semantics and access boundaries differ. Reading saved history, launching a cooperating run and subscribing to someone else's existing session require separate validation. Claude and Codex hook names cannot substitute for client-specific outcome rules. APP-CC-01; APP-CC-07; APP-CX-01; APP-CX-02; APP-CX-03

The project names Qwen 3.8 without identifying a provider, endpoint, account region, exact model or protocol. No explanation implementation was found in the scoped repository inspection. Alibaba documents relevant APIs, but its current structured-output guide contains contradictory model/region support statements. Strict output must be tested on the actual configuration; valid JSON would still not prove factual support. CODE-EXPLANATION-SCAN; APP-QW-01; APP-QW-02; APP-QW-03; APP-QW-04; APP-QW-05; APP-EG-02; APP-EG-03; APP-EG-04

1.3 Recommended decision direction

Adopt explicit observation boundaries, source identities, provenance, separate execution/verification/acceptance meanings, uncertainty and independent evaluation as requirements now. Validate a local evidence journal, deterministic state projections and supported source adapters before choosing a transport or database permanently. Compare templates with constrained LLM explanations on identical evidence. Compare the proposed pet-plus-workbench experience with an organized history on users' actual understanding. Preserve explicit question display as the narrower participation problem; defer inferred actionability until useful actions and reference labels exist. These are project-specific recommendations, not claims of completed implementation. KB §§3.01-3.04, 3.06-3.07; APP-HE-01; APP-HE-02; APP-HE-03; APP-HE-04; APP-HE-05

2. Project Goals and Current State

2.1 Decision scope and the evidence available

Yukio's immediate engineering problem is to preserve and explain observable work: tool requests and results, file-related operations, searches, checks, task and session relationships, and historical activity. The intended user should be able to distinguish current activity, an agent's completion claim, a check that actually returned, and a result that still requires acceptance. The longer-term problem is deciding when involving the user is useful. Reading hidden model reasoning, universal correctness assessment, unrestricted control of another client, and a universal project-completion percentage are outside the present scope. These priorities come from the current request and supplied project goal, not from a research paper. PROJECT-GOALS

The available sources support an application analysis with a bounded code review. They do not support a certification of the installed desktop product.

Input	Identity and inspection	What it establishes	What it does not establish
Current application-analysis request	Supplied directly in this conversation	Current requirements, exclusions, evidence categories and requested decisions	Current installed behavior
Project goal document	Local upload 02-AI-.docx; extracted in full	Activity-first scope and interest in a Qwen 3.8 API	Actual provider, endpoint, model entitlement or implementation
Earlier engineering proposal	Yukio AI Activity Detection Upgrade: Technical Assessment and Implementation Plan, dated 13 October 2026; all 15 pages read PROJECT-PLAN	Previous recommendations and the author's reported static findings	Continued accuracy of a moving branch, tests on the user's machine, or implemented proposed modules
Professional research knowledge base	17 October 2026 edition; source index contains 114 records KB	Research map, original-source identifiers, claim scope and retained inspection limits	Project-specific feasibility without further evidence
English educational textbook	Complete 30-chapter edition and Part 3 as explanatory references TEXTBOOK	Prerequisites and explanations of mechanisms	Independent experimental evidence
Public repository	main resolved to commit 460fb1935966ef2c3472a0bfa661fe90836e5855; source and relevant documentation inspected CODE-BASE	The inspected implementation at one immutable revision	The user's installed revision or a release binary's behavior
Bounded execution probes	Six constructed parser/tailer scenarios, executed in Python 3.12.14 on Linux x86_64 RUN-RECORD	Actual behavior of the imported Python functions for these inputs	Native Windows/macOS execution, real client trace representativeness or population accuracy

Input	Identity and inspection	What it establishes	What it does not establish
Current official interfaces	Official documentation investigated on 18 October 2026	Documented capability with source-specific qualifications	Availability in an unknown installed client, account or region

The repository commit is dated 13 October 2026 at 21:26:51 -07:00 and identifies a merge of plugin-chat changes. Its README advertises version 0.4.6. These are separate facts from the actual installation version, which remains unknown. The review did not launch either desktop app, subscribe to a real conversation, inspect the user's private session files, call Qwen, or issue an answer, approval, steering or interruption request. Source downloads and document-production operations are not product tests. CODE-BASE; RUN-RECORD

2.2 Evidence status and feasibility are separate axes

The report uses the six requested evidence statuses:

Status	Meaning in this report
Verified Implementation (VI)	A narrowly stated behavior exercised in an actual recorded test here. The fixture and runtime remain part of the claim.
Code-Observed, Not Tested (CO)	A behavior or reachable structure observed in the pinned source, without executing that path.
Documented Capability (DC)	A vendor's official interface description, or a clearly identified project's documentation claim.
Research-Supported Method (RS)	A theoretical result, empirical finding, standard or engineering method with its original conditions.
Proposed Solution (PS)	An application-specific design recommendation made in this analysis.
Unknown / Requires Validation (UV)	Necessary evidence is unavailable, contradictory or not yet tested.

A component can have CO evidence and a B feasibility rating. A design principle can be suitable to adopt now while the implementation remains unverified. Neither relationship upgrades it to A. A source's research label, such as K or E, answers a third question: the type of knowledge it contributes. An established mathematical result does not make a proposed Yukio integration established.

When the earlier proposal reports that code was reviewed, this report initially treats it as a document claim. Only findings independently located in the pinned revision are listed as CO. Reported historical tests in repository READMEs are not relabeled as tests performed during this analysis.

2.3 Current software structure

The inspected implementation has two native application lines. The macOS package separates YukioCore from the YukioPlayer executable. Its Swift package declares macOS 13 and a Sparkle dependency. The Windows tree contains Python event, classification, parser, source, router and application modules, with native-window integration and supporting UI code. This separation offers reusable logic and platform-specific presentation; it does not establish semantic parity between platforms. CODE-BASE; CODE-EVENTS

The current observation path is approximately: local source discovery and incremental reading, client-specific parsing, conversion to PetEvent, session routing and activity selection, then pet poses, captions and question/completion cards. This is a description of inspected responsibilities, not a claim that every source produces every activity. Existing source families include Claude Code, Codex and Deep Code. A DeepSeek model used through Claude Code still belongs to the Claude harness for observation purposes. CODE-SOURCES; CODE-PARSERS

Area	Code-observed finding	Application implication
Event representation	PetEvent carries one timestamp, source, session, event kind, optional call-like eventID, activity, tool, short detail, task-list data and a question CODE-EVENTS	Reuse it as a presentation interface. It is insufficient as the sole detailed evidence record.
Claude transcript parsing	Sidechain records are discarded; tool results become end/failure events; public text becomes a thinking or answer state CODE-PARSERS	A detailed history must capture authorized evidence before this loss of detail.
Claude optional hooks	The provided configuration covers UserPromptSubmit, PreToolUse, PostToolUse, Stop and SessionEnd CODE-HOOKS	Newer documented hooks are potential additions, not existing coverage.
Codex parsing	Rollout variants, parent-session metadata, calls, output, plans and selected UI events are recognized CODE-PARSERS	There is a useful compatibility base; correlation and evidence preservation need stronger contracts.
File following	Recent tails are replayed and byte offsets are held in memory; shrinking files reset the reader CODE-SOURCES	Current-state recovery is different from durable full-history recovery.
Pet routing	The router uses call tracking, finite deduplication memory, timestamp guards, display holds and inactivity thresholds CODE-ROUTER	Display policy must not become the authoritative execution-history model.
Detailed workbench	Retained assistant-window code does not establish an active AI-activity workbench CODE-UI	Window reuse is plausible but requires a new view and an actual activation path.
Explanation service	The scoped search found no Qwen/DashScope explanation-service implementation CODE-EXPLANATION-SCAN	Cloud explanation is a proposed extension with unresolved provider configuration.

The source tree contains task-list and session-title information, not just generic activity names. That is useful existing work. However, mutable task-list state without a goal/plan version cannot

establish what a prior progress denominator meant. A filename-sized caption also cannot support a summary of a file's contents. These are mismatches between the present data contract and the proposed use, not criticisms of its original animation purpose.

2.4 Specific differences from the earlier proposal and documentation

The public implementation still loses information before a future history page could use it. Tool outputs are reduced to activity-end or activity-failure signals, and explanatory text is reduced to display categories. Reconstructing detailed evidence later from those PetEvents is impossible for information already discarded. The justified architectural implication is to add preservation at the source/normalization boundary, not to make the detail string longer. CODE-EVENTS; CODE-PARSERS; KB §3.01.9, §3.03.4

Some source and UI events currently express display convenience rather than historical fact. Selected Codex search-completion records generate a synthetic start/end pair with the same timestamp. Source-file disappearance can generate taskAbort, and stale active sessions can be displayed as idle. These choices should retain their presentation role if useful, but cannot justify "the search started now," "the task was cancelled," or "the agent finished." A future evidence view needs recorded origins and independent observation health. CODE-PARSERS; CODE-SOURCES; CODE-ROUTER

The hook documentation is stale. The pinned macOS README says that PostToolUseFailure and StopFailure do not exist. Current official Claude documentation includes these events. Conversely, newly documented events cannot be assumed available in old installations. The correction is a versioned capability contract and a test, not a universal replacement list. Current Codex documentation also exposes Hooks; its semantics must be checked independently from Claude's similarly named hooks. CODE-MAC-README; APP-CC-01; APP-CX-03

The independent assistant page remains a candidate reuse point. Static inspection found reminder/home/personal/extensions structures in the retained macOS assistant window, but the active setup path initializes reminder components rather than instantiating that controller. Windows show_assistant returns without opening a page. Neither proves an existing activity workbench. The useful reuse is native window infrastructure and platform conventions, conditional on wiring and testing the new view. CODE-UI

Existing reply routes are more specific than a universal control connection. Current code and documentation distinguish Codex, hook/bridge and other request paths. A legacy Windows answer path may use foreground-guarded typing or clipboard behavior for certain pending-question conditions. Its actual reachability from a current client was not tested. Do not generalize a background route's properties to every dispatcher branch, or infer general steering from a question-answer path. CODE-REPLIES

Two statements in the earlier plan require stronger qualification. "Same events in a different order give the same final state" requires defined ordering semantics and a convergent projection; arbitrary reducers are not order-independent. "Hooks do not affect the observed task" should become a measured low-interference objective. A synchronous process launch, file write or timeout can affect latency even if a script always exits successfully. PROJECT-PLAN, pp.12-13; APP-CC-01; KB §3.02.7-8

2.5 Actual bounded execution results

The following are real executions of the unmodified Python logic at the pinned revision. Every input record was constructed for this analysis; no row is a captured Codex event. The supplied probe script records the input, expected fixture contract, observed output, runtime and source hashes. These checks were chosen to resolve two concrete static risks, rather than to estimate overall accuracy. RUN-RECORD

Probe	Constructed condition	Actual observed result	Legitimate conclusion
RUN-01	Failure notice for A immediately followed by A's output; then B's output	A becomes activity_failed; B becomes activity_end	This ordered fixture is handled as expected.
RUN-02	A and B started; A's failure notice; B's output; A's output	B becomes activity_failed; A becomes activity_end	A single pending-failure value can attach the failure to another call for this accepted interleaving.
RUN-03	A's failure notice; a new B start; then outputs	Both outputs become activity_end	A subsequent start clears the pending failure in this fixture.
RUN-04	A fully read file is atomically replaced at the same path with equal-length new JSON	No new object is returned	Size-only detection misses this replacement on the tested filesystem.
RUN-05	An incomplete JSON object is completed by a later append	No premature object; then the complete object is returned	The tested partial-object buffering works.
RUN-06	A read file is rewritten to a smaller size	The new object is returned	The tested shrinking-file reset works.

RUN-02 and RUN-03 establish a parser robustness counterexample, not the frequency or even reachability of this ordering in a particular released Codex client. The next evidence needed is a version-matched source trace or an official ordering guarantee. RUN-04 concerns Linux temporary files; Windows and macOS file identity and replacement behavior still require native checks. The Swift implementation contains analogous structures but was not executed.

The results already justify making call-scoped correlation and source-generation identity high-priority validation subjects. They do not justify a numerical Yukio reliability claim, a blanket statement that existing activity detection is broken, or an A rating for the proposed upgrade.

2.6 What is presently unknown

No installed Yukio binary, deployment commit, operating-system build, client build, client launch mode, permissions, authentic representative log set, Qwen account configuration, cloud-data preference, typical concurrent workload or history volume was supplied. These are explicit inputs to the validation plan. None prevents a useful candidate comparison; each limits a specific deployment conclusion.

Plain-language interpretation: there is already an implemented foundation in the source, with real parsers and display logic. The proposed upgrade asks that foundation to support a stronger

promise: a trustworthy account of what happened. Preserving evidence, distinguishing uncertainty and checking interfaces are the necessary conditions for that stronger promise.

3. Research-to-Requirement Mapping

This table is the coverage contract for the analysis. “Relevant Research” points to the primary knowledge-base dependency; the Evidence column identifies project or independently rechecked sources. The B/C/D distinctions can apply to different parts of one requirement. None of these rows establishes an end-to-end A capability.

Yukio Requirement	Relevant Research	Candidate Methods	Preconditions	Evidence	Limitations	Feasibility
REQ-01 - Identify supported clients and observation scope	KB §3.01: instrumentation boundaries	Versioned capability records; release-matched adapters	Actual build, surface, configuration and permissions	CO: CODE-SOURCES; DC: APP-CC-01, APP-CX-01; APP-CX-03	Product name alone says little about accessible records	B for validated surfaces; D for inaccessible ones
REQ-02 - Record tool requests and results	KB §§3.01, 3.02: lifecycle and identity	Hooks; structured streams; transcript/ history readers	Call identity and source-specific outcome semantics	CO: CODE-PARSERS; DC: APP-CC-01, APP-CX-02; APP-CX-03	A request is not execution; similarly named hooks differ	B within a declared source contract
REQ-03 - Explain file, search and test activity	KB §§3.02, 3.03: artifact scope and evidence	Native result records; scoped artifact evidence; optional effect comparison	Permitted content, test scope and version association	CO: CODE-PARSERS; DC: APP-CC-01; APP-CC-02, APP-CX-03	Shell writes and hosted search can bypass selected hooks; effects do not establish intent	B for explicit evidence; D for hidden categories
REQ-04 - Report source health and freshness	KB §§3.01.9, 3.02.11	Receipt timestamps, parser diagnostics, gap and connection states	Known delivery path and observation clock	CO: CODE-SOURCES; RS: APP-ENG-05	Source silence cannot decide completion or failure	B for local health; D for unknowable upstream omissions
REQ-05 - Associate project, task, session and run	KB §3.02.3	Explicit namespaced IDs; reversible user association	Stable identity fields or declared ambiguity	CO: CODE-EVENTS; DC: APP-CC-03, APP-CX-01	Directory names and shared files can mislead	B for explicit links; C for semantic inference
REQ-06 - Handle concurrency, attempts and redelivery	KB §§3.02.4-8	Call/attempt keyed records; idempotent delivery handling	Identity lifetime, source ordering and retry definitions	VI: RUN-01; RUN-02; RUN-03; RS: APP-ENG-08	Accepted constructed inputs expose failure-pairing risk; real-client incidence unknown	B after adapter and correlation validation

Yukio Requirement	Relevant Research	Candidate Methods	Preconditions	Evidence	Limitations	Feasibility
REQ-07 - Preserve reviewable history and recover	KB §3.02.7: event-based projections	SQLite journal; segmented JSONL plus index	Retained permitted evidence, durable cursors, source generations	CO: CODE-SOURCES; VI: RUN-04; RUN-05; RUN-06; RS: APP-ENG-01; APP-ENG-02; APP-ENG-03; APP-ENG-04; APP-ENG-07	Local persistence cannot recover events never captured	B for retained evidence; D for unavailable past records
REQ-08 - Reconstruct state under partial observation	KB §§3.02.6-8, 3.02.11	Deterministic multidimensional projections; explicit unknowns	Event meanings, clock domains and ordering policy	CO: CODE-ROUTER; RS: APP-ENG-05; APP-ENG-08	Display smoothing and total time order are not execution truth	B for bounded rules; C for latent-state inference
REQ-09 - Separate ending, checking and acceptance	KB §§3.02, 3.03.3-7	Separate claims, operation outcomes, artifact-bound checks and acceptance	Check scope/version and identifiable acceptance evidence	RS: APP-EG-01; APP-EG-02; PS: state/ explanation proposals	Successful commands and agent reports do not prove the user's objective	B for recording distinctions; D where evidence is absent
REQ-10 - Show meaningful task counts	KB §3.04.4	Versioned plan-item counts; named milestones	Observed plan revision and explicit denominator	CO: CODE-EVENTS; PROJECT-PLAN	Counts measure recorded plan labels, not total effort or project completion	B for scoped counts; E for a universal percentage
REQ-11 - Produce evidence-supported explanations	KB §§3.03.2-7, 3.07.6	Templates/ excerpts; structured claims plus constrained LLM generation	Sufficient selected evidence, immutable references and support checks	RS: APP-EG-02; APP-EG-03; APP-EG-04; APP-EG-06; APP-EG-07; PS: explanation contract	Citation existence and schema validity do not establish entailment	B for bounded facts; C for demonstrated narrative benefit
REQ-12 - Use the intended Qwen service	KB §§3.03, 3.07: generation and validation	Actual provider's supported API; local-only fallback	Provider/ region/ workspace/ model/protocol tuple and entitlement	UV: project configuration; DC: APP-QW-01; APP-QW-02; APP-QW-03; APP-QW-04; APP-QW-05; APP-QW-06; APP-QW-07; APP-QW-08; APP-QW-09; APP-QW-10; APP-QW-11	Current schema support documentation conflicts; no account call performed	B conditionally; D where service entitlement prevents a route

Yukio Requirement	Relevant Research	Candidate Methods	Preconditions	Evidence	Limitations	Feasibility
REQ-13 - Support rapid, accurate user understanding	KB §§3.04.1-6	Pet plus evidence workbench; organized history with anchors	Consistent projections, retained references, accessible navigation	CO: CODE-UI; RS: APP-HE-01; APP-HE-02; APP-HE-03	Building a view does not demonstrate improved understanding	B construction; C comparative user benefit
REQ-14 - Preserve multi-agent handoffs	KB §§3.02.3, 3.03.3	Typed relationship records; reviewed candidate associations	Sender/ receiver/call IDs and available input/return records	CO: CODE-PARSERS; DC: APP-CC-10, APP-CX-01; APP-CX-06	Sending, receiving and subsequent use are distinct evidence claims	B for explicit edges; D for hidden recipient context
REQ-15 - Show requests requiring user decisions	KB §§3.06.1-3	Passive display and original-client navigation; tested bound reply channel	Pending request identity, expiry, options and channel lifecycle	CO: APP-HE-15; DC: APP-CC-09, APP-CX-01	A cleared request or local receipt is not proof of approved execution	B on tested routes; D otherwise
REQ-16 - Detect useful future intervention opportunities	KB §§3.06.2-5	Explicit rules/digests; later anomaly or utility policies	Actionable episode labels, feasible user actions, timing/cost data	RS: APP-HE-04; APP-HE-05; APP-HE-08; APP-HE-09	Unusual behavior, failure and useful intervention are different targets	C for narrow policies; E for universal detection
REQ-17 - Measure observer reliability and responsiveness	KB §§3.07.1-8	Independent references; replay plus real controlled runs; user studies	Frozen matching rules, representative conditions and honest denominators	VI: RUN-RECORD; RS: APP-HE-07; APP-HE-10; APP-HE-11	Agent success, synthetic scenarios and satisfied users are insufficient substitutes	B for a bounded evaluation program
REQ-18 - Preserve privacy and resilient operation	KB §§3.01, 3.03.9, 3.07	Scoped retention and cloud excerpts; metadata-only option; inert rendering	Explicit data-flow policy, actual provider configuration, fault tests	RS: APP-HE-12; APP-HE-13; DC: APP-QW-03; APP-QW-06; APP-QW-07	Redaction can remove needed evidence; no universal secret detection guarantee	B for defined boundaries, subject to validation

4. Available Data and Observation Boundaries

4.1 Evidence status and execution boundary

The interface statements below are **Documented Capability**, unless explicitly identified as a limitation of this investigation or as a **Proposed Solution**. No Claude Code, Codex, or Deep Code session was started, resumed, interrupted, or connected to in this investigation. No model API was invoked. The user's installed client versions, deployment modes, effective configuration, permissions, and actual event samples remain **Unknown / Requires Validation**.

The primary documents were retrieved on the analysis date. Most are continuously maintained pages without a publication date or a complete minimum-version table. An access date is therefore a documentation snapshot, not a supported-client version. Named minimum versions below apply only to their particular documented features. They do not establish the first version supporting the whole proposed integration. Exact package versions and generated schemas must be recorded during validation.

Research foundations are the Professional Research Knowledge Base (KB), §§3.01.3-3.01.9, §3.01.90, and §3.02: instrumentation boundaries, partial observation, identity, delivery, causality, and lifecycle semantics. The educational bridge is Textbook Part 3, Chapters 11-15. The engineering recommendations here are a synthesis of those foundations and the newly inspected official documents. They are not client vendor guarantees.

Yukio needs to distinguish three product situations. First, the user independently starts an agent and Yukio observes available evidence. Second, the user chooses to launch a run through a cooperating integration. Third, Yukio imports a historical execution after it has ended. These situations may involve the same model and repository, but have different access and lifecycle obligations.

Technical explanation. A stream created by a subprocess belongs to that subprocess. A callback registered in a harness belongs to the configured harness. A stored transcript is a persisted representation of a run. None of those relationships automatically creates a subscription to another process. Conversely, a historical reader may offer useful evidence without needing to run the agent at all. A rich integration can improve data access while also making Yukio responsible for connection handling, pending requests, and shutdown behavior.

Plain-language explanation. There is a difference between watching an agent through its existing records, asking the agent to send you updates, and becoming the application through which the agent works. Yukio should choose deliberately which situation it supports. Obtaining a better event stream may change the user's workflow.

This distinction determines the appropriate experiment. A successful run launched by Yukio would demonstrate managed-run integration. It would not demonstrate observation of an already-open desktop session. An imported history would demonstrate retrospective readability. It would not establish that every original event arrived in real time.

4.2 Current official capability ledger

4.2.1 Claude Code: lifecycle hooks

Claude Code documents pre-call, successful-result, failed-result, permission, subagent, and turn-ending hooks. `PostToolUseFailure` and `StopFailure` are present; `StopFailure` concerns API-error termination. `PreToolUse` excludes activities that do not become tool calls: prompt file mentions can insert contents directly. An `Edit|Write` matcher misses shell-based writes. `updatedToolOutput` can replace the result delivered to Claude after telemetry has already captured the original output. These are distinct evidence layers. APP-CC-01

Application implication - Proposed Solution. Treat a hook as a record that a named checkpoint was reached. Preserve its native event name and tool identity before translating it. A pre-call checkpoint should support “requested” or “about to be processed,” not “executed.” A result needs its own evidence. A successful result still needs an application-specific interpretation: a shell command can succeed without verifying the user’s objective.

An observation-only handler should return no model-facing instructions, replacements, approvals, or continuation decisions. Its failure behavior must also be examined: merely intending to observe does not remove an interface’s ability to affect execution. A compact local handoff may reduce blocking work, but its loss and shutdown behavior require measurement.

4.2.2 Claude Code: telemetry and tracing

Monitoring documents opt-in logs, metrics, and beta traces. Prompt, tool-detail, and content switches differ. Detailed tracing additionally needs `ENABLE_BETA_TRACING_DETAILED=1` and `BETA_TRACING_ENDPOINT`; interactive CLI use has an organization-allowlist condition. Content can be truncated. `event.sequence` is process-scoped and can repeat within a resumed session. `prompt.id` and `tool_use_id` provide narrower correlation. `message.uuid` has explicit version conditions, including v2.1.214 for the listed ordinary message events. APP-CC-02

Application implication - Proposed Solution. Consider telemetry as an additional source, particularly for durations, errors, correlation, and collection diagnostics. Do not describe all telemetry modes with one “tracing enabled” switch. A capability record should state which signal, content settings, exporter, and execution surface are actually active. The absence of a content field may be intentional redaction; it must not be converted to an empty action or an invented description.

Using a standard transport can reduce custom ingestion work. It does not make the event semantics universal, establish loss-free delivery, or supply the exact artifact version tested. Those properties require source-specific checks.

4.2.3 Claude Code: managed command-line execution

The programmatic CLI offers `json` and `stream-json`; partial streaming also uses `--verbose` and `--include-partial-messages`. Subagent messages use `parent_tool_use_id`. Text forwarding requires v2.1.211+, all-depth forwarding v2.1.219+, and some forked-skill paths have later requirements. A prompt-started forked skill can arrive as a completed batch rather than live. SIGTERM can leave an unfinished turn without a result. APP-CC-03

Application implication - Proposed Solution. This is a useful controlled integration and evaluation route when the user deliberately launches that run through the integration. Retain source occurrence information separately from receipt time; batch arrival should not be animated as if it were current execution. The controller must distinguish normal stream completion, a final result, pipe closure, and an interrupted process. A missing final result should remain a specific evidence gap.

The CLI route is also useful for generating consented, reproducible observation fixtures. It need not become the default way every user starts an agent.

4.2.4 Claude Desktop execution surfaces

The Desktop documentation distinguishes local, SSH, WSL, and cloud sessions. Local/SSH setup can use plugins and hooks, while cloud plugin inheritance differs. Shared local settings do not mean identical remote configuration. The document also describes side chats that are not saved to disk. Therefore even a supported desktop product can contain interactions outside a retained local-history channel. APP-CC-04

Application implication - Proposed Solution. Identify where the work actually executes and where its records exist. A desktop window connected to a remote host is not evidence that all execution records are on the desktop machine. Expose surface-specific coverage instead of a single “Claude supported” label. The first compatibility result should name, for example, a local Code-tab environment, not the whole Claude product family.

4.2.5 Claude Agent SDK: managed execution versus historical reading

The Agent SDK runs the Claude Code harness from an application-operated process in Python or TypeScript. It is different from a model client SDK, where the application supplies its own tool loop. APP-CC-05 The sessions guide separately documents disk-reading functions: TypeScript `listSessions()`/`getSessionMessages()` and Python `list_sessions()`/`get_session_messages()`. It also states that the experimental TypeScript V2 session API was removed in SDK 0.3.142. APP-CC-07

Application implication - Proposed Solution. Compare a supported session-reading API with the existing raw-transcript route before committing to long-term internal-format parsing. This comparison does not require using the SDK to start a new agent. Evaluate whether the reader returns the records Yukio needs, its pagination and filtering, resource use on large histories, behavior during concurrent writes, and whether old transcripts remain readable. The main TypeScript reference could not be retrieved in this investigation because the browser fetch exceeded its content-size limit; precise signatures and filtering must therefore be checked against the selected SDK release.

A library reader can move some format maintenance to the vendor’s library, but it also adds a runtime dependency. A small native parser may remain appropriate for a narrow, version-pinned compatibility path. Neither option should silently claim complete history.

4.2.6 Claude SDK hooks, streaming, and user participation

The SDK hook table is not language-uniform: `StopFailure`, `SessionStart`, and `SessionEnd` are listed for TypeScript but not Python. Shared tool hooks carry a call identifier; agent identity fields also have language-specific placement. Callback timeouts have event-specific effects, including blocking a timed-out pre-tool callback. APP-CC-06

Partial-message streaming yields raw API events in addition to complete messages. Structured-output chunks are unvalidated until the final structured result. APP-CC-08 The user-input guide distinguishes tool approval from `AskUserQuestion`; `canUseTool` is not called for every automatically approved operation. APP-CC-09 Subagent documentation exposes parent-call and agent/task associations, while older invocations used `Task` where newer ones use `Agent`. APP-CC-10

Application implication - Proposed Solution. Keep stream assembly, tool execution, approval state, and task interpretation separate. Finishing a generated tool-argument block is not evidence that the tool completed. A callback capable of returning a permission decision is a control interface even when Yukio only wants to display a prompt. For the current product stage, an observed pending request can be shown with its source and scope; supplying an approval through Yukio requires an independently validated interaction path.

4.2.7 Codex: App Server

App Server represents threads, turns, and items through version-specific schemas. `thread/read` does not subscribe or load a session; history pagination is separate. `thread/start`/`thread/resume` serve the active lifecycle. `turn/completed` can contain `completed`, `failed`, or `interrupted`. `collabToolCall` carries sender/receiver/new-thread associations when available. Approval resolution can mean answered **or cleared**. The current guide labels WebSocket transport experimental and unsupported. APP-CX-01

Application implication - Proposed Solution. App Server is a strong candidate for a cooperating integration, and potentially for supported stored-history access. It is not yet an established route for passively attaching to the user’s arbitrary existing desktop session. The experiment must identify the actual server, transport, thread ownership, authorization, lifecycle side effects, and whether another client’s pending requests are delivered safely.

Schema generation from the selected installed binary is a valuable compatibility artifact. Store the resulting schema alongside the adapter’s test fixtures. Treat notifications as typed protocol messages; do not translate every method ending in “completed” into “successful.” Do not resume a conversation merely to populate a history page when a non-resuming read meets the need.

4.2.8 Codex: non-interactive execution and SDKs

`codex exec --json` exposes JSONL events such as `thread.started`, `turn.started`, `turn.completed`, `turn.failed`, and `item.*`. Item families include commands, file changes, MCP calls, search, messages, and plans. `--ephemeral` suppresses persisted rollout files. `--output-schema` concerns the final response, not the event stream. APP-CX-02

The current SDK guide documents TypeScript local-thread operation and a Python SDK controlling local App Server. Python builds include a pinned CLI runtime; the documented minimum is Python 3.10, while TypeScript requires Node.js 18+. APP-CX-04

Application implication - Proposed Solution. Use managed execution when the launch workflow itself is acceptable. Compare the CLI stream with an SDK in terms of packaging, cancellation, history, error handling, and event completeness rather than assuming a library always produces richer evidence. A final JSON summary generated by the model remains a claim-bearing output; it should not replace the execution ledger.

4.2.9 Codex: hooks

Codex now documents lifecycle hooks, including tool, permission, session, and subagent events. Hooks need trust review. `PostToolUse` includes nonzero Bash exits. Hosted `WebSearch` bypasses local tool hooks; unified-exec completion may be delivered during `write_stdin`. Subagent hooks use the parent `session_id` plus `agent_id`. Transcript paths are conveniences, not a stable format contract. Background handlers can finish out of order and are cancelled at session end. APP-CX-03

Application implication - Proposed Solution. Include Codex hooks among the candidates even if an earlier assessment considered only transcripts and App Server. Preserve separate Claude and Codex mappings. Identical names and compatibility aliases are insufficient evidence of identical outcome semantics. A hook-only Codex capture should explicitly declare hosted-search coverage absent for that channel; another channel may supply it.

The documented `main`-branch schemas can contain fields not in a release. A candidate integration must use release behavior and observed samples, not merely accept a schema downloaded from the newest repository branch.

4.2.10 Codex: telemetry, notifications, configuration, and administration

Advanced configuration documents opt-in OTel run/API/tool events with asynchronous batching and shutdown flushing; tool results include a snippet rather than a full-result guarantee. The older external `notify` mechanism is limited to `agent-turn-complete`, while terminal notifications are a different mechanism. APP-CX-05 Configuration reference lists separate log, trace, and metric exporter controls; CLI analytics settings can disable custom metric exports. APP-CX-07

Codex subagents have separate agent threads and can return summaries to the main thread. APP-CX-06 Authentication documentation distinguishes local clients from cloud access and identifies account/workspace controls. APP-CX-08 Enterprise compliance records form an additional administration-scoped route; the guide explicitly does not claim complete local command, file, screenshot, approval, or external-action coverage. APP-CX-09

Application implication - Proposed Solution. A desktop notifier is suitable for a narrow completion indication, not a comprehensive activity record. Telemetry can supplement the main ledger and expose transport behavior, but configuration precedence and redaction must be measured. Administrative audit APIs should remain a separate enterprise option; ordinary desktop operation should not depend on organization privileges the user may not possess.

4.2.11 Deep Code

The DeepSeek integration guide identifies the terminal client `@vegamo/deepcode-cli`, its shared settings with the Deep Code VS Code extension, and a `notify` script after each model turn. It does not specify a complete stable activity-event schema. APP-DC-01

The repository audit, rather than this vendor guide, should establish which native history fields Yukio currently parses. Keep this client separate from a Claude Code or Codex harness configured to use a DeepSeek model. A provider identity does not determine a harness's event format. Deep Code's precise installed build, history contents, error records, concurrency semantics, and notification payload require validation; a notification setting alone cannot establish the requested observation coverage.

4.3 Per-surface capability matrix

Here, **DOC** means a data category is documented for the relevant surface, subject to configuration and versions; **INF** means a more ambitious claim would require inference; **ABS** means the cited channel does not supply the stated category; **UNK** means this investigation has not established it. None of these symbols means that Yukio has successfully collected the data.

Surface	Direct evidence candidate	Inference boundary	Channel limitation / unknown	Yukio applicability
Claude configured hooks	DOC: named lifecycle checkpoints and call data	INF: whether a tool's result fulfills a user objective	ABS: operations outside matched checkpoints; UNK: installed event coverage	B, validate a pinned surface; D for unavailable checkpoints
Claude OTel	DOC: configured events, metrics, optional spans	INF: full workflow or artifact verification from telemetry alone	ABS: fields suppressed by configuration; UNK: completeness and exporter recovery	B as a supplementary source; C for detailed-beta dependence
Claude managed CLI	DOC: run stream and permitted subagent output	INF: semantic task acceptance	UNK: arbitrary existing desktop-session attachment	B for user-selected managed runs; D for unsupported attachment
Claude SDK historical reader	DOC: enumerating/reading stored sessions	INF: current liveness from old records	UNK: exact release filters and concurrent-write behavior	B, compare against native parsing
Claude SDK-managed agent	DOC: stream and supported callbacks	INF: inferred purpose or task meaning	ABS: complete internal cognition; UNK: language-specific callback availability	B if managed execution fits the product; E for a complete hidden-cognition claim
Surface	**Direct evidence candidate**	**Inference boundary**	**Channel limitation / unknown**	**Yukio applicability**
Claude remote/cloud Desktop	DOC: documented surface-specific mechanisms	INF: treating desktop visibility as local data availability	UNK: accessible record location, policy and connection	Unassessed pending surface inventory; B for a validated

Surface	Direct evidence candidate	Inference boundary	Channel limitation / unknown	Yukio applicability
				permitted route, D where access is withheld
Codex configured hooks	DOC: supported local lifecycle/tool data	INF: operations outside that hook path	ABS: hosted-search events through these hooks	B for covered actions; D for that channel's gap
Codex `exec` / SDK-managed run	DOC: run/event output	INF: task correctness from final response	UNK: passive attachment to another live frontend	B for a selected managed-run workflow
Codex App Server	DOC: stored history and connected-run protocol	INF: acceptance from turn termination	UNK: safely observing another frontend's live session	B for scoped integration; arbitrary attachment remains unassessed (D if the surface prevents it)
Codex OTel / external notifier	DOC: configured telemetry / narrow turn notification	INF: complete individual execution from aggregates or a toast	UNK: exact collection latency and persistence	B as supplements, not a sole rich-history promise
Enterprise compliance export	DOC: authorized, supported audit resources	INF: complete device activity	ABS: general guarantee of every local action	D without required administration access
Deep Code native history / notify	DOC: documented notification option; code evidence handled separately	INF: task correctness or full coverage from notification	UNK: stable history/schema guarantees	B for bounded parsing after validation; D where the client withholds required data

The letter B means **Likely Feasible, Requires Validation**, C means **Experimental / Research-Dependent**, and D means **Limited by External Systems**. These are conditional applicability assessments, not measured accuracy and not "Confirmed Feasible" integration results.

4.4 Source semantics that cannot be unified by name

Proposed Solution. Represent client family, execution surface, session/thread, turn/run, agent, tool call, and delivery separately. Preserve native identifiers and add a collector-owned identity when the source does not provide one. Never pretend that a generated observer identifier was emitted by the client. Include a process-incarnation identifier where ordering counters can reset. These recommendations follow KB §3.02's identity analysis.

A single "session ID" is not sufficient to determine an agent, attempt, or delivery. Likewise, the same path may refer to different worktrees or versions. Pairing should primarily use documented call identity, then explicit protocol relationship, and only then a clearly labelled heuristic. A global "pending failure" cannot safely represent two in-flight calls; this is a structural reasoning result independent of any particular current implementation.

Plain-language explanation. Imagine two tests finishing in the opposite order from which they started. Their own identifiers let Yukio attach each answer to the right test. Looking only at “the next message” can attach a failure to the wrong action.

The proposed observation model should distinguish requested arguments, accepted arguments, raw tool result, delivered tool result, agent-authored summary, and observer interpretation. If only one layer is available, say which one. Do not duplicate a tool call merely because both a hook and a transcript represent it; retain both sources as evidence for one correlated call when the identity supports that relationship.

A test record also needs the artifact version and test scope if it is to support “this version passed these checks.” A file path and a later success statement are weaker evidence. The Part 3 textbook’s delayed-result/later-edit case supplies the educational example; it remains fictional, not a Yukio runtime result.

4.5 Version and coverage contract to validate

The following is a **Proposed Solution** for a machine-readable adapter capability record. It describes information to collect, not an invented sample from the user’s environment:

Field group	Required meaning
Client identity	Harness family, executable/package version, build or schema identifier, SDK language/runtime where relevant
Execution surface	CLI, SDK-owned process, desktop local, worktree, WSL, SSH or cloud; location of execution and evidence
Collection channel	Hook, structured stream, history reader, raw transcript, OTel or other specific source
Effective configuration	Enabled event families, content gates, filters and trust state; sensitive values excluded
Identity guarantees	Namespaces/lifetimes of session, turn, agent, call and sequence identifiers
Delivery contract	Flush/teardown behavior, ordering, duplication, recovery and known retention constraints
Content scope	Original versus model-visible output; truncation/redaction; artifact-version information
Test evidence	Fixture IDs, expected/observed categories, environment, result and failure notes
Coverage statement	Supported, unsupported and unverified questions; freshness of the last validation

A version mismatch should trigger a compatibility state, not make the product silently keep interpreting new data with an old certainty level. An unknown event can be retained as an unsupported record, with its source and parser diagnostic, rather than discarded or coerced into a familiar state. This keeps the historical evidence available for later reprocessing.

The source-reading and managed-run alternatives should be tested separately. A richer official interface expands the candidate set; it does not establish installed compatibility. The route comparison in §6 and validation cards in §9 preserve this distinction.

5. Candidate Technical Methods

5.1 Activity collection and evidence boundaries

Section 4 establishes what current official interfaces document. This section compares their application mechanisms with the inspected Yukio path. All integrations beyond the pinned code are Proposed Solutions, and all installed-version assumptions remain subject to validation.

5.1.1 Semantic checkpoints versus observed effects

Technical explanation. A client-level hook or structured protocol can describe a logical tool invocation, including a call identity and a result category. An operating-system notification can instead describe an effect such as a file changing. A repository diff compares selected file states. These observations answer related but different questions. Normalization should retain that distinction.

Plain-language explanation. A work order, a worker's report and a photograph of the finished file are useful records. They are not interchangeable. The work order may never have been carried out; the report may be incomplete; the photograph may include changes made by somebody else.

Question	Stronger candidate evidence	What still cannot be assumed
Was a particular tool requested?	Identified pre-call record or structured invocation	That permission was granted, the tool started, or its side effects occurred
Did a tool return a result?	Correlated result record with source outcome semantics	That its result was correct or sufficient for the user's objective
What content was returned to the agent?	The model-visible result at the relevant transformation stage	That the original tool output or telemetry copy was identical
Did files change?	Permitted version/diff or file-effect evidence	Which actor caused every change, or whether all intermediate states are retained
What did a search return?	Recorded search result with truncation and source scope	That a query record contains the results, or that the source was exhaustive
What was tested?	Invocation, selected checks, environment/version evidence and result	That passing a command verifies all requirements or later file versions
Why did the agent act?	An attributed agent explanation or an explicit task relationship	Complete internal reasoning or a uniquely established mental cause

The proposal should therefore measure several coverage dimensions: event families, fields, lifecycle phases, execution surfaces, time intervals and permitted content. A "read event captured" metric cannot substitute for "the returned content was retained." A stronger semantic source may expose less raw content for privacy reasons, while a lower-level source may expose effects with weak task identity. KB §§3.01.3-3.01.9; §§3.03.2-3.03.4

5.1.2 Client records, hooks and managed integrations

The existing local parser is a useful baseline because it fits the application's current deployment and can read retained history. Its liabilities are source-format changes, bounded startup replay, missing generations and discarded evidence. A supported history reader is a meaningful alternative when it returns the required fields without starting or resuming work. Compare both against the same retained sessions; neither must be selected solely because it is already implemented or vendor-provided.

Hooks can provide clearer lifecycle identity and earlier delivery for covered checkpoints. They also introduce an executable integration inside the source's workflow. Keep capture handlers separate from permission and result-transforming handlers; preserve both the intended observation role and the actual timeout/output behavior. Background execution trades some blocking risk for ordering and teardown uncertainty. A local acknowledgement proves only the stage that acknowledged, unless durable persistence is explicitly part of that contract.

A managed CLI or SDK can offer a controlled stream and reproducible launch conditions. That is attractive for opt-in operation and evaluation fixtures. It changes product responsibility: Yukio must own process or connection lifecycle and represent partial shutdown. A tool that is easy to launch under Yukio is not thereby observable when launched independently elsewhere. These options are compared per surface in §4 and by maintenance and interaction cost in §6.

5.1.3 Filesystem and operating-system supplements

Candidate OS-1: scoped filesystem notification plus periodic reconciliation. Watch a consented project or known record directory, then read the relevant contents using the source-generation contract. Periodic scans provide a reconciliation path after notifications are missed. This can improve discovery latency without requiring full system tracing. It does not reveal past file contents automatically, and generic file notifications may omit actor identity or coalesce changes. The tested equal-length replacement is a reader-continuity problem, not something fixed merely by changing the polling interval. RUN-04; APP-ENG-09; KB §3.01.7

Candidate OS-2: process or kernel-level instrumentation. ETW, Endpoint Security and Linux eBPF have different providers, event vocabularies, permissions and delivery constraints. The KB documents ETW loss counters, Endpoint Security notification versus authorization behavior, and nonblocking BPF buffer reservation failure. The inspected AgentSight preprint demonstrates a system-level approach in a specific Linux/Claude Code environment; it does not validate macOS/Windows compatibility or a universal agent-error detector. These are legitimate research alternatives, with platform and data-volume costs beyond the current local-log baseline. KB §3.01.7; S-OT09; S-OT10; S-OT11; S-OT15

Candidate OS-3: version or diff inspection. Comparing allowed file versions can verify a visible change independently of an agent's statement. A commit identifier alone is insufficient for uncommitted work, generated files, dependencies or an isolated test workspace. A test-evidence record should retain the best available artifact and execution scope; if the relevant version cannot be established, show that limitation. Reading the current file after a result arrives does not reconstruct the exact bytes previously read or tested.

An OS monitor should be promoted only for a named gap, such as verifying that a claimed write has a visible effect. Broad monitoring without a specific question increases permissions,

unrelated-content collection and attribution ambiguity. User-interface inspection is a further bounded supplement for visible request state; it is dependent on layout and surface recognition and does not supply a stable execution history by itself.

5.1.4 Combining sources without manufacturing certainty

Two channels can describe the same tool call. Preserve their separate record identities, their relationship and whether they share an upstream producer. Agreement between a transcript and telemetry generated by the same harness is useful for transport checks, but may repeat the same source-semantic error. A file snapshot may provide a more independent check of an effect, while still needing actor and version attribution.

Conflict handling should first ask whether the observations concern the same call, attempt, transformation stage and artifact. A raw result and a transformed model-visible result can legitimately differ. Two payloads with the same immutable source identity and incompatible contents should instead remain an integrity conflict. More sources are worthwhile only if they improve a specified evidence question enough to justify capture, correlation, privacy and maintenance costs.

For this product stage, the preferred comparison is narrow: current parsers versus supported history readers and configured hooks, with a managed stream available for deliberately launched runs. Optional telemetry and effect evidence can address known gaps. A claim of complete cross-client observation remains outside what this investigation establishes.

5.2 Event processing, durable history and state reconstruction

5.2.1 Define the object before selecting the algorithm

Technical explanation. Yukio observes records about executions. It does not own all the events that constitute those executions. Therefore, the proposed journal is authoritative about what Yukio captured and how it interpreted that evidence; it is not automatically the authoritative event store of Claude Code or Codex. Event Sourcing supplies a useful pattern for rebuilding an observer’s projections, conditional on preserved inputs and versioned interpretation. It cannot recover absent external actions or historical tool outputs that were never captured. APP-ENG-07; KB §3.02.7; TEXTBOOK Ch.12

Plain-language explanation. A notebook can reliably preserve everything written into it. That does not mean everything that happened was written down. Yukio should be able to replay its notebook, while also showing where the notebook is incomplete.

Start with an evidence envelope and preserve the raw source vocabulary. A tool “completed” in one protocol may mean that a response is available; in another it may be an execution terminal state with a separate success/failure field. A single unified success flag would destroy that distinction. Event normalization should expose shared concepts and keep unmapped details, rather than pretending every source has an identical lifecycle. KB §3.01.90, §3.02.2

The following is a proposed semantic contract, not an implemented class or a required serialization standard:

Field group	Information to retain	Why the distinction matters
Record identity	Local observation ID; source record ID and namespace when supplied; delivery ID if useful	One occurrence can have repeated deliveries or several representations.
Source position	Source family and version, adapter version, file/stream generation, byte range or sequence	A path or sequence number may be reused after restart or replacement.
Execution identities	Project, logical task, session, turn/run, agent, tool call and attempt, with nullable fields	A session is not necessarily one task; a retry is not a duplicate.
Time	Source-reported occurrence time, collection time, commit sequence/time, source clock domain	An event that arrives late should not rewrite what Yukio knew earlier.
Action and result	Original event name; normalized lifecycle fact; tool, request, result status and available exit information	“Requested,” “returned” and “succeeded” are different assertions.
Evidence	Retained bytes or controlled reference, hash, range, encoding, artifact version, origin	A later file read is different evidence from the agent’s original tool response.
Quality	Partial, truncated, redacted, dropped, unparsed, unavailable, stale or conflicting, with reasons	Missingness should remain visible after a successful parse.
Interpretation	Goal and plan versions; parser, rule and projection versions	Reprocessing with new logic must not masquerade as the original decision.
Relations	Edge type, endpoints, evidence references and relation status	Parentage, delegation, delivery, return and later use are distinct facts.

OpenTelemetry’s log model independently distinguishes occurrence and observation timestamps and permits an absent source timestamp. This supports the distinction, but adopting these fields does not require deploying an OpenTelemetry collector or labeling local records as a standard-compliant export. APP-ENG-05

5.2.2 Identity, deduplication and retries

Candidate ID-1: explicit, source-scoped correlation with conservative gaps. Use the strongest available identifiers and documented namespace. Correlate a request and result by call identity, then retain each lifecycle event separately. If no source event ID exists, a file-generation identifier plus complete-record byte range can identify a captured record. This key identifies a record, not a real-world occurrence. Preserve additional deliveries or representations without inventing extra operations. Where two sources both describe one call, retain both provenance records and add an equivalence relation supported by the interface contract.

The main advantage is auditability: an incorrect relation can be inspected and revised. Its cost is adapter-specific mapping and explicit unmatched records. It fits the current parser architecture, but the existing eventID field cannot be repurposed as a universal unique event key because it currently serves call pairing. A local UUID can identify an unresolved record without claiming an upstream task identity. **Recommendation: adopt the identity rules now; validate each mapping before relying on it.** CODE-EVENTS; CODE-PARSERS; KB §3.02.3

Candidate ID-2: temporal or language-based association. Similar text, nearby timestamps and shared directories can suggest a relation when identifiers are absent. These clues can assist a user or prioritize review. They also merge unrelated concurrent operations, split retries, and confuse same-name projects or worktrees. Model-generated relationships require their own ground truth and error evaluation; uncertainty is not removed by fluent labeling. A tentative edge must remain distinguishable from an explicit edge and should not establish verified completion. **Recommendation: investigate for optional suggestions, not authoritative initial correlation.**

Project association deserves a separate policy. A stable local project identifier can be linked to one or more roots, worktrees and sessions. Directory names alone are insufficient: a directory can move, a process can change working directory, and two tasks can share a repository. Explicit user association and source-provided workspace identity are stronger than an unreviewed semantic guess. Record the reason for an association, and permit correction without altering the underlying source record.

The actual pending-failure probes reinforce this requirement: retaining “some operation failed” separately from the operation identity is unsafe when later records can interleave. A proposed replacement should hold evidence by call and attempt, with unresolved failure facts kept unresolved. It should not guess the missing association merely to produce a red or green display. RUN-02; RUN-03

5.2.3 Compare storage routes before choosing SQLite

Route	Mechanism and required input	Advantages	Costs, limits and compatibility	Decision
H-1: SQLite journal and projections in the application	Store captured records, evidence references, source cursors and derived views transactionally	Local queries, constraints and atomic multi-record updates; natural fit for one desktop installation	Schema migration, retention, contention and durability configuration need ownership; not implemented in the reviewed source	**Validate First**; strongest initial candidate for the stated local-history requirement
H-2: append-only JSONL segments with a rebuildable index	Write explicit segment and record boundaries; rebuild query indexes from preserved records	Transparent interchange, straightforward export and offline inspection; fits existing JSONL experience	Crash-safe cursor/index coordination, concurrent writers, rotation, large-history queries and deletion need custom design	**Validate First** as a baseline or simpler low-volume alternative
H-3: an external observability backend	Export appropriately instrumented records to a service with its own storage/query contract	Useful for shared analysis, richer trace exploration and centralized operations	Additional deployment, connectivity, credentials, data-governance and maintenance burden; does not fill source blind spots	**Consider Later** unless remote/team requirements justify it

SQLite is suitable for many embedded application-data uses, but its transaction guarantee is not a guarantee about data written outside the database or events never emitted upstream. The official documentation describes atomicity and its operating assumptions. APP-ENG-01; APP-ENG-03; APP-ENG-04 The application-specific reason to prefer H-1 for a pilot is the need to

commit an evidence record and its consumed-source position together, while querying across tasks and versions. That rationale should be tested against H-2 on actual workload shape.

A safe candidate commit boundary is: read a complete record; preserve its permitted evidence; add normalized facts or an explicit unparsed-record marker; advance the durable cursor in the same transaction. A crash before commit leaves the cursor unchanged and permits re-reading. A crash after commit may cause redelivery, which identity-aware insertion handles. This is a proposal for effectively once-applied observer facts under specified retention and recovery assumptions, not a claim of universal exactly-once acquisition.

Small evidence fragments can live in the transaction itself. If large evidence is stored as separate files, “save evidence and cursor atomically” is no longer a database-only promise. One candidate is to stage and durably finalize immutable evidence first, commit its database reference next, then clean up unreferenced leftovers. A different candidate keeps all required evidence in the database. File durability, permissions, cleanup races and deletion must be tested in either design. A hash checks byte identity; it does not authenticate the producer.

WAL and rollback journaling are separate SQLite choices. WAL can allow readers alongside a writer, but still has one writer at a time, checkpoint behavior, same-host restrictions and possible busy results. The official WAL page also identifies a rare WAL-reset race fixed in 3.51.3 and certain backports. The correct gate is to record the SQLite actually linked by each application and its fix status before adopting concurrent WAL use. Do not infer that the operating-system framework or Python package already contains the fix. APP-ENG-02

Plain-language comparison. SQLite offers a coordinated filing system: saving an event and marking it read can succeed together. JSONL is easier to inspect directly, but the application must coordinate its notebooks and indexes itself. Either can work; the comparison is about recoverability and query needs, not about which technology sounds more advanced.

5.2.4 Source recovery must identify a generation, not only a filename

The current reader successfully waits for a partial JSON object in the constructed probe and resets after a size decrease. Equal-length file replacement is not detected in that test. This is a direct reason to validate replacement and generation handling before promising durable history. RUN-04; RUN-05; RUN-06

A proposed file-source contract should distinguish an append to the same file generation, truncation, replacement, rename/rotation, temporary inaccessibility and permanent removal. Native file identity helps, but identifiers can be reused; use it with generation metadata and source-specific checks. Modification time alone can have limited resolution and is not a proof of append-only continuity. If continuity cannot be established, start a new generation and report the uncertainty instead of applying an old offset blindly.

An OS file notification can wake a reader promptly, while periodic reconciliation checks the directory and offsets. Polling is a reasonable baseline at desktop scale; event-driven discovery reduces unnecessary scans but needs overflow and rescan handling. Microsoft documents that directory-change notifications can overflow and require enumeration. Neither route alone supplies a file’s past contents, an agent’s intention or a causal link to a particular tool call. APP-ENG-09

Hook capture introduces another recovery problem: concurrent producers. The pinned shell hook appends stdin to an inbox, and the reader can truncate that inbox above its threshold. The code has no explicit shared transaction between appenders and that truncation. This is a static race concern, not a reproduced data-loss result. Compare a serialized local receiver with an acknowledged spool, or carefully coordinated append segments and rotation. Failure to start a receiver, disk-full behavior and a client's hook timeout must be part of the comparison. CODE-HOOKS; CODE-SOURCES

This also limits backfill. A session API or retained transcript may recover some activity after Yukio restarts, but recovery cannot be promised for ephemeral hooks, deleted logs, disabled retention or data never recorded. A historical import should carry its capture mode and coverage interval. A "complete" label must refer to a declared source contract and interval, not the entire project by default.

5.2.5 Separate state dimensions

A proposed state view should answer several different questions:

Dimension	Examples of supported values	Evidence needed
Execution lifecycle	Request observed; start reported; result observed; terminal reported	Correctly scoped lifecycle records
Operation outcome	Success, failure, rejection, cancellation, unknown	Source-specific result semantics, not a completion word alone
Task completion claim	Agent reported complete; no completion claim captured	Attributed agent statement
Verification	Check passed/failed for artifact V and scope S; partial checks; verification unknown	Check identity, inputs, artifact version and result
User acceptance	Accepted, rejected, not yet reviewed, unknown	An authorized, explicit acceptance record with requirement scope
Observation health	Live collection, catching up, partial, stale, disconnected, conflicting	Collector and source diagnostics

Not every field must occupy the default UI. The separation is needed in the data and explanation contracts so that simplification does not change the meaning.

Candidate S-1: keyed deterministic projection with explicit uncertainty. Store observed lifecycle facts by call and derive the current display using versioned rules. A result can exist without a captured start. Missing results produce "result not observed"; fresh source-reported liveness can support a narrower running claim. Contradictory records preserve a conflict rather than using whichever text arrived last. This method is inexpensive, testable and compatible with the current native logic. Its limitation is that it cannot supply facts the sources do not expose. **Recommendation: primary validation candidate.**

Candidate S-2: probabilistic hidden-state estimation. An HMM or related filtering model can rank unobserved states if meaningful hidden labels, transition parameters, observation likelihoods and missingness assumptions can be identified. It can support a calibrated prediction or a query policy, not replace recorded results. It brings data collection, model fitting, drift monitoring and evaluation costs. Long tasks, permission waits and collector outages can produce

similar silence through different mechanisms. **Recommendation: investigate later for a clearly defined question with independent state labels.** KB §3.02.90-91; TEXTBOOK Ch.15

Candidate S-3: language-model state assignment. A model can summarize a record set or suggest an interpretation, but free-form state classification is sensitive to context selection, prompt content and unsupported inference. It offers flexibility for open vocabulary while weakening reproducibility. **Recommendation: do not use it to overwrite authoritative lifecycle or validation fields.** Its useful role is constrained explanation of S-1 and explicitly labeled hypotheses, subject to evidence checks.

The deterministic route can preserve a set of compatible possibilities without implementing a probabilistic model. For example, after a start and no result, the system can retain “still active or result not captured” as an unresolved condition. The model should also distinguish a request that has never been confirmed started. This is more informative than hiding all uncertainty inside a numerical confidence score.

5.2.6 Time, partial order and late corrections

Lamport’s ordering model establishes that local order and message relationships can define a partial order; a chosen total order is not unique causal history. APP-ENG-08 Applied to Yukio, sort for readability while preserving whether an edge comes from a sequence contract, a request/result relationship, an explicit handoff or a display-only ordering choice.

No passive adapter can create missing vector-clock propagation after the fact. A local ingestion counter identifies a stable record order in Yukio, not the original causal order across agents. For performance durations within Yukio, use an appropriate monotonic clock; for user-facing source times, retain the source clock and units. Cross-host subtraction is not meaningful without clock assumptions.

The proposed replay contract has two modes:

1. **Historical evidence replay:** rebuild from the evidence retained at a specified collection cutoff, using specified interpretation versions. It answers what Yukio could support then.
2. **Revised historical view:** incorporate later evidence or improved parsing, while retaining the revision’s provenance. It answers what the currently available record supports about the past.

Neither mode re-executes tools. Replaying should not send notifications, answer questions, call cloud APIs by default or modify the observed workspace. If explanations are regenerated for research, label them as new outputs against an old evidence snapshot.

Determinism means that the same retained inputs, interpretation versions and ordering policy produce the same projection. Order independence is a stronger property and requires deliberate design. Idempotent fact insertion can converge after duplicates and reorderings, but a mutable plan update or a goal change must still follow its valid source order. The earlier plan’s permutation test should preserve legitimate causal/source-order constraints and explicitly include the cases that should remain unresolved.

Worked reasoning example, fictional. A test invocation explicitly bound to v1 returns at source time 6 but reaches Yukio at collection time 10. A later edit produced v2 at time 4, and the agent said “verified” at time 7. At cutoff 8, Yukio cannot cite the test result it has not received. At cutoff 10, it can report the v1 test result, while v2 remains outside that check’s proven scope. Neither receiving the result nor reordering the timeline changes the artifact version. This is the

transferable lesson from the textbook's Part 3 lab, not a measured Yukio run. TEXTBOOK Ch.13, Ch.15 and Part 3 synthesis lab

5.2.7 Where advanced methods belong

Process Mining can investigate recurring workflow patterns once case identity, event meaning and data quality are stable. Conformance Checking can test an explicit local process rule. Neither establishes that an unfamiliar research or coding path is wrong merely because it differs from a frequent path. Petri nets are useful when a specific concurrency or synchronization question warrants the model; they need not become the live application's storage format. KB §3.02.9, §3.02.12

A small invariant monitor is a more proportionate initial candidate: an explanation reference must resolve within its permitted task/snapshot; a known failure must not be relabeled as success; a check result must retain its artifact scope. These are explicit engineering invariants with testable inputs. Automatically converting arbitrary user goals into a complete correct monitor remains a separate research problem.

A graph database is similarly optional. Relational edge tables can store several relationship types without forcing a tree or introducing a graph server. An object-centered view is an analytical choice, not a mandate to adopt OCEL or PROV serialization throughout the application. The principle to retain is that one event may involve a call, an artifact and a task simultaneously. KB §3.02.9; APP-ENG-06

5.3 Multi-agent relationships within the execution history

Technical explanation. A collaboration history should represent observed interactions rather than assume a single complete organizational tree. A parent relation, a delegation request, recipient-side context, a returned result and later use are separate edges. Every edge should identify its evidence, scope and whether it is observed or inferred. The same worker can contribute to several tasks, and a task can cross sessions. KB §3.02.3, §3.03.3; APP-ENG-06

Plain-language explanation. Seeing A send work to B does not prove exactly what B received or whether A later used B's answer. Yukio can preserve the visible parts of that exchange and show which parts remain unavailable.

Compare two practical routes. **MA-1** uses explicit parent/call/message identifiers and a small relationship table, with user correction for uncertain project membership. It is easy to audit and fits a local database, but offers incomplete graphs when clients hide recipient data. **MA-2** infers semantic handoffs from adjacent messages, shared filenames or a model's description. It may surface useful candidates, but its errors can contaminate the apparent task history. MA-1 is the initial validation candidate; MA-2 should remain a reviewed suggestion until relation-specific evaluation demonstrates value.

Do not use a desktop pet's selected session as the root of truth. Selection is a presentation decision. The pinned Codex source preserves some parent-session metadata, whereas the Claude transcript path discards sidechains and discovers a narrower directory shape. Removing one filter therefore cannot guarantee subagent coverage: discovery, identity, message content and source contracts all matter. CODE-SOURCES; CODE-PARSERS

Relationship question	Acceptable positive evidence	If it is missing
Was work delegated?	An identified delegation/tool request and sender	Record an unlinked candidate, not a confirmed delegation.
Which worker was created or selected?	Source-provided worker identity and parent/delegation linkage	Keep recipient identity unresolved.
What did the worker receive?	Recipient-side input with known visibility/filtering scope	State that complete received context is unavailable.
What returned?	Correlated result/message and referenced artifacts	Distinguish worker termination from result availability.
Was the return used?	Explicit citation or a supported downstream action/reference	Say “subsequent use not observed,” not “ignored.”
Did a requirement get lost?	Versioned requirement plus sufficient sender/recipient evidence	Treat as unassessable; absence from an incomplete transcript is insufficient.

The next stage should validate a small two-worker handoff with known inputs, a missing-recipient-context variant and a return that the main agent does not visibly cite. The goal is to establish which relation claims are supportable. Detecting general collaboration errors or diagnosing an agent’s private reasoning is not an initial deliverable.

5.4 Evidence-grounded explanation

The following methods apply KB §§3.03 and 3.07. Textbook Chapters 16-18 explain the concepts; Chapters 25-26 cover their assessment. No model experiment or Yukio explanation accuracy estimate was produced in this investigation.

5.4.1 What an explanation can legitimately claim

Technical explanation. Grounding asks whether an identified evidence set supports a statement in its context. AIS makes contextual meaning part of attribution, including time and reference resolution; its experiments concern conversational QA, summarization, and table-to-text, not Yukio logs. PROV distinguishes entities, activities, and responsible agents; recording that an activity used A and produced B does not, by itself, establish that A influenced B. APP-EG-01; APP-EG-02; KB §§3.03.2-3.03.3, S-GE01, S-GE03.

For Yukio, the useful unit is a claim with a subject, predicate, object, execution identity, applicable artifact version, observation cutoff, and evidence relationship. “The tests passed” is underspecified: which test invocation, which selected tests, which working-tree version, and according to which reporting channel? A syntactically valid record can support “the tool reported success” while the underlying operation remains incorrectly implemented or insufficiently checked.

Plain-language explanation. Yukio should tell users what the available record establishes and make the supporting record easy to inspect. A receipt saying a test was requested is different from a result saying that test passed. Neither automatically proves that the user’s bug is fixed.

The following distinctions should survive generation and presentation:

Evidence situation	Defensible explanation	Additional claim requiring separate evidence
A source records a read request	“A read of `auth.py` was requested.”	The agent received the complete file or understood it.
A tool result reports a nonzero exit code	“This command reported an unsuccessful exit.”	The whole task failed, or its output has no useful information.
An agent says the repair is finished	“The agent reports that the repair is finished.”	Independent verification or user acceptance occurred.
Selected tests pass on version v1	“These selected tests passed on v1, according to this result.”	Tests cover the full requirement or also verify v2.
A subagent result is recorded	“This subagent result was recorded.”	The parent received or correctly incorporated that result, or the subagent received every intended instruction.
Several reads precede an edit	“The sequence is consistent with investigating this issue.”	The observer knows the agent’s actual internal reason.

These are engineering deductions from evidence semantics, not measurements. A purpose explanation should identify whether it is a quoted agent report, a documented task relationship, or an external interpretation. A visible `reasoning_content` field from the explaining model is also generated text; it is not independent evidence of the observed agent’s internal computation.

Four different failures must remain distinguishable: a false source report, a missed source record, a wrong association between records, and an unsupported explanation of correctly associated records. Improving the last stage cannot recover information absent from the earlier stages. Repeated copies of one agent statement are not independent corroboration. KB §§3.03.4, 3.03.8; §§3.07.2, 3.07.6.

5.4.2 Candidate explanation methods

The following comparisons are **Proposed Solutions** based on mechanisms and research. Their relative performance in Yukio requires a paired evaluation.

5.4.2.1 Deterministic summaries and evidence excerpts

Technical explanation. Map a documented event contract to a bounded sentence template. Resolve display text from parsed fields and attach the underlying record. Derive status separately through the state projection. For example, a correctly paired result can produce “Test call T1 returned exit code 1”; the command’s semantic category can remain unknown if its meaning was not established.

Plain-language explanation. This resembles filling in a carefully written form. It gives a reliable account of familiar actions with little waiting and can continue offline.

The route needs well-defined adapter semantics, stable references, escaping of displayed text, and explicit treatment of missing fields. It is compatible with the existing caption approach, although a fuller evidence record is needed to support detailed history. Complexity is low for a small event vocabulary and increases with tool-specific interpretation, localization, and

contextual summaries. No model request is required, but locally stored excerpts still need retention and access controls.

Its limitation is informational ambition: “read X; edited Y; command failed” may be accurate but may not explain how these actions relate to a user’s goal. Extracting a sentence can also remove a negation or its version qualifier. Templates are therefore a baseline and a fallback, not a guarantee of correct upstream interpretation. KB §3.03.5.

5.4.2.2 Retrieval-augmented free-form generation

Technical explanation. Select relevant task context and activity evidence, then ask a model to explain them with record references. This permits flexible grouping, comparisons across attempts, and accessible descriptions of technical output. Retrieval and generation are separate components: missing the decisive failed test during selection cannot be repaired by asking the generator to be more careful.

Plain-language explanation. This gives a writer a packet of records and asks for a readable account. The account can be useful, but the writer may connect facts incorrectly or miss a crucial qualification.

The route needs a provider contract, authorized evidence transmission, a stable evidence packet, and semantic evaluation. It adds model latency, usage cost, and prompt maintenance. It can run behind existing captions without changing animation behavior, but it is unsuitable as the authoritative producer of lifecycle state or acceptance status. Long histories, same-named files, ambiguous task boundaries, and conflicting reports increase the risk of misleading narratives. A document-sized context window is not a demonstrated solution to these association problems.

ALCE separates content quality from citation quality, while FActScore decomposes output into supported atomic facts. Their methods inform evaluation; neither proves that unrestricted prose becomes correct merely because it includes citations. APP-EG-03; APP-EG-04; KB §§3.03.5-3.03.7.

5.4.2.3 Structured claims with constrained language generation

Technical explanation. Build a bounded factual core from validated events; allow a model to propose contextual claims or paraphrases; then apply reference, identity, version, scope, and semantic checks before display. The host owns execution status. A rejected claim is removed, narrowed, or displayed as unavailable without changing the underlying event history.

Plain-language explanation. Yukio first establishes a small set of things it can support, then asks the model to explain those things clearly. The model can improve the wording without deciding that unobserved work succeeded.

This hybrid has a strong fit to the stated need for readable, auditable activity explanations. It offers a deterministic fallback and local rejection of impossible references. It costs more to define the claim vocabulary and test the interaction between rule-derived and model-generated content. Structured fields can still carry false meanings, and a model can convert a cautious structured claim into overconfident prose. If a second model performs semantic checks, its correlated errors and additional latency must be measured.

For narrow facts, deterministic realization is a credible alternative to another LLM call. For open-ended purpose and synthesis, keep a separate inferred category. The recommendation is

Validate First for the hybrid and **Adopt Now** for the separation of factual state, evidence, and generated explanation as a design principle. This does not imply immediate deployment.

5.4.2.4 Comparative decision table

Route	Preconditions and compatibility	Principal benefit	Failure and maintenance burden	Provisional position
Templates plus expandable records	Defined event contracts; strongest continuity with existing captions	Predictable, inexpensive, offline account	Limited task meaning; growing template vocabulary; source errors remain	Establish as comparison baseline and fallback; B
Free-form RAG	Good selection, provider, independent labels; additive service	Flexible contextual explanation	Unsupported connections, omissions, version mixing; model and prompt drift	Controlled comparison, not authoritative state; C
Structured hybrid	Evidence identities, factual projection, schema and semantic checks	Auditable claims with flexible language	More contracts and checks; no general semantic guarantee	Validate First; B for bounded claims, C for open purpose inference
Human review of selected explanations	Reviewers and time; appropriate for tests or sensitive cases	Resolves nuanced scope and ambiguity	Does not scale to every live event; reviewers also disagree	Evaluation/reference route, not default live dependency

Here B means likely feasible subject to target validation, and C means experimental for the specified claim class. No route has earned A from this analysis.

5.4.3 Selecting evidence without changing its meaning

Technical explanation. Start with structural retrieval: the selected project, session, run, call/attempt, artifact, and explicit relationship edges. Include paired start/result records, relevant requirements, and failure or correction records even when they are older than a recent time window. Preserve original evidence separately from selected and redacted views. Record the selector version and exactly which evidence was available to a generation request.

Plain-language explanation. The model needs the right packet, not necessarily the largest packet. A result from yesterday’s test can look relevant by filename while referring to the wrong code.

Three reasonable selection strategies have different conditions:

Selection method	Good use	Important limitation	Validation requirement
Identity joins and explicit dependencies	Known calls, task-linked results, handoffs, versions	Cannot discover relationships the source never recorded	Inject similar records from different runs; verify no cross-run substitution
Local keyword/full-text search with identity filters	User searches known files, commands, errors, history	Vocabulary mismatch and missing context	Annotate essential records for real questions, including negative evidence

Selection method	Good use	Important limitation	Validation requirement
Semantic retrieval or reranking after structural restriction	Paraphrased goals, long unstructured results	Similarity is not proof of identity, relevance, or entailment	Compare incremental evidence recall and contamination against simpler selection

A vector database is not required merely to call this process RAG. For modest histories, exact lookup and local search offer a simpler comparator. Embeddings become justified only if they improve the specific evidence-selection workload enough to offset indexing, privacy, versioning, and maintenance costs. This is an engineering recommendation; the present investigation did not benchmark search implementations.

Missing evidence should have reasons where known: source never exposes it, collection gap, delayed arrival, retention deletion, redaction, truncation, or selector omission. Do not label all these cases “the agent did not do it.” A claim about absence must state its observation boundary, such as “No matching verification result is present in the retained records received by cutoff c.” The host can establish this bounded query result; it cannot establish an unobserved global negative.

Contradictions require identity checks before conflict handling. “Failed on v1” and “passed on v2” may describe legitimate change. Opposite outcomes for the same immutable result identity require retaining the conflicting records, not selecting the newest phrasing. A prior summary must never become independent evidence for its own successor. The retained derivation should lead back to the underlying records. KB §§3.03.3, 3.03.6, 3.03.8.

There is direct evidence that locally supported facts can still make an incorrect combined narrative: Chiang and Lee examine aggregation across different people with the same name. The proposed extension to same-named files and repeated runs is a Yukio-specific inference, not their experiment. APP-EG-06; KB S-GE29.

5.4.4 Proposed claim contract and validation boundaries

The following is an **illustrative Proposed Solution**. Identifiers and events are fictional. It refines the proposal’s `SummaryEnvelope` rather than asserting that such a type exists in Yukio.

```
{
  "explanation_id": "EX-17",
  "request_context": {
    "project_id": "P1",
    "task_id": "U1",
    "run_id": "R1",
    "task_context_version": 3,
    "evidence_manifest_id": "M42",
    "received_through": "2026-10-18T10:00:10Z"
  },
  "claims": [{
    "claim_id": "C1",
    "text": "The test result reports that three selected checks passed on v1.",
    "basis_type": "tool_report",
    "subject": {"call_id": "T1", "attempt": 1},
    "artifact_version": "v1",
    "scope": "three selected checks",
    "evidence_refs": [{
      "record_id": "tests.p17/E2",
      "view_id": "EV2",
```

```
      "pointer": "/result"
    }],
    "limitations": ["This result does not verify the later version v2."]
  }],
  "unknowns": [{
    "question": "Has v2 been verified?",
    "reason": "No matching result in this evidence manifest."
  }]
}
```

The host should create or bind the request context, evidence manifest, and explanation identity. The model must not be allowed to approve its own scope by changing these fields. Suggested basis categories distinguish `observed_event`, `tool_report`, `agent_report`, and `inference`; independent verification requires its own recorded checker and criterion rather than an optimistic model label.

A validation sequence can be implemented incrementally:

1. **Format:** parse JSON; check required fields, types, sizes, enum values, and unsupported extras.
2. **Reference integrity:** resolve each reference against the authorized manifest and verify its pointer or excerpt against that saved view.
3. **Association:** compare project/run/call/attempt/artifact scope; reject references from other identities unless an explicit relationship is represented.
4. **Deterministic semantics:** check values, counts, result categories, and version relationships that have executable contracts.
5. **Open semantics:** evaluate whether the prose actually follows from the cited evidence; preserve insufficient evidence or disagreement.
6. **Presentation eligibility:** compare the request's context and evidence generation with the current view, keeping late answers attached to their original historical scope.

The first four steps can catch important errors, but an existing citation to the correct file can still fail to entail the sentence. A local schema library checks structure, not the truth of "all requirements are satisfied." Likewise, FEVER's distinction between supported, refuted, and insufficient-information judgments is a useful labeling model, not a ready-made execution-log validator. APP-EG-07; KB §§3.03.6-3.03.7.

A timestamp or a single highest event time is not a completeness guarantee across asynchronous sources. The proposed manifest should identify the selected records and source positions/cutoffs, including gaps, instead of treating a numeric "watermark" as proof that every earlier event arrived. If content is deleted, references should become explicitly unavailable rather than silently pointing to a different record.

5.5 Qwen provider and interface feasibility

5.5.1 Configuration is a tuple, not a model name

Record the actual provider, protocol, endpoint class, region, deployment scope, billing plan, workspace, model/snapshot, thinking setting, SDK or HTTP-client version, and feature configuration. Secrets should remain in an appropriate credential facility and be excluded from the evidence bundle. This tuple is currently **Unknown / Requires Validation**.

Alibaba's `qwen3.8-max` page, updated October 11, documents that model ID and the `qwen3.8-max-0902` snapshot/`qwen3.8-max-2026-09-02` alias. Its capability tables include function calling and structured output. These are **Documented Capabilities**; the marketing claims about long autonomous projects are not evidence of Yukio explanation quality. APP-QW-01.

The region documentation distinguishes the access/storage region from the geographical scope in which inference executes. It also distinguishes workspace-dedicated, shared DashScope, and trial endpoints; region-specific keys and model lists cannot simply be interchanged. The inspected page states that the shared DashScope domain stopped receiving new features after September 30, 2026. That statement does not say every existing shared-domain call has stopped working. APP-QW-03.

Examples of documented OpenAI-compatible base URLs are `https://{WorkspaceId}.cn-beijing.maas.aliyuncs.com/compatible-mode/v1` and `https://{WorkspaceId}.ap-southeast-1.maas.aliyuncs.com/compatible-mode/v1`; the braces denote configuration placeholders, not callable addresses for this project. The endpoint must match the actual region, workspace, and billing-plan credentials. A compatible protocol does not identify OpenAI as the inference provider. APP-QW-04.

5.5.2 Critical structured-output qualification

The current dedicated documentation distinguishes `json_object` from `json_schema` with `strict: true`. JSON Object requires a JSON instruction and does not constrain the expected fields. However, its support information is internally inconsistent: a summary table restricts Schema to selected Plus models, while the detailed list includes Qwen3.8-Max; it says Singapore Schema is unavailable yet immediately shows Singapore examples. It also documents multimodal fallback to JSON Object, qualified `oneOf` semantics, ignored external references, and warns that token caps can truncate JSON. Consequently, the exact text-only model/region/mode/schema combination needs a controlled test; neither universal support nor universal non-support follows from this page. APP-QW-02, Usage; Supported models; Getting structured output; Configuration guide; Going live.

Plain-language explanation. A model may promise to fill out a form, but the service used to reach it can change what "follow the form" means. Yukio needs to test its actual form with its actual account. A correctly filled form can still contain an unsupported statement.

5.5.3 Relevant documented behavior and engineering consequences

Interface issue	What was documented	Consequence proposed for Yukio
Chat Completions versus native API	Compatible Chat uses messages/choices; the native reference uses its own request/result organization. APP-QW-05; APP-QW-11	Implement one verified protocol first; keep transport translation separate from claim validation.
Tool calling	Qwen-specific restrictions qualify generic `required` and forced-tool settings; arguments need validation. APP-QW-05; APP-QW-11	Ordinary explanation needs no executable tools. A function-shaped response is not automatically strict output.
Streaming	SSE fragments must be assembled; optional usage appears at the end. APP-QW-06	Buffer and validate a complete explanation before presenting it as established; distinguish transport fragments from evidence.
Termination	Chat distinguishes `stop`, `length`, and `tool_calls`; non-streaming timeout behavior can return partial content. APP-QW-05	Inspect termination plus content, not HTTP success alone. An incomplete answer must not replace a valid historical explanation.
Authentication and failure	Error documentation separates parameter, authorization, quota/rate, and service failures. APP-QW-07	Preserve the provider error code/request ID; retry transient classes with a bounded policy, not all errors indiscriminately.
Model identity	Model and usage fields are documented; `system_fingerprint` is presently null in the compatible reference. APP-QW-05	Store requested and returned identifiers; do not infer a complete immutable model fingerprint.

For a native Swift or Python desktop application, direct HTTPS against a verified compatible endpoint and a supported vendor SDK are both plausible. Direct HTTP avoids requiring an additional language runtime but owns request encoding, SSE framing, and error parsing. An SDK reduces boilerplate in its supported language but adds version dependencies and may omit newer parameters. Existing Yukio compatibility is an inference from its observed languages, not a compiled integration result.

5.5.4 Provider alternatives

Provider route	Conditions, advantages, and compatibility	Limits, privacy, maintenance, and recommendation
Direct Alibaba Model Studio	Appropriate if the intended account is Alibaba; exact regional endpoint and eligible model are confirmed. One service contract and direct native/compatible choices.	Region, scope, plan, schema support, account permission, and measured latency remain unresolved. B for bounded integration, D for unavailable provider features. Validate First.
Provider route	**Conditions, advantages, and compatibility**	**Limits, privacy, maintenance, and recommendation**
OpenRouter serving an eligible Qwen endpoint	Its current catalog includes Qwen models, including a 3.8 Max snapshot. Could fit an existing OpenRouter account without pretending its configuration is DashScope. APP-QW-10	Adds a routing intermediary. Its documentation says endpoint support and strict enforcement vary; route filters and parameter requirements need verification. Do not assume a catalog entry guarantees the same schema behavior as Alibaba.

Provider route	Conditions, advantages, and compatibility	Limits, privacy, maintenance, and recommendation
		APP-QW-08; APP-QW-09 Validate First only if this is the actual or deliberately selected provider.
Another managed or self-hosted Qwen service	Conceptually possible if its independently documented deployment meets the required contract.	No concrete alternative account, deployment, hardware, or equivalent model established here. Investigate Further only when there is a real requirement; do not substitute a family name for evidence.

OpenRouter documents default provider routing and options to restrict providers or fallback behavior; its structured-output guide explicitly qualifies enforcement by endpoint. A useful validation configuration would identify allowed endpoints and require needed parameters, with fallback behavior deliberately chosen. This is a research comparison, not a recommendation to migrate services. Region and data-handling claims would require the actual service contract and configuration, not a generic “cloud is secure” assurance. APP-QW-08; APP-QW-09.

5.5.5 Runtime behavior, cost and privacy

Technical explanation. Keep collection and local state projection independent of explanation requests. An event burst should not create an unbounded request queue. Compare a simple request-on-demand route with coalesced updates at meaningful activity changes. Give every request an immutable evidence/context generation, allow historical completion, and reject stale overwrites of the current view.

Plain-language explanation. Yukio can show that a test has failed immediately from a result record. It can add a clearer explanation later. A slow explanation of yesterday’s state must not replace today’s state.

A cache key should include the relevant task context, evidence content/manifest, selector and redaction versions, model configuration, prompt, schema, and output policy. Cache invalidation is about meaning as well as tokens: a new requirement can make an old summary insufficient even if the activity records did not change. Coalescing should merge work awaiting explanation without discarding the historical failure evidence.

No response time, token price, monthly cost, or accuracy is estimated here. For a confirmed provider, measure completed and failed requests and calculate total cost from actual billable categories:

$$C = \sum_j (I_j p_{in,j} + O_j p_{out,j} + K_j p_{cache,j} + F_j),$$

where input, output, and cache quantities are **disjoint billable categories**, prices use matching units, and additional charges are represented by F. Include retries and validator calls. If the provider reports reasoning as part of output, do not count it again. Missing terminal usage after a dropped stream means unknown local accounting, not a zero-cost call. Record queue delay, first response delay, complete validated explanation latency, and time spent showing an obsolete explanation separately. Provider streaming documentation also makes cancellation billing depend on when the server receives termination; canceling locally is not proof of no charge. APP-QW-06.

The inspected local capture/caption path and the proposed cloud explanation introduce different data flows; no complete runtime egress audit was performed. A claim that the upgraded product never uploads conversation material would need revision if excerpts are sent. Proposed controls include project-level authorization, a previewable description of transmitted categories, excluded paths/fields, payload limits, credential separation, and documented retention/deletion. Redaction must retain an explicit transformation boundary, because the model cannot ground a claim in a field that was removed.

Untrusted logs, command output, code comments, and search excerpts can contain instructions addressed to the observer. AgentDojo demonstrates tool-data injection in agent environments and evaluates utility and attacks separately; applying that threat model to Yukio's explanation input is a justified engineering hypothesis, not an observed Yukio exploit. APP-EG-05; KB §3.03.9, S-GE12.

Treat these inputs as data, retain clear boundaries, avoid giving the explanation service command execution or authorization powers, and validate reference targets locally. Prompt wording alone is not a demonstrated complete defense. An output-only model can still harm the product by concealing failure, inventing success, or producing a misleading alert. Test those explanation-specific outcomes, including injection into evidence supplied to an automated judge. Local evidence links should be resolved by the host, not arbitrary URLs or file-opening instructions invented by the model.

5.6 Human understanding and visualization

5.6.1 Compare two credible presentation routes

The first design question is what the returning user needs to establish: which task is active, what changed, which result belongs to the current version, and whether a question still needs an answer. Situation awareness provides a useful perception-comprehension-projection framework, but does not prove that adding a dashboard improves those abilities. Professional KB §§3.04.1.1, 3.04.6; S-HI01

Decision dimension	Compact pet plus linked activity workbench	One organized conversation/timeline view
Mechanism	A small projection signals relevant activity; a linked view supplies task, result, history and evidence context.	An integrated transcript combines messages, tool events and explanations, with filters and anchored evidence.
Required data	Shared event identity, consistent status semantics, selected task identity and navigation to the exact record.	Stable message/event identity, intelligible grouping, temporal semantics and retained original content.
Strength	Fits the stated desktop-assistant goal and permits intermittent checking; details need not occupy the desktop continuously.	Keeps conversational context nearby and reduces switching between surfaces; a useful baseline for context-heavy questions.
Limitation	A reassuring animation can conceal unknowns; users may never open the workbench; two surfaces can disagree.	Long histories can bury results, while grouping can conceal concurrency or the difference between a request and its outcome.

Decision dimension	Compact pet plus linked activity workbench	One organized conversation/timeline view
Complexity and compatibility	Moderate: retain the pet presentation where practical, add a data-backed view, and synchronize selection and revision state. Existing window code is only a partial starting point.	Low to moderate for an event list; substantially more if a complete conversation renderer or cross-client chat replacement is attempted. No such integration is confirmed.
Privacy and performance	Compact summaries reduce incidental exposure, but the detailed page still needs access and retention controls. Frequent updates need batching.	Greater default exposure of conversation content; large histories need bounded rendering. A text view avoids some graphical complexity.
Conditional preference	**Validate First.** Strong fit to the project goal; benefit must be shown on comprehension tasks.	**Validate First as comparator.** Prefer it if the additional workbench surface produces no useful gain or makes context harder to recover.

These are engineering comparisons, not an empirical ranking. A table with expandable results can be a sufficient initial workbench. A separate node graph is justified only by questions that require reliable dependency edges; adding one merely to depict “multi-agent intelligence” would add unsupported meaning. Professional KB §3.04.4; PROJECT-PLAN, pp.6, 8

Technical explanation. Both views should project the same committed facts. A displayed item needs an object, scope, evidence basis, update time and reconstruction version. Animation smoothing may stabilize presentation, but should not remove history or become a source of task truth. A task tree represents decomposition only when a plan and parent relation are observed; a timeline represents time, not proof of causation.

Plain-language explanation. The pet can help you notice that something deserves a look. The activity page should help you answer a specific question and check its evidence. Whether that division is actually easier than one well-organized history must be tested with users.

5.6.2 Preserve meaning in the default view

The proposal’s current-work, progress/result, timeline and evidence sections are reasonable candidates. Four refinements are necessary:

1. **Show execution, verification and observation freshness separately.** “The run ended,” “three selected checks passed on version v1,” and “the latest observed file is v2” can all be true. Neither the pet nor a large green card should merge them into “verified complete.” If current-version evidence is unavailable, say so without asserting that the file is incorrect.
2. **Make historical knowledge distinguishable from current knowledge.** A late result can update the reconstructed history while preserving what Yukio had received at the earlier time. Label an old explanation with its evidence cutoff; do not silently rewrite yesterday’s explanation as though it knew a later result.
3. **Keep decision-changing qualifications visible.** Evidence can be expandable, but “result applies to an earlier version,” “receiving agent’s input unavailable,” and “source disconnected” should not disappear inside a generic details panel when they change the displayed conclusion. This visibility rule is a proposed semantic safeguard, not a proven optimal layout.

4. **Give every count a named denominator.** “3 of 5 items marked complete in plan revision 2” is permissible when those data exist. It measures that plan’s recorded labels. It does not measure project work, elapsed effort or acceptance. A phase label without plan data also needs an explicit rule and evidence; absence of a plan does not license guessing a phase. Professional KB §§3.04.4.1-3.04.4.2; Textbook Chapters 19-20

For multi-agent work, compare a sender-side assignment, observed receiving-side input, a returned result and a later explicit use as separate relations. A collapsed handoff card should not turn “recipient input unavailable” into either “all requirements arrived” or “requirements were lost.” An expandable list is preferable to inferred network edges until those relations are established.

5.6.3 Research prevents three premature product claims

Amershi et al.’s guidelines support reviewing capability communication, recovery, dismissal and control. Their practitioner evaluation assessed guideline relevance and clarity; it did not establish that a compliant Yukio screen improves novice understanding. APP-HE-01

Poursabzi-Sangdeh et al.’s preregistered apartment-estimation experiments found that transparency could improve model simulation without consistently improving correction of large mistakes. Some conditions changed the latter effect. Information overload was an interpretation, not a demonstrated universal mechanism. Consequently, neither “more detail” nor “less detail” is an evidence-based default for all Yukio questions. APP-HE-02

Buçinca et al.’s nutrition-choice experiment found reductions in some overreliance outcomes from cognitive forcing, with effort and preference tradeoffs; overall joint performance did not uniformly improve. Requiring a confirmation click on every Yukio summary is therefore unjustified. Reserve deliberate checks for decisions whose benefit can be demonstrated. APP-HE-03

5.7 Human intervention: requests, issues and useful actions

5.7.1 A constrained present scope

Explicit request relay is a plausible near-term capability when an identified request is observable. It is a different problem from deciding that an agent ought to have asked. Preserve the source request, scope, options, expiry and lifecycle. A quoted question in ordinary text is not automatically a pending request; a permission request is not evidence that the action was executed.

The preferred initial alternatives are **passive request display with navigation to the original client**, and **an in-place reply through a tested, specifically bound channel**. The first has weaker convenience but fewer connection assumptions. The second reduces context switching but requires request revalidation, target identity, expiry checks, acknowledgement semantics and handling of ambiguous delivery. Reuse the existing gates where appropriate; validate their behavior and all fallback branches. Do not extend a successful connected-route test to arbitrary desktop clients. APP-HE-15; PROJECT-PLAN, pp.2, 8

A useful request state sequence is *observed → presented → user response submitted → transport receipt observed → external request resolution observed*. Some channels provide only a subset.

The unresolved states must remain expressible. A receipt from a local bridge does not establish successful downstream work; an absent receipt does not prove the answer was never sent.

Plain-language explanation. Yukio can first help you find and answer questions that actually exist. Before letting it answer inside its own window, test that the answer reaches the correct still-open question and that a lost receipt cannot cause repeated submissions.

5.7.2 Compare notification policies without assuming universal detection

Candidate	Inputs and benefit	Main limitation, cost and decision
Explicit request plus narrow contract rules	Pending question/permission records or an independently defined violation. Easy to explain and audit.	Requires version-specific source semantics; a failure can already be recovering. **Validate First**, with passive history as the baseline.
Fixed digest or user-initiated review	Current unresolved episodes and user-chosen timing. Low policy complexity and predictable exposure.	May miss a short useful action window; still needs accurate episode state. **Validate First** for nonurgent information.
Statistical or LLM-based anomaly review	Duration/frequency baselines or goal-linked evidence. Can propose candidates beyond explicit requests.	Needs representative data, stable targets and independent labels; rarity, hesitation words and plan divergence do not establish actionable error. **Investigate Further**.
Learned deferral or expected-utility policy	Human response, alternative actions, timing and outcome-cost data. Can compare when asking is useful.	These inputs are absent; open tasks change goals and human availability. **Consider Later** for a narrowly defined decision family.

Horvitz’s mixed-initiative framework supports comparing dialogue, action and inaction under uncertainty and cost. Applying it here is a **Proposed Solution**, not a deployed Yukio policy. The engineering question is whether a feasible user response can improve the outcome enough to justify attention and delay. Low confidence alone cannot answer that question. APP-HE-04; Professional KB §§3.06.1-3.06.3

Define an **issue episode**, a responsible actor and a useful response window before counting notifications. A failed test, repeated error lines, a retry and recovery may describe one episode. An anomaly can be benign; a real error can recover; an unresolved preference can warrant a question despite no error. Proposed alerts should identify the available action and relevant evidence. They must not invent a pause or correction control. APP-HE-05; Professional KB §§3.06.2, 3.06.4

Interruption timing requires a cost model, not a universal delay constant. The KB’s breakpoint studies motivate comparing immediate delivery with deferred delivery for nonurgent messages; they do not establish a best delay for Yukio. Mark et al.’s experiment also found faster measured primary work alongside higher stress, with interruption handling time excluded from that primary-work measure. Measuring speed alone would miss a material cost. Professional KB §3.06.5, S-HI18/S-HI19; APP-HE-08

Dismissal must not become a training label for “bad alert.” Ancker et al.’s clinical-log study associated repetition with some lower acceptance outcomes, but did not establish a general habituation pattern or adjudicate each alert’s appropriateness. Users may ignore a duplicate

rationally. Record repetition, recovery, exposure, response and stated reasons separately; do not import clinical effect sizes into desktop development. APP-HE-09

5.8 Evaluation: measure the observer

5.8.1 Three references and two kinds of correctness

An agent can fail its task while Yukio correctly explains the failure. An agent can succeed while Yukio associates its test with the wrong version. Agent success rate is therefore not an observer accuracy metric. Professional KB §§3.07.1-3.07.2; Textbook Chapter 23

Use three independently specified reference layers: **world/task outcomes**, **what the configured source was expected to emit**, and **what Yukio captured and represented**. An external fixture can verify a test outcome, a source-side copy can expose collector loss, and manual review can evaluate claim support. None alone proves all three layers. Two exports produced by the same defective parser do not provide independent corroboration.

For decision-time evaluation, maintain both the eventual world result and the strongest claims warranted by records received at the cutoff. A process may truly be finished while “outcome not observed” remains Yukio’s correct evidence-grounded statement. Penalize unsupported certainty separately from an inevitable lack of information.

5.8.2 Candidate metrics with legitimate denominators

Layer	Candidate measure	Definition and material boundary
Source capability	Supported activity/field categories and known exclusions	A capability inventory; not a recall percentage over every real action.
Collection	Matched distinct in-scope emissions / independent reference emissions	Recall applies only to the declared source contract and test interval.
Collection precision	Matched distinct events / emitted candidate events	Deduplicate legitimate redelivery under a declared identity rule; assess delivery preservation separately.
Association	Correct call/attempt pairs and parent/return edges	Unmatched and ambiguously associated records remain visible. Timestamp proximity is not reference truth.
State	Per-state confusion, transition delay, unsupported terminal claims and wrong-state duration	Report unknowns and source availability separately; long easy idle intervals must not hide missed failures.
Explanation	Claim support, contradiction, insufficient-evidence rate; citation validity; critical omission	Correct IDs do not establish semantic support; version, time, object and scope are separate error classes.
User value	Correct evidence-based judgments, evidence-location success, time and appropriate reliance	Preference, satisfaction and click counts remain secondary outcomes.

Layer	Candidate measure	Definition and material boundary
Notifications	Useful episodes detected in time, false alerts per exposure hour, redundant notices per episode	Count actionability windows rather than every error line; natural prevalence matters.
Runtime	Availability, capture gaps, latency components, queue size, dropped updates, recovery and resource cost	Include failed/unfinished observations; record hardware, load, client and adapter versions.

Before event precision/recall, construct permitted matches using identity, event kind, attempt and declared timing tolerances. Enforce **one-to-one matching**; if identity is incomplete, a deterministic bipartite matching procedure with declared tie handling is one candidate. Preserve unmatched/ambiguous cases. Ten duplicates must not earn ten hits; a genuine retry must not disappear as a duplicate. The matching algorithm is an evaluation design choice, not a standardized cross-client truth definition. Professional KB §3.07.3

For state intervals, point accuracy, interval overlap and transition latency answer different questions. Tatbul et al. provide adjustable range-based criteria including coverage and fragmentation. Yukio should not adopt their most generous setting automatically or reward one brief hit as complete detection of a long interval without disclosure. APP-HE-07

5.8.3 Replay, explanation assessment and uncertainty

Compare **fixed-trace replay** with **new controlled client runs**. Replay is inexpensive and repeatable for parsing, state and evidence regressions; real runs validate whether the source still behaves as assumed. Neither replaces the other. Inject duplicate deliveries, missing terminal events, delayed outcomes, partial records, source restarts and two simultaneous tasks into declared fixtures. These are constructed fault conditions, not estimates of their natural prevalence. Professional KB §3.07.8; Textbook Chapter 25

Replay must deliver records in receipt order up to each decision cutoff. Later events may inform the reference answer but must not enter the observer’s earlier input. Include cached summaries, background retrieval and prior-run state in this isolation: clearing a visible timeline while retaining answer-bearing cache data is insufficient. Save historical assertions and later corrections separately. A late correction does not prove the earlier uncertainty was an error.

For explanation comparison, use the same evidence snapshots for **deterministic templates/extraction** and **LLM or hybrid generation**. Label atomic claims as supported, contradicted or insufficiently supported, with attributed agent reports separate from observations. Score required fact coverage against a reference inventory; otherwise a system can win by saying almost nothing. Evaluate the produced summary with its actual retrieved evidence, and separately supply a complete permitted evidence set to isolate retrieval failure from generation failure.

Use deterministic checks for IDs and exact fields, then independent human review for semantics. An LLM judge can assist triage only after a held-out audit of its own version, negation, scope and citation errors. The KB’s MT-Bench and contextual-judge evidence does not make human preference agreement equivalent to factual correctness. Annotator agreement itself measures consistency and cannot rule out shared mistakes. Professional KB §§3.07.6-3.07.7; S-GE16/S-GE19; APP-HE-11

Report **selective risk and coverage together**: how often Yukio answers and how often its answered claims are wrong. “Unknown” should be justified and useful, not an escape from all difficult cases. If probabilities are later shown, name the target and evaluate held-out calibration, proper scores and relevant subgroups. Guo et al.’s classification calibration evidence does not validate free-text model confidence or an overall “Yukio reliability percentage.” Professional KB §3.07.5, S-GE21/S-GE31; APP-HE-10

5.8.4 Interpret the proposal’s two numerical targets correctly

The proposal’s **approximately 20 scenario classes** are a sound starting inventory for executable requirements. They are not 20 independent samples from daily use and cannot support a general accuracy estimate. Vary the consequential factors-client/version, platform, source, concurrency, truncation, restart and user interaction-and document which combinations remain untested. PROJECT-PLAN, p.11

The suggested **95% within one second** concerns ingress-to-local-card latency under an agreed load. Retain it as a **candidate service objective**, not a result. Define ingress, durable commit, projection availability and visible rendering timestamps. Measure upstream emission delay separately when comparable clocks exist, and report cloud explanation latency separately. An event never rendered has no completed latency observation; it must be counted as missing or not rendered by the cutoff, not silently excluded. Compute the within-deadline fraction over all eligible inputs alongside latency quantiles among completed renders. Missing start/end timestamps remain missing. PROJECT-PLAN, pp.7, 11

Use paired comparisons on the same tasks where appropriate. Events within one run and answers from one user are dependent; preserve that structure in uncertainty estimates. Split development and evaluation by complete task/run, and by repository or user where the intended generalization requires it. A second client release is a new compatibility condition, not something event-level resampling can simulate. Professional KB §3.07.7; Textbook Chapters 25-26

5.9 Privacy and security across the observation path

The proposed Qwen service changes the data path from local observation to selected external disclosure. Distinguish **what is collected**, **what is retained**, **what the model receives**, **what the UI reveals**, and **what an export contains**. A setting controlling cloud use does not automatically govern local history. No provider retention, region or account-policy property is inferred here from a model name. PROJECT-PLAN, pp.7, 9

Compare **local retained evidence plus minimized cloud excerpts** with **metadata-only retention plus on-demand authorized evidence access**. The first supports better replay and historical support checks but retains more sensitive material. The second reduces stored content but cannot guarantee later reconstruction after files change or logs disappear. The appropriate choice depends on project sensitivity and required history; neither route justifies uploading all chats by default.

OWASP’s logging guidance addresses untrusted sources, sensitive fields, restricted access and log protection. Its general advice must be adapted: Yukio’s purpose may require deliberately selected code evidence, unlike a generic security log. Keep excluded or transformed fields and unavailable evidence explicit. A checksum supports identity/integrity checks under stated conditions, not truth or anonymization. APP-HE-13; Professional KB §§3.01, 3.03

External files and tool results can contain prompt injection. An explanation-only model with no action tools still can distort summaries, hide failed checks or invent evidence references. Enforce authority and output boundaries in program logic; distinguish data from instructions; render inert text; validate links and payload sizes. Disabling action tools reduces one harm path but is not proof that explanations resist manipulation. APP-HE-12

A deletion design must address event payloads, summaries, indexes, exported copies under Yukio’s control and later re-import. State what can and cannot be removed, rather than promising erasure from systems Yukio does not control. Synthetic secret canaries and hostile evidence fixtures can test defined boundaries without uploading private working material. These are proposed tests, not a claim that every sensitive value can be detected.

6. Comparative Analysis

The following comparisons synthesize the detailed methods in §§4-5. They are **Proposed Solutions** unless a narrower evidence status is stated. Complexity and cost assessments are qualitative engineering judgments for the inspected desktop structure, not measured estimates.

6.1 Observation routes: existing work versus cooperating execution - DEC-02/03

Technical comparison. A native transcript parser reuses current source discovery and avoids another runtime, but Yukio owns internal-format compatibility, partial-record handling and retention gaps. A supported historical reader can transfer some format maintenance to a vendor library; compare its returned fields, filtering, child records and concurrent-write behavior before accepting the dependency. Claude documents such a reader; this is separate from starting an SDK-owned agent. CODE-SOURCES; APP-CC-07

Hooks add named checkpoints but require configuration, trust and interference tests. Synchronous handlers place work on the execution path; background handlers introduce ordering and teardown risks. Telemetry supplies useful signals under content/configuration gates, with buffering and possible truncation. Adding channels also creates cross-source deduplication and privacy obligations. APP-CC-01; APP-CC-02; APP-CC-06; APP-CX-03; APP-CX-05

Managed CLI/SDK/App Server routes offer deliberate process or connection ownership. They can be preferable if users choose to launch work through Yukio; they impose packaging, lifecycle and pending-request responsibilities. They do not establish passive attachment to an arbitrary open desktop session. APP-CC-03; APP-CC-05; APP-CX-01; APP-CX-02; APP-CX-04

Decision and reversal condition. Preserve native parsing as a bounded comparator; prefer a supported reader when it retains required evidence at acceptable runtime cost. Add hooks or telemetry for a measured gap. Reconsider managed execution only when its workflow is explicitly wanted or necessary for the required evidence. In everyday terms, a better channel is useful only if it observes the user’s actual work without silently changing what “observe” means.

6.2 Identity and multi-agent relationships - DEC-01/04/11

Technical comparison. Explicit source-scoped call, attempt and parent identifiers support auditable joins. A relationship table can represent delegation, observed receipt, return and use without forcing a tree. Timestamp/text similarity can discover candidate links where IDs are missing; it can also merge retries, simultaneous work or identically named files. Keep these candidates reviewable and reversible. The constructed failure probes make arrival-based pairing a concrete risk in the current parser. RUN-01; RUN-02; RUN-03; APP-ENG-06; APP-ENG-08; APP-CC-10; APP-CX-01; APP-CX-06

Explicit joins are inexpensive and compatible with local storage, but adapter semantics require maintenance. Semantic association adds model or index cost and may disclose more content. Neither route can manufacture receiver-side evidence. Recording “sent” cannot establish that every requirement arrived or influenced the return. APP-EG-01

Decision and reversal condition. Adopt provenance and identity separation as requirements; validate explicit linkage first. Promote a heuristic only for a named relation after independently labeled tests show useful coverage without unacceptable wrong links. Missing links are a coverage problem; they are not permission to fill the factual history with plausible relationships. Broader graph visualization should follow reliable relationships, not precede them.

6.3 Durable history and recovery - DEC-05/06

Technical comparison. SQLite can commit observations, indexes and cursors in one transaction, support constraints and query historical projections. It adds schema migration, contention and retention responsibilities. Append-only JSONL segments with a rebuildable index remain credible for modest histories and inspectable exports, but coordinated cursors, partial writes, compaction and deletion become application logic. Current JSONL familiarity favors the second route's initial reuse; richer historical queries favor evaluating the first. CODE-SOURCES; APP-ENG-01; APP-ENG-03; APP-ENG-04; APP-ENG-07

SQLite WAL improves some reader/writer concurrency but still has one writer, checkpoint duties and same-host constraints. Rollback journaling may suffice for lower contention. The actually linked library's relevant fixes must be checked rather than inferred from the OS or language version. Evidence blobs outside the database require their own crash-consistency protocol. Neither database choice implies exactly-once observation. APP-ENG-02

Polling is a straightforward recovery baseline. Native file notifications can reduce scanning, but should trigger reconciliation; notifications may overflow and file identity can change. Equal-length replacement defeated the tested size-based reader. File-generation identity, complete-record boundaries and durable checkpoints matter under either storage choice. RUN-04; RUN-05; RUN-06; APP-ENG-09

Decision and reversal condition. Validate SQLite against segmented JSONL using the same restart, replacement, disk-pressure and query fixtures. Prefer the simpler route if it satisfies bounded history requirements. Both retain sensitive evidence and consume disk; choose explicit retention, not indefinite accumulation. Add external infrastructure only when demonstrated workload or sharing needs exceed the local approach.

6.4 State and progress - DEC-07

Technical comparison. Deterministic projections apply source-specific rules to separately represent execution, operation outcome, agent claims, artifact checks, acceptance and observation health. They are inspectable, inexpensive and compatible with existing routing concepts; existing animation timing should remain a presentation policy. A statistical hidden-state model can combine uncertain observations only after its hidden targets, transition assumptions, missingness and calibration are defined. LLM state assignment offers flexible language interpretation but does not validate unseen execution. CODE-ROUTER; KB §§3.02.6-8, 3.02.90-91

Keep occurrence, receipt and interpretation time distinct. Replay with the same evidence and rule versions can be deterministic without arbitrary order independence. A revised plan requires valid source ordering, and a delayed test still belongs to the artifact version it checked. APP-ENG-05; APP-ENG-08; TEXTBOOK Ch.13/15

Decision and reversal condition. Validate explicit rules first. Consider probabilistic inference only if a specific, consequential unknown can be measured against independent labels. Display a plan-revision count when its denominator exists; otherwise show observed milestones or unknown scope. Plainly, a finished process, a passed check and accepted work answer different questions. No single reassuring status should erase those differences.

6.5 Evidence selection and explanation - DEC-08

Technical comparison. Deterministic templates and excerpts offer low latency, offline availability and direct field binding, at the cost of limited narrative synthesis. Free-form retrieval-augmented generation can connect larger contexts, but retrieval errors, omitted qualifiers and unsupported associations can yield convincing mistakes. A hybrid keeps deterministic facts and references while allowing a model to propose structured claims or explanatory language. Local validation must still check identity, snapshot, artifact scope and semantic support. JSON validity only addresses output shape. APP-EG-02; APP-EG-03; APP-EG-04; APP-EG-06; APP-EG-07

Exact ID joins and scoped full-text search are suitable initial selection methods. Semantic retrieval is a useful comparator for diffuse context after task/time/version filtering; it adds indexing and content exposure, and cannot correct a foreign-run join. Generated citation completeness also differs from coverage of important facts absent from the summary. APP-EG-03; APP-EG-04

Decision and reversal condition. Retain templates as the baseline and fallback. Validate hybrid generation on identical evidence, with independent support and comprehension labels. Prefer templates where LLM output adds no meaningful understanding benefit or introduces excessive error, delay or disclosure. Keep model calls outside the critical observation path; compare on-demand explanation with coalesced updates using actual user needs and workload. A reader should always be able to ask what observation supports a sentence.

6.6 Qwen service and output protocol - DEC-09

Technical comparison. A direct Alibaba service and an eligible intermediary such as OpenRouter are different provider choices, even when both offer a named Qwen model. Direct access follows Alibaba’s regional workspace, endpoint and account constraints; an intermediary adds routing, fallback and additional data-handling boundaries. An existing account may favor either route. Catalog availability is not account entitlement or equivalent structured-output enforcement. APP-QW-01; APP-QW-03; APP-QW-04; APP-QW-08; APP-QW-09; APP-QW-10

Within the actual provider, ordinary text, JSON Object, strict JSON Schema and tool-shaped output have different guarantees. Alibaba’s schema guide contradicts itself on some model/region support. Compatible and native APIs also have different envelopes. Streaming can expose progress earlier but requires fragment assembly, termination checks and accounting for missing final usage. Tool calling is unnecessary for an explanation-only model. APP-QW-02; APP-QW-05; APP-QW-06; APP-QW-11

Decision and reversal condition. Resolve the real provider/region/workspace/model snapshot/protocol/schema tuple before selecting an adapter. Test small valid, missing-field and interrupted responses. Retain local validation even if strict generation works. Compare cost using

disjoint billable categories, retries and validators; no account-specific price or latency is established here. If the service cannot enforce the needed shape, evaluate a simpler supported schema with local rejection or retain templates; do not silently downgrade the factual contract.

6.7 Presentation and user participation - DEC-10/12/13

Technical comparison. A compact pet plus linked activity workbench fits the project goal and supports intermittent checking; two surfaces require consistent status, selection and freshness. One organized history keeps conversation context together and avoids some switching, but can bury important results. Both require stable evidence anchors and bounded rendering. Existing window code is a reuse opportunity, not an activated activity view. Satisfaction and attractiveness cannot decide between them; use comprehension and evidence-location tasks. CODE-UI; APP-HE-01; APP-HE-02; APP-HE-03; APP-HE-14

For participation, passive display with navigation to the original request needs less connection authority than an in-place answer. The latter reduces switching but needs exact target binding, expiry and receipt semantics. Existing route-specific code includes different dispatch behavior and untested branches. APP-HE-15

Explicit requests or a user-chosen digest are plausible initial notification policies. Anomaly scoring may suggest issues beyond requests, but needs actionability labels; a learned utility policy additionally needs human response and interruption-cost evidence. Rare activity, an error and a useful action window are not interchangeable. APP-HE-04; APP-HE-05; APP-HE-08; APP-HE-09

Decision and reversal condition. Validate the workbench against a credible history baseline. Prefer original-client navigation until a particular reply path proves safe target binding and handles ambiguous delivery. Promote inferred reminders only if they improve actionable outcomes enough to justify false alerts and interruptions; more clicks alone do not satisfy that condition.

6.8 Evaluation, data policy and infrastructure - DEC-14/15/16

Technical comparison. Replay makes parsing, timing and support regressions reproducible; fresh controlled client runs test the real emission contract. Each has a distinct reference layer. Human semantic review is slower than an LLM judge but can independently inspect version/scope errors; an automated judge is an assistant only after its own held-out evaluation. Preserve useful-fact coverage alongside support, and unknowns alongside error rates. KB §§3.07.1-8; APP-HE-07; APP-HE-10; APP-HE-11

Retained local evidence plus minimized cloud excerpts enables stronger historical review while storing sensitive content. Metadata-only retention reduces content exposure but loses future explanatory ability when sources change or disappear. Both need explicit UI/export/deletion boundaries. Hostile records remain untrusted inputs even when the model cannot execute tools. APP-HE-12; APP-HE-13; APP-EG-05

Decision and reversal condition. Adopt independent references, replay isolation and explicit data policy as requirements. Validate storage and cloud boundaries with canaries and faults. Process mining, vector indexes, graph services or remote telemetry backends become candidates

only when stable data and a concrete unresolved query justify their maintenance, latency and disclosure costs. Their sophistication cannot repair missing observations.

7. Feasibility Assessment

Ratings name a capability and condition; they are not probabilities or implementation labels.

Grade	Application to Yukio	Evidence required to change the assessment
A - Confirmed Feasible	Reserved for a specified implementation and tested environment. No proposed end-to-end upgrade receives A. RUN-01, RUN-05 and RUN-06 establish only their exercised ordered-pairing, buffering and reset behaviors; RUN-02; RUN-03; RUN-04 establish counterexamples.	Native deployment evidence covering the capability's actual source, workload, failure boundary and result.
B - Likely Feasible, Requires Validation	Bounded client adapters, local retained history, explicit association, deterministic state, template explanations, artifact-scoped checks and reliable references have suitable engineering foundations. Construction of an activity view also fits here.	Version-matched capture and recovery tests; native platform checks; independent state/support labels; accepted workload and privacy constraints.
C - Experimental / Research-Dependent	Comparative understanding benefit, flexible narrative synthesis, semantic handoff inference and useful anomaly-based reminders need dedicated evidence. Detailed beta dependencies also remain conditional.	Narrow targets, representative examples, independent outcome labels and comparisons showing value beyond simpler baselines.
D - Limited by External Systems	Arbitrary-session attachment, omitted/ephemeral history, hidden receiver input, withheld content, unavailable account features and unsupported reply paths depend on third parties.	A documented and exercised permitted route that supplies the required evidence or interaction. Better local inference alone cannot remove this limitation.
E - Unsupported or Infeasible Under Current Constraints	Complete hidden reasoning, universal correctness/off-task detection, guaranteed full project progress and reliable factual reconstruction of never-recorded events lack a defensible current path.	Materially different scope or independent evidence; persuasive model output is insufficient.

Two distinctions affect investment. **A buildable representation may still have experimental user value:** the workbench is B to construct and C as a claim of improved understanding. **A useful observer may remain incomplete:** B for accurately reporting visible calls can coexist with D for hidden search activity or recipient context. The correct product promise names that boundary. APP-HE-01; APP-HE-02; APP-HE-03; APP-CC-01; APP-CC-04; APP-CX-01; APP-CX-02; APP-CX-03

The current code is neither a blank slate nor a verified complete upgrade. Reuse should follow responsibility: parsers and routing contain useful compatibility knowledge; `PetEvent` and UI

timers should retain presentation roles unless strengthened and validated for another purpose. CODE-EVENTS; CODE-PARSERS; CODE-ROUTER

8. Critical Unknowns and Technical Risks

These are ranked by the decisions they can overturn, not by invented occurrence probabilities.

Priority / unknown	Why it matters	Required evidence or response
P0 - Actual deployment identity	Unknown Yukio revision, OS build, Claude/Codex/Deep Code release, local/SSH/WSL/cloud surface and permissions prevent a support claim. README 0.4.6 is not the installed version.	Record non-secret executable/package versions, schema identity and effective configuration for the selected surface.
P0 - Capture contract and loss	Different operations bypass different channels. Sidechains, content reduction, retention and display gates may remove needed evidence.	Independent known-action fixtures and source-emission references; preserve explicit omissions and unsupported records.
P0 - Call and file identity	Constructed probes show wrong failure association and missed same-length replacement. Native incidence and platform behavior remain unknown.	Release-matched concurrency evidence; native replacement/restart tests; call-scoped state and file-generation contracts.
P0 - Qwen configuration and schema support	No actual provider, region, workspace, model snapshot, entitlement, SDK/protocol or schema mode is supplied; documentation conflicts.	Resolve the tuple and perform a minimal authorized contract test before choosing the adapter.
P1 - Artifact/goal version binding	A real test can support an obsolete file; revised plans change denominators. Plausible summaries can mix correct facts incorrectly.	Retain scope/version evidence; reject unsupported cross-version conclusions.
P1 - Cloud and retention policy	Local collection, cloud transmission, display and export expose different content. Redaction can also destroy useful support.	Explicit data-flow decisions; synthetic canary and deletion/re-import checks.
P1 - Native UI and reply behavior	Existing source paths do not prove reachable, correctly bound interactions. Hidden UI may affect some request polling.	Native entry-point and target/receipt tests, including expired requests and ambiguous delivery.
P1 - Performance envelope	Unknown concurrency, history volume and hardware prevent cost or responsiveness claims. Cloud delay can make an explanation stale.	Separate upstream, ingestion, commit, projection, rendering and generation timings; include unfinished observations.
P2 - User benefit and actionability	More detail may increase burden; unusual execution may require no action.	Intended-user comprehension studies and episode-level intervention references before optimizing notifications.

Source support: CODE-BASE; CODE-SOURCES; CODE-PARSERS; RUN-RECORD; APP-QW-02; APP-QW-03; APP-QW-04; APP-QW-05; APP-QW-06; APP-QW-07; APP-EG-06; APP-HE-02; APP-HE-03; APP-HE-14; APP-HE-15.

The proposal's "approximately 20 scenarios" is a candidate coverage inventory, not a representative sample size. "95% within one second" is a proposed ingress-to-local-card service objective under specified load, not an observed result or a guarantee about upstream/cloud delay. Preserving those distinctions prevents a test plan from turning into an unsupported performance claim. PROJECT-PLAN, pp.7, 11; KB §§3.07.3, 3.07.7

9. Validation and Experimental Plan

9.1 Status, purpose and common test contract

Status: PLANNED. Every validation card in this section describes future work. None reports an observed outcome. The six executed Python fixture probes, RUN-01; RUN-02; RUN-03; RUN-04; RUN-05; RUN-06, are reported separately in §2.5 and RUN-RECORD. Their parser and file-reader counterexamples motivate additional checks; they neither execute these plans nor establish real-client failure frequencies. This report does not require running the proposed experiments.

The objective is to resolve a specific engineering uncertainty with the smallest repeatable test that can falsify the relevant assumption. Start with constructed fixtures, then use a disposable project and explicitly authorized real-source records when that environment becomes available. Test only routes under consideration. An unsupported optional interface is a documented boundary, not a reason to build every alternative.

Each test record should preserve its question, fixture provenance, independent reference, exact Yukio/client/adapter versions, operating system, execution surface, effective non-secret settings, input schedule, expected contract, actual output and interpretation. Record provider/endpoint/model configuration for model experiments, and storage-library versions for persistence experiments. Synthetic records must remain labeled synthetic. Keep failures, unavailable cases and unresolved annotations in the record. KB §§3.07.2-3.07.8

A passing card permits only its stated judgment. It does not automatically produce an A feasibility rating for an entire feature. Expected observations below are hypotheses, not acceptance evidence already obtained. Define any cost, latency or error-tolerance threshold before inspecting comparison results; do not retrofit one to the winning method.

9.2 Staged gates and stopping rules

Gate	Smallest useful work	Judgment permitted and stopping condition
G0 - Identify the environment	V-CL-01; select source routes, claim classes and privacy boundaries; specify independent references	Proceed only where the required interface and test data exist. If a prerequisite is absent, record Unknown or an external limitation and stop that branch.
G1 - Check offline invariants	CORE cards, replay and deliberately mutated evidence; stub provider failures before paid calls	Repair or reject the responsible method after a critical invariant fails. Stop broad testing while the same defect prevents meaningful downstream assessment.
G2 - Check real source contracts	Relevant V-CL cards and APP-VH-02 on the actual native/client surface	Publish a bounded capability and recovery statement. Stop once the named routes and fault conditions are resolved; do not expand to unrelated clients.
G3 - Check explanation and presentation paths	Relevant VAL-EG cards, APP-VH-04, APP-VH-06 and APP-VH-07	Select a bounded explanation route only when evidence support, fallback behavior and the declared resource/privacy boundaries hold. A template route can remain the outcome.

Gate	Smallest useful work	Judgment permitted and stopping condition
G4 - Check human value and optional action	APP-VH-01; APP-VH-05 only for supported reply routes; APP-VH-08 later	Choose a representation from demonstrated understanding, and an intervention policy from useful decisions. Stop or revise if only satisfaction, clicks or alert volume improves.

These are dependency gates, not a product implementation schedule. A change should reopen the affected contract and its dependents: for example, a new client schema reopens parsing and downstream explanation cases. It need not rerun unrelated rendering tests. End a bounded investigation when its predefined question is answered or its required evidence is unavailable. Further sampling is justified by a remaining decision uncertainty, not by a desire for larger test totals.

9.3 Client interfaces and observation contracts

V-CL-01 - Surface inventory

Question: Which installed execution surface and evidence location are actually available? **Environment/data:** One intended CLI and, if relevant, desktop/remote session; non-secret configuration metadata. **Method:** Record package/build, launch mode, effective settings, schema identity and where execution and records reside. **Expected:** A consistent surface-to-source mapping. **Refutation:** Remote execution mistaken for local, inactive settings, incompatible package/schema or inaccessible records. **Measure:** Verified, unavailable and unresolved inventory fields. **Limit:** Inventory is not capture testing. **Pass permits:** Selecting an integration probe for that named surface; no completeness or universal client-support claim.

V-CL-02 - Operation and checkpoint coverage

Question: Which known operations generate usable evidence through each selected channel? **Environment/data:** Disposable project, independent action manifest and captured source records. **Method:** Perform a native read, prompt-only file mention, native edit, shell edit, search and deliberate failure; distinguish command invocation from actual filesystem effects. **Expected:** Observed categories agree with the declared source contract. **Refutation:** A promised operation is absent or classified as a different action. **Measure:** Per-operation emitted, captured, unsupported and unknown counts. **Limit:** One tool path cannot establish all equivalent paths. **Pass permits:** A version/configuration-specific coverage statement; APP-VH-02 validates its denominator.

V-CL-03 - Call pairing, retries and copied deliveries

Question: Are real-source identities sufficient for correct pairing? **Environment/data:** Two distinguishable concurrent calls, a genuine retry and a repeated delivery on the chosen client. **Method:** Arrange opposite completion order where supported; preserve original identifiers and independently label outcomes. Reuse VAL-CORE-02 for exhaustive small fixture variants. **Expected:** Each result remains with its call/attempt; duplicate delivery adds no operation. **Refutation:** Arrival-order pairing, cross-call failure transfer or retry suppression. **Measure:** Wrong links, unmatched records, duplicate actions and preserved attempts. **Limit:** A client may not emit a hypothesized interleaving. **Pass permits:** Matching claims for observed, documented input patterns.

V-CL-04 - Supported history reader versus raw parser

Question: Which history route preserves the required evidence? **Environment/data:** One normal, resumed and interrupted history, including visible child work; an authorized supported reader where available. **Method:** Compare reader and raw-parser outputs against independently identified records, maintaining content-layer distinctions. **Expected:** Required kinds, identities and relationships remain retrievable. **Refutation:** Essential omissions, changed content mistaken for raw evidence or incompatible schema assumptions. **Measure:** Omission categories, linkage differences, load time and resource use. **Limit:** Neither route is automatically complete ground truth. **Pass permits:** Choosing a reader for specified historical questions, with missing fields documented.

V-CL-05 - Source delay and recovery faults

Question: What survives source/recorder disruption? **Environment/data:** Native platform, controlled writer and fixture manifest; chosen hook, file or stream route. **Method:** Separately exercise sleep, termination, rotation, equal-size replacement, partial records, permission loss and notification overflow where applicable. For the existing inbox pattern, include concurrent appenders and reader truncation. **Expected:** Recovery preserves eligible records or exposes a bounded gap. **Refutation:** Silent loss, reused offsets across generations or invented terminal state. **Measure:** Lost/duplicated records, gap detection and recovery delay. **Limit:** Fault injection is not every storage failure. **Pass permits:** That recovery envelope; durable commit behavior additionally requires VAL-CORE-01.

V-CL-06 - Observation disturbance

Question: How does collection affect the observed task? **Environment/data:** Repeated matched workloads with no collector, minimal local handler, slow/unavailable receiver and relevant background route. **Method:** Compare execution timing, failures, permission behavior and model-visible outputs, controlling obvious workload variation. **Expected:** Measured effects stay within predefined tolerances. **Refutation:** Blocking, material slowdown, unintended context insertion or altered permission decisions. **Measure:** Latency distributions, failures and changed outputs by condition. **Limit:** Variable agent behavior can confound comparisons; use deterministic operations where possible. **Pass permits:** A workload-specific overhead bound, never a claim of zero interference.

V-CL-07 - Existing-client attachment

Question: Can the proposed authorized reader observe the intended existing session? **Environment/data:** Isolated compatible client and documented connection mode with known ownership. **Method:** First test non-resuming history access; separately test a documented cooperating live connection. Inspect session identity, prompts, lifecycle and pending requests. **Expected:** The intended session is observed under the declared contract. **Refutation:** An unrelated session is created, resume changes runtime or requests are misrouted. **Measure:** Identity continuity, event visibility and unintended side effects. **Limit:** SDK-owned execution does not establish desktop interception. **Pass permits:** Only the tested attachment mode; reply safety remains APP-VH-05.

V-CL-08 - Raw versus model-visible evidence

Question: Which result was actually made available to the agent? **Environment/data:** A controlled, permitted result-transformation path, only where supported. **Method:** Compare original output, transformed result, transcript/telemetry and Yukio attribution; inspect recipient-side evidence if exposed. **Expected:** Each layer retains its origin and uncertainty. **Refutation:** The explanation equates raw output with model input without evidence. **Measure:** Layer-label errors and unavailable recipient observations. **Limit:** Hidden receipt remains unknown; a hook name does not imply transformation capability. **Pass permits:** Correct attribution for that path, not proof of the agent’s private interpretation.

V-CL-09 - Handoff source coverage

Question: Which delegation, receipt and return evidence does the client expose? **Environment/data:** Two child tasks with distinct harmless sentinel constraints, one return and one interruption. **Method:** Inspect sender and recipient records separately; map visible IDs and source filters. Feed retained records into VAL-CORE-04. **Expected:** The capability map names the observable subset. **Refutation:** Send-only records are treated as proof of full receipt or later use. **Measure:** Available edge types, missing endpoints and preserved context scope. **Limit:** Sentinels do not reveal all context or reasoning. **Pass permits:** Specific relationship claims, with unavailable edges explicit.

V-CL-10 - Collection configuration and destinations

Question: Do content settings control every enabled collection channel? **Environment/data:** Harmless secret-like canaries, isolated sinks and explicit configuration states. **Method:** Toggle content capture, redaction, destination and history settings; inspect actual records and outbound payloads across channels. **Expected:** The stated data-flow boundary holds. **Refutation:** Disabled content appears through another channel or reaches an unintended sink. **Measure:** Boundary violations and lost useful evidence. **Limit:** A synthetic canary set is incomplete. **Pass permits:** A configuration-specific collection statement; retention, export and deletion require APP-VH-07.

9.4 Durable history, state and relationship invariants

VAL-CORE-01 - Durable cursor, deduplication and crash boundary

Question: Can records and source progress recover without silent loss? **Environment/data:** Minimal SQLite and/or JSONL candidate, native filesystem, known records, source generations and linked storage versions. **Method:** Stop the writer before/after evidence persistence, index/transaction commit and cursor advancement; restart and redeliver. Include partial records, replacement, disk-full behavior and separate evidence files if used. **Expected:** Committed facts remain readable; uncommitted work is retried; duplicates do not create actions; missing evidence is explicit. **Refutation:** Cursor advances past lost evidence, reference points to absent bytes or retry becomes a second operation. **Measure:** Lost facts, duplicates, unresolved references and recovery time. **Limit:** Process termination does not simulate every power/storage failure. **Pass permits:** Selecting the tested persistence boundary, not universal exactly-once capture. [§5.2.3-§5.2.4]

VAL-CORE-02 - Per-call state and valid interleavings

Question: Does the state reducer preserve call/attempt outcomes? **Environment/data:** Small labeled fixtures, including RUN-01; RUN-02; RUN-03 inputs, cancellation, result-without-start, duplicate delivery, conflicting result and missing result. **Method:** Vary arrival order within the declared source constraints; compare keyed facts and projections. Test goal/plan updates in their valid order. **Expected:** Failure stays with its call; absence stays unresolved; admissible reorderings converge where the contract promises convergence. **Refutation:** Another call consumes a failure, silence becomes completion or invalid order is silently accepted. **Measure:** Transition and outcome errors, conflicts preserved and unsupported assertions. **Limit:** Arbitrary permutation is not necessarily a valid source history. **Pass permits:** Those reducer invariants; real-client reachability is V-CL-03. RUN-01; RUN-02; RUN-03; §5.2.2, §5.2.5-6

VAL-CORE-03 - Available-at replay and artifact versions

Question: Are historical knowledge and later revisions distinguishable? **Environment/data:** Test result for v1 delivered after an edit to v2, versioned goals/plans and frozen interpretation versions. **Method:** Replay receipt-time cutoffs, then a revised historical view; include clock skew, duplicates and restart. Use APP-VH-03 for caches and rendered claims. **Expected:** No unavailable evidence enters an earlier snapshot; v1 verification never certifies v2; revisions identify their basis. **Refutation:** Future leakage, replaced historical meaning or replay sends live replies/notifications. **Measure:** Cutoff violations, version errors, deterministic projection differences and side effects. **Limit:** Replay cannot recover unretained evidence. **Pass permits:** Named temporal/version semantics and side-effect-free replay, not source completeness. [§5.2.6]

VAL-CORE-04 - Multi-agent edge projection

Question: Does relationship reconstruction preserve evidential distinctions? **Environment/data:** Labeled graph fixtures and V-CL-09 records, including reused worker names, missing recipient context, returns without visible use and cross-session tasks. **Method:** Remove or duplicate specific edge evidence; replay and inspect relationship provenance. **Expected:** Delegation, receipt, return and use remain separate; missing endpoints remain unresolved. **Refutation:** Parentage implies receipt, uncited return means “ignored,” or text similarity creates an authoritative edge. **Measure:** Per-edge-type precision/recall, wrong endpoints and unresolved cases. **Limit:** Explicit citation may establish reference without full causal influence. **Pass permits:** Supported relationship reconstruction, not general collaboration-error detection. [§5.3]

9.5 Explanation contracts and model operation

VAL-EG-01 - Actual provider output contract

Question: Does the chosen service/configuration support the intended response contract? **Environment/data:** Exact account, endpoint, region, model/snapshot and settings; harmless known/missing-field fixtures. **Method:** Compare plain, JSON Object and intended Schema modes where documented; retain raw responses and termination/usage information. **Expected:** Explicit supported behavior and safe handling of absent values. **Refutation:** Rejection, silent fallback, ignored constraints or invented missing facts. **Measure:** Request acceptance, parse/schema outcomes, omissions and usage. **Limit:** Syntactic conformity does not establish faithfulness. **Pass permits:** That protocol/configuration combination, subject to semantic cards below.

VAL-EG-02 - Decisive evidence selection

Question: Does selection retrieve necessary evidence without importing another scope? **Environment/data:** Independently labeled questions and records, including same-named files across runs, old attempts and long irrelevant output. **Method:** Compare structural selection, text retrieval and optional semantic reranking under matched budgets. **Expected:** Decisive records survive and foreign evidence stays excluded. **Refutation:** Missing essential failure/result or cross-run contamination. **Measure:** Essential-record coverage, contamination, retrieval cost and abstention. **Limit:** Complete selection is distinct from correct generation. **Pass permits:** A selector for tested question classes; retain the complete-evidence comparison in APP-VH-04.

VAL-EG-03 - Explanation methods against a template baseline

Question: Does free-form or hybrid generation add justified value? **Environment/data:** Frozen evidence packets, templates, model variants and blinded reviewers. **Method:** Compare outputs on identical evidence; score atomic claims, major omissions and scope errors. APP-VH-04 adds retrieval decomposition and APP-VH-01 tests human understanding. **Expected:** Useful coverage improves within predefined factual/cost tolerances. **Refutation:** Better style without useful coverage, or added unsupported claims. **Measure:** Support, contradiction, insufficiency, critical omissions, latency and cost. **Limit:** Sampled task classes bound the result. **Pass permits:** A constrained explanation route, not an unrestricted correctness assessor.

VAL-EG-04 - Semantic citation validation

Question: Can validation catch meaningful errors beyond valid JSON and existing IDs? **Environment/data:** Correct claims paired with deliberately altered claims/references. **Method:** Substitute an unrelated valid record, v2 for v1, “all tests” for selected checks, or a start for a result. **Expected:** Invalid support is rejected or marked unresolved. **Refutation:** Schema/ID success is treated as semantic approval. **Measure:** Detection and false rejection by mutation class, including difficult valid controls. **Limit:** Mutation coverage is not naturally occurring error prevalence. **Pass permits:** Validation of tested error classes, with independent review of any model judge.

VAL-EG-05 - Missing, conflicting and late evidence

Question: Does explanation respect evidence insufficiency? **Environment/data:** VAL-CORE-03 schedules plus missing result, conflicting same-ID payload and redacted-detail variants. **Method:** Generate at each permitted cutoff and compare claims with independently labeled support. **Expected:** Missing/conflicting facts remain qualified; later evidence updates only the appropriate view. **Refutation:** Unwarranted completion/failure, guessed conflict resolution or future citation. **Measure:** Unsupported resolutions, conflicts retained and stale-claim duration. **Limit:** Honest explanation cannot establish collection completeness. **Pass permits:** Handling of the tested evidence conditions; replay machinery is assessed separately.

VAL-EG-06 - Instructions embedded in evidence

Question: Can hostile records change explanation policy or judge behavior? **Environment/data:** Attack/benign pairs containing instructions to hide failure, invent success, switch tasks or fabricate references. **Method:** Place them in realistic log/output positions; evaluate generator and optional judge independently. **Expected:** Text remains evidence, without changing authority or claims. **Refutation:** Policy change or a false explanation accepted as supported. **Measure:** Violations and benign-content rejection by attack class. **Limit:** No finite fixture set proves universal resistance. **Pass permits:** Bounded attack-test results; rendering/data-flow protections also require APP-VH-07.

VAL-EG-07 - Runtime failure and stale responses

Question: Can model failure leave local observation useful? **Environment/data:** Controllable transport/stub, fixed scope versions and measurable queues. **Method:** Inject disconnect, partial JSON, wrong schema, quota response, timeout, retry and delayed response after a context change. **Expected:** Local history remains available; stale or invalid output cannot replace current meaning. **Refutation:** Observation blocks on the model or old scope overwrites new scope. **Measure:** Availability, stale overwrites, queue growth, retries and accounting gaps. **Limit:** Stub schedules cannot establish provider uptime. **Pass permits:** Tested fallback/isolation behavior; APP-VH-06 measures presentation latency.

9.6 Human understanding, safe participation and system boundaries

APP-VH-01 - Understanding from alternative interfaces

Question: Does pet-plus-workbench improve evidence judgments over an organized history? **Environment/data:** Equivalent-evidence prototypes, matched traces and intended-user participants. **Method:** Randomize/counterbalance views and trace allocation; ask which version was tested, what remains unknown, the next justified action and its source. Include agent and observer errors. **Expected:** Better judgments without unacceptable burden. **Refutation:** Satisfaction alone improves, or unsupported completion judgments increase. **Measure:** Rubric accuracy, evidence finding, timeouts, time and initial/final confidence. **Limit:** A small pilot locates problems, not population superiority. **Pass permits:** A bounded interface decision or a justified larger study.

APP-VH-02 - Validate the coverage denominator

Question: Are reported capture metrics meaningful? **Environment/data:** V-CL-02/-03 runs with separate action, source-emission and collected-record references. **Method:** Apply a frozen one-to-one matching rubric, including duplicates, retries and ambiguous records; independently review disputed labels. **Expected:** Each denominator names eligible entities and missingness. **Refutation:** Duplicate inflation, invented matches or unsupported activities excluded without disclosure. **Measure:** Source-specific precision/recall, wrong links, ambiguity and observable-action coverage separately. **Limit:** Source-emission truth may share semantic errors with the client. **Pass permits:** Accurate reporting for that contract, not coverage of all hidden activity.

APP-VH-03 - Temporal truth in caches and views

Question: Can display or caching defeat VAL-CORE-03’s cutoff/version rules? **Environment/data:** The same replay fixtures, clean/warm caches and old/new explanation configurations. **Method:** Visit historical and current views after late delivery, context change and restart. **Expected:** Rendered claims preserve snapshot and artifact scope. **Refutation:** An earlier screen knows a later result or v2 inherits v1 verification. **Measure:** Unsupported visible claims, stale duration and cross-run cache reuse. **Limit:** Passing fixtures cover their schedules. **Pass permits:** View/cache regression confidence; it does not replace the core replay test.

APP-VH-04 - Isolate retrieval, generation and judging errors

Question: Where do explanation gains and failures originate? **Environment/data:** VAL-EG-03 packets, independent claim inventory and held-out labels. **Method:** Compare normal retrieval with complete permitted evidence; blind reviewers and audit any model judge on negatives, numbers and versions. **Expected:** Improvements survive independent support and omission review. **Refutation:** Fluent but unsupported claims, hidden decisive omissions or judge agreement mistaken for truth. **Measure:** Claim labels, citation support, critical omissions, answered coverage and judge errors. **Limit:** Reviewer disagreement and task distribution persist. **Pass permits:** Tested explanation classes with fallback, not universal semantic certification.

APP-VH-05 - Request-bound response delivery

Question: Does a claimed reply route reach the intended live request? **Environment/data:** Isolated compatible clients, harmless questions/approvals and same-title sessions. **Method:** Expire/change requests, disconnect, lose receipts and repeat clicks; inspect recipients and actual dispatcher fallbacks separately. **Expected:** Correct binding or explicit uncertainty, without broadened permission or ambiguous automatic resend. **Refutation:** Wrong target, stale acceptance or success inferred only from simulated input. **Measure:** Identity, dispatch/receipt/resolution states, duplicates and wrong targets. **Limit:** Routes and UI versions differ. **Pass permits:** That constrained response feature; no arbitrary cross-client control.

APP-VH-06 - Local responsiveness and explanation independence

Question: Does the local path meet its declared objective under model delay and bursts? **Environment/data:** Specified hardware/load, monotonic timestamps and bounded queues. **Method:** Measure ingress, commit, projection and visible rendering; include offline/slow-model conditions and unfinished items. **Expected:** Local evidence remains available while explanations degrade explicitly. **Refutation:** Model-dependent rendering, silent loss or selective exclusion of failures. **Measure:** Deadline coverage, completed-render quantiles, loss, CPU/memory, queue growth and recovery. **Limit:** Local measurements exclude unknown upstream delay. **Pass permits:** Performance claims only for the tested workload envelope.

APP-VH-07 - Retention, export, deletion and safe display

Question: Do promised privacy boundaries hold beyond collection? **Environment/data:** V-CL-10 canaries, mock transport, storage/export fixtures and hostile output. **Method:** Exercise cloud gating, redaction, truncation, rendering, export, deletion and re-import; inspect derivatives and caches. **Expected:** Exclusions persist and evidence cannot execute or gain authority. **Refutation:** Canary leakage, unsafe execution or deleted content returning through a promised protected path. **Measure:** Boundary violations and over-redaction affecting utility. **Limit:** Cloud-side erasure requires separate provider evidence; fixtures cannot cover every secret. **Pass permits:** Specified local/data-flow protections, not absolute privacy.

APP-VH-08 - Future inferred-reminder utility

Question: Does a narrow issue policy improve decisions over explicit requests plus a digest? **Environment/data:** Later-stage consented episodes with goals, actionable windows and usable controls. **Method:** First assess candidates silently; then compare policies in reversible tasks with normal exploration, recovery and goal changes. **Expected:** More useful timely decisions at acceptable burden. **Refutation:** More clicks only, post-recovery alerts or no feasible user action. **Measure:** Episode precision/recall, false alerts/hour, window coverage, decision outcomes and resumption burden. **Limit:** Silent replay cannot establish human benefit. **Pass permits:** A policy for that decision family, not universal deviation detection.

9.7 Interpretation, sample limits and completion criteria

Keep **scenario coverage** separate from **accuracy estimation**. Approximately 20 classes from the earlier proposal can expose important invariants; they are not 20 independent samples from normal use. For rates, define a sampling frame that includes ordinary and difficult cases, preserve client/version strata, and account for observations clustered within tasks or participants. Select sample size from the precision or decision effect needed, rather than selecting a convenient total and declaring reliability. PROJECT-PLAN; KB §3.07

Report critical omissions separately from the fraction of generated claims supported by evidence. Abstention can avoid false claims while leaving users uninformed. Retain contradicted, insufficient and unresolved labels; adjudicator agreement is not independent proof. If probabilities are evaluated, name the predicted outcome and evaluate calibration on held-out relevant cases. The coding agent’s task success is not Yukio’s detection accuracy.

The proposed local p95 ≤1 second objective needs a declared ingress-to-visible endpoint and load. Report completed-render latency together with the fraction of **all eligible inputs** visible by the deadline and with lost/unfinished counts. Separate upstream delivery, local processing and cloud generation; excluding never-rendered events would reward failure.

Complete a gate by recording which claims its evidence supports, which remain unresolved, what falsified the candidate and the conditions requiring revalidation. Preserve actual outputs rather than replacing expected columns with optimistic summaries. Stop optional testing once the relevant decision is supported; retain narrowly scoped unknowns when the required environment or reference is absent.

10. Architecture Implications

10.1 What the evidence changes

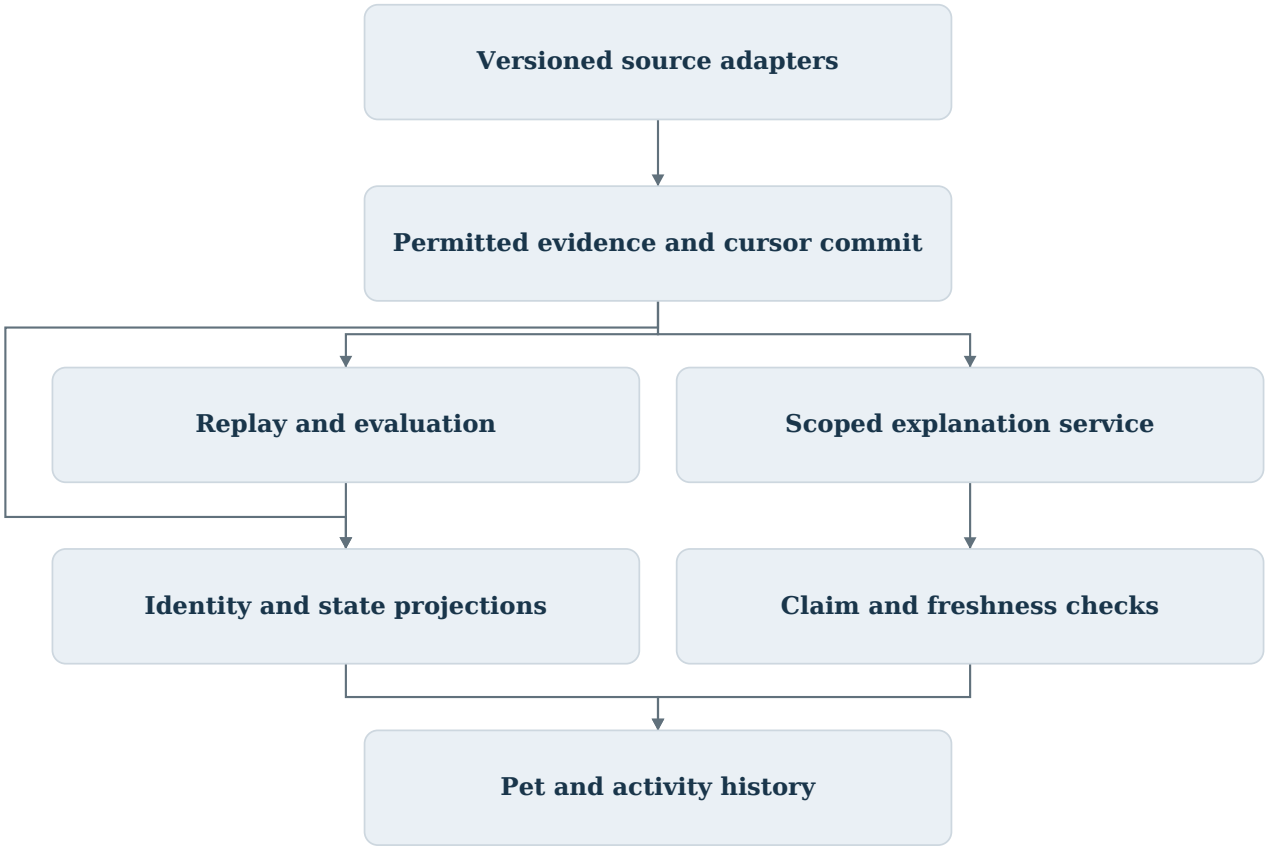
Status: Proposed Solution throughout this section. The preceding comparisons support an incremental evidence layer beside the current pet application. They do not establish that a new framework, database or provider should be deployed without the validation gates. The most consequential change is the meaning of stored data: a short display event should no longer be the only surviving account of a richer source record.

The current source/parser/router separation is useful. Its boundaries already isolate much client-specific logic from animation and UI selection. Preserve that investment where the tested behavior remains appropriate. Add a capture contract before lossy conversion, and generate a compatible PetEvent projection for the existing router. This allows the old display to remain a consumer while a new history view uses richer records. The compatibility claim is inferred from the inspected responsibilities; no such adapter has been compiled or launched here. CODE-EVENTS; CODE-SOURCES; CODE-PARSERS; CODE-ROUTER

Plain-language explanation. Yukio already has machinery for noticing activity and choosing a pose or caption. The upgrade needs a reliable notebook underneath that machinery. The notebook should keep enough information to explain an action later, even if the pet showed only a few words at the time.

10.2 Candidate responsibility boundaries

The following diagram shows responsibilities and permitted information flow. It is not a class diagram of existing code. “Committed” means stored under the chosen local persistence contract; it does not mean the source record is complete or factually infallible.



The evidence path reaches local state and the UI without waiting for a language model. The explanation service receives a bounded evidence view and returns claims that pass through checks before presentation. Replay feeds the same interpretation contracts in a mode that suppresses external actions. No arrow authorizes the explaining model to answer a permission request, change source events or control the coding agent.

Responsibility	Candidate retained foundation	Proposed addition or separation	Condition before reliance
Source discovery and reading	Existing client source families and native file readers	Versioned capability records, source-generation identity, recovery diagnostics and optional supported readers/hooks	V-CL-01-06 and VAL-CORE-01-02
Evidence preservation	Existing parsing knowledge and local file access patterns	Capture permitted original records, quality flags and references before conversion to display events	Explicit retention boundary; crash and privacy checks
Identity and state	Existing call/session handling as compatibility knowledge	Namespaced identities, call/attempt facts, independent observation health and versioned projections	No unsupported terminal states under the tested schedule
Pet display	Existing animation, captions, selection and hold logic	Consume committed projection revisions; label stale/unknown status where decision-relevant	Native UI and latency tests; no event loss caused by display filtering

Responsibility	Candidate retained foundation	Proposed addition or separation	Condition before reliance
Activity history	Retained window conventions and list/card components	An actually connected history surface, bounded queries, evidence expansion and historical revisions	Entry-point and comprehension validation
Explanation	Existing deterministic captions as a baseline	Optional provider adapter, evidence selector, immutable request context, validators and deterministic fallback	Exact provider conformance and held-out explanation comparison
Request participation	Existing provider-specific dispatch and review mechanisms	Explicit request lifecycle and route-specific capability guard	Receiving-side identity and resolution evidence for each enabled route
Evaluation	Existing unit-test and fixture conventions	Independent references, controlled replay, actual-result manifests and human-study rubrics	Development/evaluation separation and source-specific denominators

These are logical responsibilities. They need not become eight processes, services or packages. A small application can keep them as modules and data contracts. A deployment boundary should be added only when it solves a concrete lifecycle, packaging or reliability problem.

10.3 One application process or a separate local collector?

Candidate A: collection and storage within the existing application. This has the fewest new installation and communication obligations and fits the inspected native structure. One owner can serialize journal writes while the UI queries bounded projections. The tradeoff is lifecycle coupling: quitting, crashing or upgrading the pet application can stop live capture. Historical backfill may cover some of that interval only where the source retained the necessary records. This is the appropriate baseline if capture is promised only while the application is running.

Candidate B: an independently managed local collector. A helper can continue collecting while the visual application is hidden, closed or restarting, and can offer a short local handoff target for hooks. It adds an installation and update lifecycle, local authentication/permissions, protocol versioning, connection recovery, queue limits, shutdown and deletion coordination. An always-running helper still cannot capture source categories that are unavailable or disabled. It should not become an implicit broad file monitor.

Recommendation. Validate Candidate A first unless continuous capture while the visual application is closed is a confirmed requirement. If that requirement is confirmed and source backfill is insufficient, evaluate Candidate B against a specific missed-interval test. Hiding a window and quitting its process are different conditions; the coverage contract must state both. The present code has observation gates that differ between ordinary logs and some question paths, so “UI visibility never affects capture” cannot be inherited as a universal fact. CODE-SOURCES; APP-HE-14

This choice is orthogonal to SQLite versus JSONL. Either persistence method can be owned by one process or a helper; a process split does not solve transaction and file-generation errors automatically.

10.4 Data decisions that become expensive to reverse

Identity scope should be explicit early. A session ID without source namespace, a call ID reused as event identity, or one mutable record for all retries makes later repair ambiguous. Keep unavailable upstream identities null and retain the original record. A generated local ID organizes the observer's data without pretending it was supplied by the agent.

Preserve time and interpretation versions. Retaining only the latest state erases the distinction between an earlier justified unknown and a later confirmed result. Record collection cutoffs, source order where available, parser/rule versions, task/plan revisions and artifact/check scope. A newer parser may revise a historical interpretation; retain the basis of that revision instead of silently laundering it into the earlier observation.

Keep display smoothing out of evidence semantics. The current router's finite deduplication cache, inactivity policy and minimum display durations may be reasonable visual choices. The history ledger requires its own retention, identity and reconciliation rules. A hidden or coalesced card should not erase the corresponding evidence. CODE-ROUTER

Make content-retention policy part of the reference contract. If full tool output is intentionally not stored, an explanation can only cite what is retained or presently accessible, with that limitation. An external file path is a location, not an immutable historical artifact. Conversely, retaining every transcript indefinitely is not required to build a useful notebook. Compare the declared historical questions against the cost of storing their evidence.

Keep provider coupling narrow. Transport-specific response fields should end at the provider adapter. Claims, evidence references and local state should survive a provider outage or a deliberate model change. Revalidation is still necessary because a stable application interface does not make different models semantically interchangeable. APP-QW-05; APP-QW-11; KB §3.03.7

10.5 Complexity that can be deferred

The investigation has not established a requirement for Kafka, a distributed stream-processing cluster, a graph database, a vector service, autonomous reviewer agents or a universal event ontology. Local transactions, explicit joins, simple edge tables, bounded full-text search and deterministic projections provide meaningful baselines for the current scope. This is a workload-specific recommendation, not a judgment that larger systems are inherently inferior.

Likewise, no evidence requires rewriting both native applications into a shared UI framework. A shared event schema, fixture corpus and semantic conformance tests may be a less disruptive route to cross-platform consistency. The actual reuse cost remains unknown until both applications are built and tested in their native environments. The Linux Python probes do not resolve native packaging, Swift integration or Windows UI behavior.

An initial mechanism should remain replaceable at the boundaries that matter: adapter inputs, evidence storage, projection versions, explanation provider and UI queries. Decisions such as a distributed backend, semantic retrieval index, learned state estimator or optimized reminder policy can wait until a measured limitation supplies a reason to adopt them.

10.6 Failure behavior is part of the proposed architecture

Source loss should degrade coverage; parser uncertainty should preserve an unsupported record; storage failure should expose an uncommitted or lost interval; explanation failure should leave local facts usable; a stale explanation should remain attached to its old scope; an ambiguous reply receipt should not trigger automatic resubmission. Each behavior has a separate validation card. They are proposed invariants, not tested properties of the upgrade.

A particularly important tradeoff is storage failure during synchronous hooks. Blocking the agent until evidence is durable improves capture assurance but interferes with its work; returning quickly preserves responsiveness but can lose uncommitted evidence. A bounded local spool with explicit health reporting is one candidate compromise. The product must state the chosen contract and measure it. “Read-only observer” describes intended authority, not zero timing or resource effects.

Architecture selection should therefore follow the evidence gates: establish the real source and provider surfaces; preserve and associate the data they actually supply; validate history and explanation semantics; then choose the smallest integration that meets the measured workload and user-understanding requirements. This ordering is the engineering implication of the research, not an instruction to begin development immediately.

11. Recommended Priorities

11.1 Current decision gate: establish what a supported observation means

Adopt the semantic requirements in DEC-01, DEC-07, DEC-14 and DEC-15. Select one actual client surface and native environment for validation rather than claiming family-wide support. Determine which evidence survives before display conversion, and resolve the concrete correlation/recovery counterexamples. This gate precedes confident historical explanation because missing identities or discarded outputs invalidate downstream claims. It does not require selecting every future adapter. CODE-PARSERS; RUN-RECORD

11.2 Next gate: demonstrate a reviewable, reproducible record

Compare storage/recovery candidates using the same bounded workloads and faults. Require exact source/record references, explicit gaps and replay by receipt cutoff. Then evaluate simple state and template explanations against independent reference cases. Passing this gate supports a limited history capability; it does not establish universal agent correctness or complete external visibility. APP-ENG-01; APP-ENG-02; APP-ENG-03; APP-ENG-04; APP-ENG-05; APP-ENG-07; APP-ENG-08; APP-ENG-09; KB §3.07.8

11.3 Value gate: justify language generation and presentation

Resolve Qwen’s actual configuration, then compare templates and hybrid explanations on identical evidence. Evaluate the workbench against organized history using version judgment, uncertainty interpretation and evidence-finding tasks. Retain the lower-cost alternative wherever more elaborate generation or visualization adds no useful benefit. Cloud cost and latency should be recorded with the resulting explanation quality, not optimized in isolation. APP-EG-02; APP-EG-03; APP-EG-04; APP-QW-02; APP-QW-03; APP-QW-04; APP-QW-05; APP-QW-06; APP-QW-07; APP-HE-01; APP-HE-02; APP-HE-03

11.4 Later gate: demonstrate useful participation

Validate explicit request display and any chosen reply path first. Investigate inferred reminders only after defining an issue episode, useful response, responsible actor and action window. Defer general anomaly diagnosis, autonomous correction and broad remote infrastructure until a concrete need and evidence justify reconsideration. These are research-to-engineering gates, not a commitment to a development schedule. APP-HE-04; APP-HE-05; APP-HE-08; APP-HE-09

12. Research and Development Dependencies

The main dependency is evidential: **source contract → retained identity and evidence → time-aware reconstruction → supportable claims → understandable presentation → useful intervention**. Evaluation and privacy apply throughout this sequence. They should not wait until a polished interface exists. KB §§3.01-3.04, 3.06-3.07

Dependency	Decision enabled	What remains unjustified if it is absent
Release/surface contract and representative source samples	Adapter choice, declared coverage	General client support or complete capture
Durable record identity and source-generation recovery	Repeatable history, safe deduplication	Loss-free replay or stable historical references
Task/call/artifact/plan scope	State, check and progress claims	Cross-run conclusions, current-version verification
Evidence manifest, selection and claim labels	Explanation-method comparison	Treating citations or model confidence as accuracy
Actual Qwen tuple and data policy	API/schema/cost experiment	Provider availability, privacy or price assumptions
Consistent projections and intended-user tasks	Presentation comparison	Improved comprehension from layout alone
Bound request channel and receipt semantics	In-place replies	Correct delivery to arbitrary existing conversations
Actionable episodes and outcome/cost references	Reminder-policy experiment	Utility from anomaly scores or click counts

Several decisions can proceed in parallel once their inputs exist: historical storage and event-schema comparisons; template-support evaluation and UI prototypes using explicitly constructed evidence; provider contract investigation and local-only recovery testing. None requires implementing a universal controller. A provider delay should not block collection research, and an unavailable multi-agent edge should not block accurate single-call history.

Preserve the decision record when evidence changes: environment, source revision, fixture, observed result, interpretation and superseded judgment. Reopen only the affected decision. A new client release may change an adapter’s coverage; it does not automatically invalidate the distinction between execution and acceptance.

13. Decision Matrix

“Adopt Now” below can designate a requirement or evaluation rule. All proposed integrations retain their stated validation conditions; no row silently upgrades implementation evidence to A.

ID / candidate decision	Decision category	Current evidence and feasibility	Reason and condition for reconsideration
DEC-01 - Preserve native evidence, provenance and separate identities	**Adopt Now**, as a design requirement	CO: lossy display boundary; RS: provenance; PS: journal contract. B	Necessary for review and reprocessing. Retention details depend on privacy; reduced retention must explicitly reduce later explanation claims.
DEC-02 - Bounded native/history/hook adapters	**Validate First**	CO: current parsers; DC: official alternatives. B/D by surface	Compare needed fields and actual delivery. Replace or supplement a parser when a supported route demonstrates better coverage or maintenance without unacceptable overhead.
DEC-03 - Managed CLI/SDK/App Server workflow	**Validate First**, only for a selected cooperating workflow	DC; existing-session access UV. B managed, D unavailable attachment	Richer lifecycle can justify ownership costs. Defer when users require independent existing sessions and no supported attachment exists.
DEC-04 - Explicit call/attempt correlation; tentative heuristic links	**Validate First**	VI: constructed failure counterexamples; RS: identity/order. B explicit, C heuristic	Correct association precedes interpretation. Promote heuristics only with relation-specific independent evidence and visible uncertainty.
DEC-05 - Local journal: SQLite versus segmented JSONL	**Validate First**	RS: transactions and event projections; PS: integration. B	Choose from measured recovery, query and retention needs. Favor simpler segments if adequate; database choice alone cannot restore unseen events.
DEC-06 - Source generations, complete-record cursors and reconciliation	**Validate First**	VI: replacement/partial/shrink probes; RS: notification limitations. B	Required for reliable recovery. Choose polling or notification-assisted reconciliation after native identity, overflow and restart tests.
DEC-07 - Distinct execution, outcome, claim, check, acceptance and health	**Adopt Now** as meanings; **Validate First** for rules	CO: display-state conflation risk; RS/PS. B	Prevents unsupported terminal claims. Consider statistical inference only for a defined latent target with independent labels; never treat silence as sufficient.

ID / candidate decision	Decision category	Current evidence and feasibility	Reason and condition for reconsideration
DEC-08 - Templates baseline; constrained hybrid explanation	**Validate First**	RS: attribution/citation methods; PS: Yukio contract. B bounded facts, C narrative benefit	Require support, important-fact coverage and user value. Retain templates if generation’s gains do not justify errors, delay, disclosure and cost.
DEC-09 - Provider-specific Qwen adapter and locally validated schema	**Validate First**	DC: APIs with conflicting support details; project tuple UV. B conditional	Exact service configuration controls availability. Change mode/provider only after explicit comparison; no silent schema or data-boundary downgrade.
DEC-10 - Pet plus linked evidence workbench	**Validate First**	CO: reusable UI patterns; RS: HCI findings. B build, C benefit	Strong goal fit, but organized history is a credible comparator. Simplify if users cannot locate evidence or distinguish old results and unknowns.
DEC-11 - Typed, evidence-backed multi-agent relationships	**Validate First**	CO/DC: partial parent/worker records; PS: relation model. B/D	Preserve visible delegation and returns without claiming complete receipt/use. General collaboration diagnosis remains deferred.
DEC-12 - Explicit request display; scoped in-place reply	**Validate First**	CO: specific dispatch branches; DC: request protocols. B/D	Display is useful without universal control. Prefer original-client navigation until exact target, expiry and ambiguous receipt handling pass.
DEC-13 - Inferred actionable reminders and learned timing	**Investigate Further**; learned policy **Consider Later**	RS: mixed initiative/interruption; current labels UV. C	Requires feasible actions and utility evidence. Advance beyond explicit requests/digests only after episode-level and user-outcome comparisons.
DEC-14 - Independent references, replay and real-client validation	**Adopt Now**, as evaluation requirements	RS; VI limited probes. B	Separates source, collector, explanation and user errors. A sole model judge is **Do Not Adopt** as ground truth; validate any judging assistance independently.
DEC-15 - Explicit retention/cloud boundaries and inert evidence handling	**Adopt Now** as requirements; **Validate First** for enforcement	RS: logging/injection; DC: provider behavior; PS. B	Necessary because histories and prompts expose content differently. Compare retained evidence with metadata-only loss of capability using actual data requirements.

ID / candidate decision	Decision category	Current evidence and feasibility	Reason and condition for reconsideration
DEC-16 - Advanced inference and external infrastructure	**Consider Later**, or **Do Not Adopt** for unsupported present claims	RS methods; local necessity UV. C/D by use	Promote process mining, semantic indexes or remote backends only for a demonstrated question/workload. Do not adopt universal correctness, hidden-thought or project-percentage promises under current constraints.

The decisions are conditional commitments to an evidence standard and a comparison process. The validation cards in §9 specify what would justify a narrower deployment claim or overturn a candidate recommendation.

14. Supporting Evidence and References

14.1 How to interpret and continue this evidence record

This report combines supplied project documents, the Professional Research Knowledge Base, a pinned public code inspection, current primary documentation and selected research rechecks. The evidence index records the underlying source metadata and inspection scope. Source-record counts are not counts of independent studies: some records revisit the same original source for different claims, and code locators belong to one repository snapshot.

Knowledge types: K - Established Knowledge; E - Empirical Findings; P - Engineering Practices; M - Proposed Methods; H - Hypotheses; O - Open Research Questions. The technical decisions in this report are engineering synthesis unless explicitly presented as a vendor statement, a research finding or a recorded runtime result.

Inspection depths: A0 - metadata/lead; A1 - abstract or publication information; A2 - specified relevant text, listing or static code; A3 - relevant substantive methods, results or specification clauses; A4 - a narrowly described independent local execution. Always read the literal inspection scope with the label. A3 does not mean every page, linked reference or implementation was verified. No research study was experimentally reproduced here. Only RUN-01; RUN-02; RUN-03; RUN-04; RUN-05; RUN-06 have local execution evidence, and all six use constructed inputs.

Conflict rule: current scope comes from the latest user requirements; current code claims require the pinned code; interface claims require the relevant official surface and version. A newer document does not override an older installed binary. The Qwen support conflict is retained as an unresolved provider-conformance question. A source unavailable in full remains a lead or a limited-depth record, not fully verified evidence.

Search and verification strategy: English, topic-specific searches and direct primary-source retrieval were used for client contracts, model service behavior, storage semantics, HCI and evaluation. The code review inspected definitions and call sites rather than inferring behavior from filenames. Irrelevant search results and inaccessible reference pages were excluded from supporting claims. No private credentials, user transcript directories or real client sessions were accessed.

To continue without repeating the investigation, first record the actual installation and provider tuple, then select a decision and its prerequisite validation cards. Preserve the immutable code revision, source/adapter/schema version, evidence cutoff and interpretation version with every result. Reopen only the source claims affected by a newer release or a newly observed contradiction. Keep this report’s VI, CO, DC, RS, PS and UV distinctions in any later implementation assessment.

14.2 Research and educational navigation

Application subject	Professional knowledge base	Educational bridge	Important boundary
Data collection and observation limits	§§3.01.2-3.01.9, §3.01.90	Chapters 8-10	A collection channel observes a defined subset; a familiar interface name is not a coverage guarantee.
Identity, replay, causality and missing events	§§3.02.2-3.02.9, §3.02.90-91	Chapters 11-15; expanded Part 3	Rebuilding observer state is conditional on retained evidence and valid ordering.
Claims, attribution and explanation	§§3.03.2-3.03.9	Chapters 16-18	A valid reference, a source report and real-world correctness are different properties.
User understanding and visual representation	§§3.04.1-3.04.6	Chapters 19-20	Trust and satisfaction do not substitute for demonstrated understanding.
Participation and intervention utility	§§3.06.1-3.06.5	Chapters 21-22	Anomaly, error and useful intervention are distinct.
Observer evaluation and reliability	§§3.07.1-3.07.8	Chapters 23-26	Declare reference layers, denominators, cutoffs and task dependence.
Integrated reasoning and reading research	KB §§5-9	Chapters 27-30	Constructed cases teach mechanisms; they are not Yukio runtime measurements.

The source index preserves original KB identifiers where a primary source was rechecked. Inherited KB records below retain their 17 October inspection level and explicitly do not claim a new full-text reading. The complete supplied knowledge base and textbook remain the fuller teaching/research context; the following annotations identify the material used for this application decision.

14.3 Project inputs, pinned implementation and actual probes

CODE-BASE - Pinned repository identity and application structure

Author/institution: Yukio project maintainers. **Date/version:** Commit dated 2026-10-13. **Evidence:** P; Code-Observed, Not Tested.

Original source: Pinned repository identity and application structure.

Inspection: A2; Relevant definitions and call sites in the listed files. Checked 2026-10-18.

Supports: Declared release, package/platform structure and reviewed repository identity; installed binaries remain unknown.

Conditions and limits: Commit 460fb1935966ef2c3472a0bfa661fe90836e5855; installed revision unknown. Static observation only unless a specific RUN record states the exercised function, input and runtime.

Pinned code locators:

- README.md
- YukioPlayer/Package.swift
- YukioWin/README.md
- YukioWin/requirements.txt

CODE-EVENTS - Compact presentation event contract

Author/institution: Yukio project maintainers. **Date/version:** Commit dated 2026-10-13. **Evidence:** P; Code-Observed, Not Tested.

Original source: Compact presentation event contract.

Inspection: A2; Relevant definitions and call sites in the listed files. Checked 2026-10-18.

Supports: PetEvent/event fields and status vocabulary; these contracts do not preserve a complete source result.

Conditions and limits: Commit 460fb1935966ef2c3472a0bfa661fe90836e5855; installed revision unknown. Static observation only unless a specific RUN record states the exercised function, input and runtime.

Pinned code locators:

- YukioPlayer/Sources/YukioCore/Events.swift
- YukioWin/yukio/events.py

CODE-PARSERS - Client-specific parsing and lossy projection

Author/institution: Yukio project maintainers. **Date/version:** Commit dated 2026-10-13. **Evidence:** P; Code-Observed, Not Tested.

Original source: Client-specific parsing and lossy projection.

Inspection: A2; Relevant definitions and call sites in the listed files. Checked 2026-10-18.

Supports: Sidechain filtering, tool result reduction, synthetic search pairs, parent identity and pending-failure logic; runtime claims are limited to the separately recorded Python probes.

Conditions and limits: Commit 460fb1935966ef2c3472a0bfa661fe90836e5855; installed revision unknown. Static observation only unless a specific RUN record states the exercised function, input and runtime.

Pinned code locators:

- YukioPlayer/Sources/YukioCore/ClaudeParsers.swift
- YukioPlayer/Sources/YukioCore/CodexParsers.swift
- YukioWin/yukio/parsers_codex.py

CODE-SOURCES - Source discovery and incremental reading

Author/institution: Yukio project maintainers. **Date/version:** Commit dated 2026-10-13. **Evidence:** P; Code-Observed, Not Tested.

Original source: Source discovery and incremental reading.

Inspection: A2; Relevant definitions and call sites in the listed files. Checked 2026-10-18.

Supports: Discovery bounds, recent-tail bootstrap, cursors, shrink handling and disappearance-generated display events.

Conditions and limits: Commit 460fb1935966ef2c3472a0bfa661fe90836e5855; installed revision unknown. Static observation only unless a specific RUN record states the exercised function, input and runtime.

Pinned code locators:

- YukioPlayer/Sources/YukioCore/LiveSources.swift
- YukioPlayer/Sources/YukioCore/AgentSources.swift
- YukioWin/yukio/sources.py
- YukioWin/yukio/tailer.py

CODE-ROUTER - Display routing and lifecycle heuristics

Author/institution: Yukio project maintainers. **Date/version:** Commit dated 2026-10-13. **Evidence:** P; Code-Observed, Not Tested.

Original source: Display routing and lifecycle heuristics.

Inspection: A2; Relevant definitions and call sites in the listed files. Checked 2026-10-18.

Supports: Call tracking, bounded deduplication, time guards, display holds and inactivity behavior in the Swift implementation.

Conditions and limits: Commit 460fb1935966ef2c3472a0bfa661fe90836e5855; installed revision unknown. Static observation only unless a specific RUN record states the exercised function, input and runtime.

Pinned code locators:

- YukioPlayer/Sources/YukioCore/ActivityRouter.swift

CODE-HOOKS - Configured Claude hook adapter

Author/institution: Yukio project maintainers. **Date/version:** Commit dated 2026-10-13. **Evidence:** P; Code-Observed, Not Tested.

Original source: Configured Claude hook adapter.

Inspection: A2; Relevant definitions and call sites in the listed files. Checked 2026-10-18.

Supports: Five configured hooks and a synchronous append script; no hook was installed or exercised.

Conditions and limits: Commit 460fb1935966ef2c3472a0bfa661fe90836e5855; installed revision unknown. Static observation only unless a specific RUN record states the exercised function, input and runtime.

Pinned code locators:

- YukioPlayer/integrations/claude-hooks/yukio-hook.sh
- YukioPlayer/integrations/claude-hooks/settings-snippet.json

CODE-MAC-README - Pinned macOS documentation and outdated hook claim

Author/institution: Yukio project maintainers. **Date/version:** Commit dated 2026-10-13. **Evidence:** P; Code-Observed, Not Tested.

Original source: Pinned macOS documentation and outdated hook claim.

Inspection: A2; Relevant definitions and call sites in the listed files. Checked 2026-10-18.

Supports: Project documentation states absent failure hooks, which conflicts with current official documentation; reported zero overhead was not measured.

Conditions and limits: Commit 460fb1935966ef2c3472a0bfa661fe90836e5855; installed revision unknown. Static observation only unless a specific RUN record states the exercised function, input and runtime.

Pinned code locators:

- YukioPlayer/README.md

CODE-EXPLANATION-SCAN - Explanation implementation search and deterministic caption path

Author/institution: Yukio project maintainers. **Date/version:** Commit dated 2026-10-13. **Evidence:** P; Code-Observed, Not Tested.

Original source: Explanation implementation search and deterministic caption path.

Inspection: A2; Relevant definitions and call sites in the listed files. Checked 2026-10-18.

Supports: Scoped static search found no named Qwen/DashScope explanation service; existing provider enum concerns observed clients. Absence is limited to materialized source/document types and searched names.

Conditions and limits: Commit 460fb1935966ef2c3472a0bfa661fe90836e5855; installed revision unknown. Static observation only unless a specific RUN record states the exercised function, input and runtime.

Pinned code locators:

- YukioPlayer/Sources/YukioCore/ActivityCards.swift
- YukioPlayer/Sources/YukioCore/AgentProvider.swift

PROJECT-GOALS - Current Yukio goals and application-analysis requirements

Author/institution: User-provided requirements and project goal document. **Date/version:** Undated goal document; current requirements supplied 2026-10-18. **Evidence:** P; Project requirement or supplied reference.

Input/result file: 02-AI-.docx. Its exact identity is preserved in the accompanying evidence index.

Inspection: A2; User-provided input; scope of consultation stated in support. Checked 2026-10-18.

Supports: Defines observation, history, explanations, independent activity UI and future intervention; the current conversation controls scope.

Conditions and limits: The exact file and hash identify the input used. A supplied plan, synthesis or textbook is not proof that a proposed integration works.

PROJECT-PLAN - Yukio AI Activity Detection Upgrade: Technical Assessment and Implementation Plan

Author/institution: Supplied project report, 2026-10-13. **Date/version:** 2026-10-13. **Evidence:** P; Project requirement or supplied reference.

Input/result file: 03-Yukio_AI-_-.pdf. Its exact identity is preserved in the accompanying evidence index.

Inspection: A2; User-provided input; scope of consultation stated in support. Checked 2026-10-18.

Supports: All 15 pages and embedded repository link inspected; suggestions and historical reports are not implementation verification.

Conditions and limits: The exact file and hash identify the input used. A supplied plan, synthesis or textbook is not proof that a proposed integration works.

KB - Yukio Professional Research Knowledge Base

Author/institution: Supplied research synthesis, 2026-10-17. **Date/version:** 2026-10-17. **Evidence:** supplied requirements/reference; Project requirement or supplied reference.

Input/result file: Yukio_Professional_Research_Knowledge_Base.md. Its exact identity is preserved in the accompanying evidence index.

Inspection: A2; User-provided input; scope of consultation stated in support. Checked 2026-10-18.

Supports: Primary research basis; six domain chapters and relevant comparisons/limitations consulted, with original source IDs and inspection depth retained.

Conditions and limits: The exact file and hash identify the input used. A supplied plan, synthesis or textbook is not proof that a proposed integration works.

TEXTBOOK - Yukio Beginner’s Educational Textbook - English

Author/institution: Supplied educational reference, 30-chapter edition. **Date/version:** Research snapshot 2026-10-17. **Evidence:** supplied requirements/reference; Project requirement or supplied reference.

Input/result file: Yukio_Beginners_Educational_Textbook_EN.md. Its exact identity is preserved in the accompanying evidence index.

Inspection: A2; User-provided input; scope of consultation stated in support. Checked 2026-10-18.

Supports: Used for prerequisite and chapter navigation, especially Chapters 8-26; not independent experimental evidence.

Conditions and limits: The exact file and hash identify the input used. A supplied plan, synthesis or textbook is not proof that a proposed integration works.

TEXTBOOK-P3 - Part 3 - Event Tracing and State Reconstruction

Author/institution: Previously completed educational expansion. **Date/version:** 2026-10-18. **Evidence:** supplied requirements/reference; Project requirement or supplied reference.

Input/result file: Yukio_Textbook_Part_3_Event_Tracing_State_Reconstruction_EN.md. Its exact identity is preserved in the accompanying evidence index.

Inspection: A2; User-provided input; scope of consultation stated in support. Checked 2026-10-18.

Supports: Expanded Chapters 11-15 and the constructed synthesis lab; examples are fictional, not Yukio telemetry.

Conditions and limits: The exact file and hash identify the input used. A supplied plan, synthesis or textbook is not proof that a proposed integration works.

RUN-RECORD - Yukio bounded parser and tailer probes

Author/institution: This analysis. **Date/version:** 2026-10-18T19:31:27.030020+00:00. **Evidence:** E; Verified Implementation, narrowly scoped.

Input/result file: actual_results.json. Its exact identity is preserved in the accompanying evidence index.

Inspection: A4; Six executed constructed scenarios; Python parser and tailer only. Checked 2026-10-18.

Supports: Records actual outputs for ordered/interleaved failure notices and file replacement/partial/shrink behavior.

Conditions and limits: Python 3.12.14, Linux x86_64, pinned unmodified source; every input is synthetic. No native app or real client test; no event-order prevalence or overall accuracy inference.

APP-HE-14 - Yukio pinned UI lifecycle and observation-gating source inspection

Related original/source identifiers: CODE-UI.

Author/institution: Yukio project maintainers. **Date/version:** 2026. **Evidence:** P; Code-Observed, Not Tested.

Original source: Yukio pinned UI lifecycle and observation-gating source inspection.

Inspection: A2; Static reads plus source-wide search for AssistantWindowController instantiation; no application execution.. Checked 2026-10-18.

Supports: Retained reminder window exists; current entry paths are suspended or settings-only; ordinary logs and question channels do not have identical display-gating conditions.

Conditions and limits: Only this pinned source tree; shipped binaries and user settings not inspected. Static reachability is not runtime verification. No current activity workbench or measured UI benefit established.

Pinned code locators:

- docs/ASSISTANT.md - Current suspension notice contrasted with retained historical usage instructions.
- YukioPlayer/Sources/YukioPlayer/AssistantWindow.swift - Reminder pages, window constructor, store dependency, planned extensions.
- YukioPlayer/Sources/YukioPlayer/main.swift - Question/approval gates versus feed polling; reopen settings and reminder-only setup.
- YukioWin/yukio/app.py - Retained reminder state, feed/request gates and disabled show_assistant.

APP-HE-15 - Yukio pinned question dispatch, receipts and fallback source inspection

Related original/source identifiers: CODE-REPLIES.

Author/institution: Yukio project maintainers. **Date/version:** 2026. **Evidence:** P; Code-Observed, Not Tested; project documentation separately identified.

Original source: Yukio pinned question dispatch, receipts and fallback source inspection.

Inspection: A2; Static dispatch call paths, local bridge checks and Codex IPC receipt checks. No messages, approvals or client actions sent.. Checked 2026-10-18.

Supports: Some routes validate request state and observe receipts; Windows retains a conditional legacy clipboard/input route. Documentation is scoped to particular connected channels.

Conditions and limits: Source-level branches only; actual clients, installed versions, UI recognizers and request reachability not validated. No universal interface guarantee, no live end-to-end success evidence and no claim that receipts prove downstream task success. Foreground executable guards are weaker than conversation/request identity.

Pinned code locators:

- YukioPlayer/Resources/BACKGROUND-ANSWERS.md - Documented connected scope, experimental UI limits and receipt caveats.
- YukioPlayer/Sources/YukioPlayer/main.swift - Approval review, in-flight guard, owner dispatch and UI success/error branches.
- YukioWin/yukio/app.py - Approval review, background dispatch and conditional AnswerDelivery fallback.
- YukioWin/yukio/answer.py - Clipboard copy, foreground and clipboard-change guards, text/Return path.
- YukioWin/yukio/answer_bridge.py - Payload bounds, expiration/liveness, displayed-question equality and matching receipt.
- YukioWin/yukio/codex_answer.py - Thread-owner discovery, dispatch, snapshot confirmation and ambiguous post-dispatch failure.

APP-HE-16 - Yukio 0.3.2 testing record and stated boundaries

Author/institution: Yukio project maintainers. **Date/version:** 2026. **Evidence:** P; Project-reported historical tests; not independently repeated.

Original source: Yukio 0.3.2 testing record and stated boundaries.

Inspection: A2; Full 36-line testing document, including September 26 date, tested commit ef094c3, platform report scope and exclusions.. Checked 2026-10-18.

Supports: The project reports bounded historical tests and explicitly excludes some end-to-end input and planned AI features.

Conditions and limits: Document concerns an earlier implementation snapshot; linked CI run not independently audited in this contribution. Reported pass counts cannot be carried forward to the pinned October implementation or proposed workbench/Qwen service. No new operational performance was measured.

Pinned code locators:

- docs/TESTING.md - Historical tested snapshot and exclusions.

14.4 Current client and provider documentation

APP-CC-01 - Hooks reference

Author/institution: Anthropic / Claude Code. **Date/version:** Undated continuously maintained documentation; no publication date asserted. **Evidence:** P; Documented Capability.

Original source: Hooks reference.

Inspection: A3; PreToolUse checkpoints and matcher coverage; PostToolUse and result transformation; Notifications and StopFailure; Subagent identifiers; Asynchronous limitations. Checked 2026-10-18.

Supports: Named lifecycle payloads and tool IDs; successful-result versus failure events; output transformation; asynchronous teardown constraints.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Coverage depends on the actual checkpoint and matcher. File mentions and shell-side effects defeat simplistic Read/Edit coverage. Relevant sections, not every hook example, were reviewed.

APP-CC-02 - Monitoring usage

Author/institution: Anthropic / Claude Code. **Date/version:** Undated continuously maintained documentation; no publication date asserted. **Evidence:** P; Documented Capability.

Original source: Monitoring usage.

Inspection: A3; Telemetry enablement and exporters; Beta and detailed tracing; Correlation and sequence identity; Content gates, truncation and privacy; Tool-result semantics. Checked 2026-10-18.

Supports: Opt-in signals, gated detailed tracing, process-scoped event.sequence, call/prompt correlation and version-qualified message identifiers.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Not a complete trace guarantee. Interactive detailed tracing has an allowlist requirement; content is configurable and can be truncated. Only relevant sections were inspected.

APP-CC-03 - Run Claude Code programmatically

Author/institution: Anthropic / Claude Code. **Date/version:** Undated continuously maintained documentation; no publication date asserted. **Evidence:** P; Documented Capability.

Original source: Run Claude Code programmatically.

Inspection: A3; JSON and streaming output; Subagent streaming and version notes; Termination and partial results. Checked 2026-10-18.

Supports: Managed CLI JSON/JSONL output, parent-call association, forwarding conditions and interrupted-result behavior.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Launching a process does not observe an unrelated existing process. Some forked output is batched; missing final results can reflect interruption.

APP-CC-04 - Use Claude Code Desktop

Author/institution: Anthropic / Claude Code. **Date/version:** Undated continuously maintained documentation; no publication date asserted. **Evidence:** P; Documented Capability.

Original source: Use Claude Code Desktop.

Inspection: A3; Local, SSH, WSL and cloud execution; Plugins and shared/local configuration; Side-chat persistence. Checked 2026-10-18.

Supports: Execution surface and configuration differ; side chats can fall outside saved disk history.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. The desktop product name does not establish local access to remote records. No desktop installation or connection was tested.

APP-CC-05 - Agent SDK overview

Author/institution: Anthropic / Claude Code. **Date/version:** Undated continuously maintained documentation; no publication date asserted. **Evidence:** P; Documented Capability.

Original source: Agent SDK overview.

Inspection: A3; SDK versus model client; Execution ownership and harness comparison. Checked 2026-10-18.

Supports: The Python/TypeScript Agent SDK provides a Claude Code harness operated by the embedding application.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. The overview does not provide a universal subscription to independently started desktop sessions.

APP-CC-06 - Intercept and control agent behavior with hooks

Author/institution: Anthropic / Claude Code. **Date/version:** Undated continuously maintained documentation; no publication date asserted. **Evidence:** P; Documented Capability.

Original source: Intercept and control agent behavior with hooks.

Inspection: A3; Supported-hook language table; Payload and call identity; Callback timeouts and decision behavior. Checked 2026-10-18.

Supports: SDK languages expose different hook sets and payload placement; callback deadlines can affect execution.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. A Python integration cannot assume the TypeScript event table. No selected SDK release was installed or invoked.

APP-CC-07 - Work with sessions

Author/institution: Anthropic / Claude Code. **Date/version:** Undated continuously maintained documentation; no publication date asserted. **Evidence:** P; Documented Capability.

Original source: Work with sessions.

Inspection: A3; List and read stored sessions; Session storage; Resumption versus disk reading; Removed experimental TypeScript API. Checked 2026-10-18.

Supports: Documented list/read helpers provide a historical-reading candidate distinct from resuming an agent.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. The complete TypeScript API reference exceeded the retrieval limit; exact signatures, filters and release behavior still need validation.

APP-CC-08 - Stream responses in real-time

Author/institution: Anthropic / Claude Code. **Date/version:** Undated continuously maintained documentation; no publication date asserted. **Evidence:** P; Documented Capability.

Original source: Stream responses in real-time.

Inspection: A3; Partial-message enabling; Raw API stream composition; Structured-output limitations. Checked 2026-10-18.

Supports: Raw partial API events coexist with completed messages; final structured results have a different validation status from chunks.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Generated content-block completion is not proof of tool execution completion; that distinction is our protocol interpretation.

APP-CC-09 - Handle approvals and user input

Author/institution: Anthropic / Claude Code. **Date/version:** Undated continuously maintained documentation; no publication date asserted. **Evidence:** P; Documented Capability.

Original source: Handle approvals and user input.

Inspection: A3; canUseTool routing; Tool permission and AskUserQuestion; Pending callbacks. Checked 2026-10-18.

Supports: Approval and question callbacks expose selected interactions; automatically approved tools need not trigger canUseTool.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Displaying an observed request does not authorize Yukio to answer it. Callback availability does not establish a working user interaction path.

APP-CC-10 - Subagents in the SDK

Author/institution: Anthropic / Claude Code. **Date/version:** Undated continuously maintained documentation; no publication date asserted. **Evidence:** P; Documented Capability.

Original source: Subagents in the SDK.

Inspection: A3; Invocation and isolated context; parent_tool_use_id and agent/task association; Tool naming and resumability limitations. Checked 2026-10-18.

Supports: Subagent lifecycle records can expose parent-call and agent/task relationships, subject to the documented execution path.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. A delegation record is not the complete receiver input or evidence of later use. Historical Task versus Agent naming needs version-aware parsing.

APP-CX-01 - Codex App Server

Author/institution: OpenAI. **Date/version:** Undated continuously maintained documentation; no publication date asserted. **Evidence:** P; Documented Capability.

Original source: Codex App Server.

Inspection: A3; Protocol, transports and schema generation; Thread read/list and active lifecycle; Turn and item states; Collaboration items; Approvals and request resolution. Checked 2026-10-18.

Supports: Versioned thread/turn/item protocol; historical reading differs from subscription; termination and request-resolution states require interpretation.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. WebSocket transport is labelled experimental and unsupported. Existing-frontend attachment, authorization and side effects were not tested.

APP-CX-02 - Non-interactive mode

Author/institution: OpenAI. **Date/version:** Undated continuously maintained documentation; no publication date asserted. **Evidence:** P; Documented Capability.

Original source: Non-interactive mode.

Inspection: A3; exec JSONL events; Item categories; Ephemeral runs; Structured final output. Checked 2026-10-18.

Supports: A launched exec run can emit typed JSONL; persistence and final-response schemas are separate options.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. The event interface applies to the launched process; it is not passive access to arbitrary desktop runs.

APP-CX-03 - Hooks

Related original/source identifiers: APP-CX-HOOKS.

Author/institution: OpenAI. **Date/version:** Undated continuously maintained documentation; no publication date asserted. **Evidence:** P; Documented Capability.

Original source: Hooks.

Inspection: A3; Event coverage and trust; Tool execution paths; Subagent payloads; Asynchronous processing; Transcript and release-schema caveats. Checked 2026-10-18.

Supports: Local hooks cover named checkpoints with client-specific outcome semantics; hosted search and background shutdown have explicit limitations.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Main-branch schemas may exceed released behavior. Claude-compatible names do not guarantee Claude-equivalent semantics; no client hook was installed.

APP-CX-04 - Codex SDK

Author/institution: OpenAI. **Date/version:** Undated continuously maintained documentation; no publication date asserted. **Evidence:** P; Documented Capability.

Original source: Codex SDK.

Inspection: A3; TypeScript local threads; Python SDK and packaged runtime; Runtime prerequisites. Checked 2026-10-18.

Supports: Current TypeScript and Python integration paths have distinct runtime and App Server relationships.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. The page documents managed execution, not universal desktop attachment. No SDK package was installed.

APP-CX-05 - Advanced Configuration

Author/institution: OpenAI. **Date/version:** Undated continuously maintained documentation; no publication date asserted. **Evidence:** P; Documented Capability.

Original source: Advanced Configuration.

Inspection: A3; OpenTelemetry event/export controls; External notify versus TUI notification; History retention; Configuration precedence. Checked 2026-10-18.

Supports: Opt-in telemetry uses batching; external notify is a narrow turn-completion mechanism, not comprehensive activity tracing.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Exported snippets are not complete outputs; settings and history files do not guarantee all events or retained content.

APP-CX-06 - Subagents

Author/institution: OpenAI. **Date/version:** Undated continuously maintained documentation; no publication date asserted. **Evidence:** P; Documented Capability.

Original source: Subagents.

Inspection: A3; Separate agent threads; Delegation and returning results; Surface visibility. Checked 2026-10-18.

Supports: Delegation can use separate threads and return summaries to a coordinating agent.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Returned summaries do not establish a full receiver transcript. Context-quality promotional claims are not treated as experimental evidence.

APP-CX-07 - Configuration Reference

Author/institution: OpenAI. **Date/version:** Undated continuously maintained documentation; no publication date asserted. **Evidence:** P; Documented Capability.

Original source: Configuration Reference.

Inspection: A3; OTel log/trace/metric exporter keys; Prompt-content control; CLI analytics switch; History settings. Checked 2026-10-18.

Supports: Separate configuration keys govern signal exporters, content and analytics.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Only relevant key definitions were inspected, not all reference entries. Values must be checked against the installed build and effective configuration.

APP-CX-08 - Authentication

Author/institution: OpenAI. **Date/version:** Undated continuously maintained documentation; no publication date asserted. **Evidence:** P; Documented Capability.

Original source: Authentication.

Inspection: A3; OpenAI authentication and local versus cloud work; Workspace and API-organization controls. Checked 2026-10-18.

Supports: Local and cloud surfaces differ in supported sign-in paths; account choice changes applicable workspace and data policies.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. No login, credential, workspace permission or account was inspected. Authentication is not evidence of subscription access to another client.

APP-CX-09 - Compliance API and audit events

Author/institution: OpenAI. **Date/version:** Undated continuously maintained documentation; no publication date asserted. **Evidence:** P; Documented Capability.

Original source: Compliance API and audit events.

Inspection: A3; Administration-scoped records; Codex audit versus local telemetry; Explicit completeness limitations. Checked 2026-10-18.

Supports: Compliance export is an administration route with bounded resource coverage, distinct from local OTel collection.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. The guide disclaims universal local-action coverage; actual privileged API resources and access were not exercised.

APP-DC-01 - Integrate with Deep Code

Author/institution: DeepSeek. **Date/version:** Undated continuously maintained documentation; no publication date asserted. **Evidence:** P; Documented Capability.

Original source: Integrate with Deep Code.

Inspection: A3; Client identity and prerequisites; Shared terminal/VS Code settings; Per-model-turn notification. Checked 2026-10-18.

Supports: The guide identifies @vegamo/deepcode-cli and a configurable notification after each model turn.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. No complete stable activity schema is specified. Source-code parser evidence belongs to the separate repository audit; DeepSeek as a provider does not identify the harness.

APP-QW-01 - qwen3.8-max

Author/institution: Alibaba Cloud Model Studio. **Date/version:** 2026-10-11. **Evidence:** P; Documented Capability.

Original source: qwen3.8-max.

Inspection: A3; Model ID; Regional capability tables; Snapshot versions. Checked 2026-10-18.

Supports: The catalog identifies qwen3.8-max, its 0902 snapshot and date alias, and advertised function-calling and structured-output capabilities.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Not account access, installed integration, explanation quality, schema behavior for every region, or an immutable model fingerprint. Marketing claims excluded from feasibility evidence.

APP-QW-02 - Structured output

Author/institution: Alibaba Cloud Model Studio. **Date/version:** 2026-10-12. **Evidence:** P; Documented Capability.

Original source: Structured output.

Inspection: A3; Usage and supported models (lines 20-79); Getting structured output (1144-1224); Configuration guide (1309-1380); Going live (1381-1392). Checked 2026-10-18.

Supports: Distinct JSON Object/Schema modes and documented restrictions; summary/detail model lists conflict, as do Singapore restriction and examples.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Conflicting documentation cannot settle the actual region/model/mode/schema tuple. Multimodal downgrade and schema-keyword qualifications prevent a universal strictness claim; semantic support remains separate.

APP-QW-03 - Regions and endpoints

Author/institution: Alibaba Cloud Model Studio. **Date/version:** 2026-10-13. **Evidence:** P; Documented Capability.

Original source: Regions and endpoints.

Inspection: A3; Region overview; Service deployment scope; Endpoint comparison; Regional access information. Checked 2026-10-18.

Supports: Separates storage/access region from inference scope and endpoint class; keys/model lists are region-bound; shared endpoint feature-freeze date is stated.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. No claim about the user's actual account or geographic configuration; feature freeze is not total endpoint shutdown.

APP-QW-04 - Base URL overview

Author/institution: Alibaba Cloud Model Studio. **Date/version:** 2026-10-11. **Evidence:** P; Documented Capability.

Original source: Base URL overview.

Inspection: A3; Introduction; Pay-as-you-go; DashScope Domain; Workspace-dedicated Domain. Checked 2026-10-18.

Supports: Lists protocol-specific regional URLs and matching workspace/plan key requirements.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Example placeholders are not configured project endpoints; no credentials or subscription verified.

APP-QW-05 - OpenAI compatible - Chat

Author/institution: Alibaba Cloud Model Studio. **Date/version:** 2026-10-16. **Evidence:** P; Documented Capability.

Original source: OpenAI compatible - Chat.

Inspection: A3; Request body: stream, enable_thinking, tool_choice; Chat completion object; Chat response chunk object. Checked 2026-10-18.

Supports: Documents request/response shapes, termination, partial timeout behavior, Qwen-specific tool restrictions, usage, and null fingerprint.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Generic fields have model-specific qualifications. No full API conformance test or target account invocation.

APP-QW-06 - Streaming output

Author/institution: Alibaba Cloud Model Studio. **Date/version:** 2026-10-13. **Evidence:** P; Documented Capability.

Original source: Streaming output.

Inspection: A3; How it works; Billing; How to use: enabling stream and usage. Checked 2026-10-18.

Supports: SSE assembly, terminal usage option, and server-receipt-dependent cancellation accounting are documented.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Not demonstrated latency, a guarantee that a

disconnected client receives usage, or evidence that partial output is displayable as a verified claim.

APP-QW-07 - Error codes

Author/institution: Alibaba Cloud Model Studio. **Date/version:** 2026-10-11. **Evidence:** P; Documented Capability.

Original source: Error codes.

Inspection: A3; 400 InvalidParameter; 401 authorization/key issues; 429 categories; 500/503 errors. Checked 2026-10-18.

Supports: Documents configuration, authorization, quota/rate, overload, and service failure classes.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Not every 429 is a transient rate error; examples span different models and plans. Error handling must retain exact provider codes.

APP-QW-08 - Provider Routing

Author/institution: OpenRouter. **Date/version:** Undated documentation. **Evidence:** P; Documented Capability.

Original source: Provider Routing.

Inspection: A3; Provider object; Default routing; Provider restrictions and parameter requirements. Checked 2026-10-18.

Supports: Documents multi-provider routing and controls including only, allow_fallbacks, and require_parameters.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. A routing setting is not an independently verified provider contract, quality guarantee, or configuration of Yukio.

APP-QW-09 - Structured Outputs

Author/institution: OpenRouter. **Date/version:** Undated documentation. **Evidence:** P; Documented Capability.

Original source: Structured Outputs.

Inspection: A3; Model Support; Best Practices; Streaming; Error Handling. Checked 2026-10-18.

Supports: Support and strict enforcement vary by endpoint; required parameter filtering is documented.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Some routes translate schemas or use hints; generic support is not exact conformance for a chosen Qwen endpoint.

APP-QW-10 - Qwen API and Models

Author/institution: OpenRouter. **Date/version:** Undated changing catalog. **Evidence:** P; Documented Capability.

Original source: Qwen API and Models.

Inspection: A2; Qwen model listing, including Qwen3.8 Max (0902). Checked 2026-10-18.

Supports: Shows an alternative managed service publicly listing Qwen family and snapshot offerings.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Catalog presence is not tested availability, verified schema support, or the user's selected provider. No catalog prices transferred to Yukio.

APP-QW-11 - DashScope API Reference

Author/institution: Alibaba Cloud Model Studio. **Date/version:** 2026-10-13. **Evidence:** P; Documented Capability.

Original source: DashScope API Reference.

Inspection: A3; Regional native HTTP paths; Request body organization; Tool calling qualifications. Checked 2026-10-18.

Supports: Documents native protocol organization and distinguishes text-generation from multimodal-generation paths.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Native route and SDK class must match the selected model, not simply the family name. No live request made.

14.5 Independently inspected engineering and research sources

APP-ENG-01 - SQLite Is Transactional

Author/institution: SQLite developers. **Date/version:** Undated maintained documentation. **Evidence:** P; Documented Capability.

Original source: SQLite Is Transactional.

Inspection: A3; Atomicity and transaction description; complete short page. Checked 2026-10-18.

Supports: Describes all-or-nothing transactions and SQLite's transaction-testing approach.

Conditions and limits: Operations lie within a correctly used SQLite transaction and its storage assumptions. No local Yukio database integration, crash experiment or performance measurement; upstream capture and external files are outside this guarantee.

APP-ENG-02 - Write-Ahead Logging

Author/institution: SQLite developers. **Date/version:** Maintained page inspected 2026-10-18; section 11 identifies a March 2026 fix. **Evidence:** P; Documented Capability.

Original source: Write-Ahead Logging.

Inspection: A3; Reader/writer concurrency; Checkpoints and durability; Same-host restriction; Section 11 WAL-reset bug. Checked 2026-10-18.

Supports: Describes one-writer WAL concurrency, checkpoint/busy behavior and a rare concurrent WAL-reset race fixed in 3.51.3, with specified backports.

Conditions and limits: The linked SQLite version, connection pattern, journaling and synchronization configuration matter; fix status must include backports. A vendor issue/fix description, not a reproduced Yukio incident or evidence that every older deployment has experienced corruption.

APP-ENG-03 - Atomic Commit In SQLite

Author/institution: SQLite developers. **Date/version:** Undated maintained technical article. **Evidence:** P; Documented Capability.

Original source: Atomic Commit In SQLite.

Inspection: A3; Sections 1-5 selected passages: storage assumptions, commit, recovery and multiple files. Checked 2026-10-18.

Supports: Explains journal-based commit and recovery under explicit filesystem, locking and flush assumptions.

Conditions and limits: Correct operation of required OS storage primitives and declared transaction boundaries. Selected sections only; historical implementation examples do not independently validate modern native integration or external evidence-file atomicity.

APP-ENG-04 - Appropriate Uses For SQLite

Author/institution: SQLite developers. **Date/version:** Undated maintained documentation. **Evidence:** P; Documented Capability.

Original source: Appropriate Uses For SQLite.

Inspection: A2; Embedded applications and application file format. Checked 2026-10-18.

Supports: Identifies embedded and desktop application-data storage as intended SQLite use cases.

Conditions and limits: Workload and deployment fit local application storage; actual query and write load must be tested. Use-case guidance is not a benchmark or a guarantee that SQLite is preferable to JSONL for every Yukio workload.

APP-ENG-05 - OpenTelemetry Logs Data Model

Related original/source identifiers: S-OT18.

Author/institution: OpenTelemetry Authors. **Date/version:** Maintained specification; accessed 2026-10-18. **Evidence:** P; Documented Capability.

Original source: OpenTelemetry Logs Data Model.

Inspection: A3; Timestamp and ObservedTimestamp definitions; Optional trace/span correlation fields. Checked 2026-10-18.

Supports: Separates source occurrence time from collection observation time and allows absent source timestamps.

Conditions and limits: Field semantics are applied according to the selected specification; client adoption remains separate. The inspected field definitions do not establish transport completeness, a clock-synchronization bound or standard conformance of a proposed local schema.

APP-ENG-06 - PROV-DM: The PROV Data Model

Related original/source identifiers: S-GE01.

Author/institution: Luc Moreau and Paolo Missier (editors); W3C. **Date/version:** 2013-04-30. **Evidence:** P; K; Research-Supported Method.

Original source: PROV-DM: The PROV Data Model.

Inspection: A3; Entity/activity concepts; Derivation and revision; Provenance bundles. Checked 2026-10-18.

Supports: Supplies vocabulary for provenance, derivation and version-related relationships.

Conditions and limits: Relationships are asserted with appropriate evidence and identified entities/activities. Selected specification clauses only; a provenance edge is not independent proof that information had a particular semantic influence.

APP-ENG-07 - Event Sourcing

Related original/source identifiers: S-OT24.

Author/institution: Martin Fowler. **Date/version:** 2005-12-12. **Evidence:** P; Research-Supported Method.

Original source: Event Sourcing.

Inspection: A3; State reconstruction; External updates; External queries; Code changes and replay. Checked 2026-10-18.

Supports: Explains reconstructing state from retained domain events and additional handling required for external effects and historical queries.

Conditions and limits: The event history and interpretation assumptions support the intended reconstruction; replay effects are controlled. An engineering pattern article, not a theorem that arbitrary observer logs contain all source events.

APP-ENG-08 - Time, Clocks, and the Ordering of Events in a Distributed System

Related original/source identifiers: S-OT19.

Author/institution: Leslie Lamport. **Date/version:** 1978. **Evidence:** K; Research-Supported Method.

Original source: Time, Clocks, and the Ordering of Events in a Distributed System. **DOI:** 10.1145/359545.359563.

Inspection: A3; Happened-before definition and logical/physical ordering, selected first four pages. Checked 2026-10-18.

Supports: Establishes a partial order from process order and communication, and a compatible logical-clock ordering.

Conditions and limits: The specified processes and message relationships are available and satisfy the model. A display total order is not unique causal history; this theory does not recover missing propagation or reveal private decision reasons.

APP-ENG-09 - ReadDirectoryChangesW function

Author/institution: Microsoft. **Date/version:** 2022-09-15. **Evidence:** P; Documented Capability.

Original source: ReadDirectoryChangesW function.

Inspection: A3; Change notifications; Asynchronous result handling; Buffer overflow and enumeration. Checked 2026-10-18.

Supports: Documents scoped directory notifications and the need to enumerate again after overflow.

Conditions and limits: Correct handles, permissions, buffers and completion processing for the selected Windows environment. No native Windows run here; notification is not historical file content, complete operation counting or agent/task attribution.

APP-EG-01 - PROV-DM: The PROV Data Model

Author/institution: Luc Moreau and Paolo Missier, editors; W3C. **Date/version:** 2013-04-30. **Evidence:** K; P; Research-Supported Method.

Original source: PROV-DM: The PROV Data Model.

Inspection: A3; Entity/Activity/Agent definitions; 2.1.2 Derivation; 5.3.5 Influence. Checked 2026-10-18.

Supports: Usage plus generation is insufficient to establish derivation; influence is additionally required.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Represents provenance; it does not discover true causal influence or establish source truth.

APP-EG-02 - Measuring Attribution in Natural Language Generation Models

Author/institution: Hannah Rashkin; Vitaly Nikolaev; Matthew Lamm; Lora Aroyo; Michael Collins; Dipanjan Das; Slav Petrov; Gaurav Singh Tomar; Iulia Turc; David Reitter. **Date/version:** 2023; Computational Linguistics 49(4). **Evidence:** M; E; Research-Supported Method.

Original source: Measuring Attribution in Natural Language Generation Models. **DOI:** 10.1162/coli_a_00486.

Inspection: A3; Introduction and source-quality boundary; 3.1 attribution with time; 3.2 contextual meaning. Checked 2026-10-18.

Supports: AIS evaluates contextual source support; temporal and linguistic meaning precede attribution judgments.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Read selected portions of the 64-page PDF. Source truth and Yukio performance are outside its demonstrated scope.

APP-EG-03 - Enabling Large Language Models to Generate Text with Citations

Author/institution: Tianyu Gao; Howard Yen; Jiatong Yu; Danqi Chen. **Date/version:** 2023; EMNLP. **Evidence:** M; E; Research-Supported Method.

Original source: Enabling Large Language Models to Generate Text with Citations. **DOI:** 10.18653/v1/2023.emnlp-main.398.

Inspection: A3; 3.3 Citation Quality; 6 human evaluation; F annotation criteria. Checked 2026-10-18.

Supports: Citation support and irrelevance are separately measured; generated-statement citation recall differs from important-fact coverage.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Question-answer generation conditions, not execution logs; automated entailment has error and precision does not require a minimal citation set.

APP-EG-04 - FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation

Author/institution: Sewon Min; Kalpesh Krishna; Xinxi Lyu; Mike Lewis; Wen-tau Yih; Pang Wei Koh; Mohit Iyyer; Luke Zettlemoyer; Hannaneh Hajishirzi. **Date/version:** 2023; EMNLP. **Evidence:** M; E; Research-Supported Method.

Original source: FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. **DOI:** 10.18653/v1/2023.emnlp-main.741.

Inspection: A3; Atomic-fact method; 4.2 estimation error; B.2 aggregate versus segment evaluation; Limitations. Checked 2026-10-18.

Supports: Atomic support assessment; reported small ER is an aggregate-score estimation difference, not per-claim error.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Predominantly biography/Wikipedia experiments; does not establish observer accuracy, complete coverage, or joint entity consistency.

APP-EG-05 - AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents

Author/institution: Edoardo Debenedetti; Jie Zhang; Mislav Balunović; Luca Beurer-Kellner; Marc Fischer; Florian Tramèr. **Date/version:** 2024; inspected arXiv v3 revised 2024-11-24. **Evidence:** M; E; Research-Supported Method.

Original source: AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. **DOI:** 10.48550/arXiv.2406.13352.

Inspection: A3; Introduction: tool-data injection; 3 environment/state/tasks; Utility-security distinction. Checked 2026-10-18.

Supports: Tool-provided data can carry attacks; user utility and attack outcomes require separate evaluation.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Selected agent environments and attack classes; observer-log contamination is a reasoned transfer, not a demonstrated Yukio exploit.

APP-EG-06 - Merging Facts, Crafting Fallacies: Evaluating the Contradictory Nature of Aggregated Factual Claims in Long-Form Generations

Author/institution: Cheng-Han Chiang; Hung-yi Lee. **Date/version:** 2024; Findings of ACL. **Evidence:** M; E; Research-Supported Method.

Original source: Merging Facts, Crafting Fallacies: Evaluating the Contradictory Nature of Aggregated Factual Claims in Long-Form Generations. **DOI:** 10.18653/v1/2024.findings-acl.160.

Inspection: A3; 2 ambiguous-name construction; 3 atomic-score limitations; D-FActScore discussion. Checked 2026-10-18.

Supports: Individually supported facts from different same-named people can form an incorrect aggregate biography.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Transfer to files/runs is engineering inference; this paper did not test Yukio or its identity scheme.

APP-EG-07 - FEVER: a Large-scale Dataset for Fact Extraction and VERification

Author/institution: James Thorne; Andreas Vlachos; Christos Christodoulopoulos; Arpit Mittal. **Date/version:** 2018; NAACL. **Evidence:** M; E; Research-Supported Method.

Original source: FEVER: a Large-scale Dataset for Fact Extraction and VERification. **DOI:** 10.18653/v1/N18-1074.

Inspection: A3; 3 claim generation and labeling; Supported/refuted/insufficient-information evidence task. Checked 2026-10-18.

Supports: Separates evidential contradiction from insufficient evidence with explicit source selection.

Conditions and limits: See the explicitly named interface, study or code scope; no target-environment transfer is assumed. Wikipedia-derived task; labels provide a useful pattern but no execution-log validation performance.

APP-HE-01 - Guidelines for Human-AI Interaction

Related original/source identifiers: S-HI03.

Author/institution: Saleema Amershi; Dan Weld; Mihaela Vorvoreanu; Adam Fourney; Besmira Nushi; Penny Collisson; Jina Suh; Shamsi Iqbal; Paul N. Bennett; Kori Inkpen; Jaime Teevan; Ruth Kikin-Gil; Eric Horvitz. **Date/version:** 2019. **Evidence:** M; E; P; Research-Supported Method.

Original source: Guidelines for Human-AI Interaction. **DOI:** 10.1145/3290605.3300233.

Inspection: A3; Table 1 and four-stage development/modified heuristic-evaluation methods, including the 49-practitioner, 20-product scope.. Checked 2026-10-18.

Supports: Guidelines support capability communication, dismissal, correction and control; their evaluation does not establish Yukio user benefit.

Conditions and limits: Forty-nine HCI practitioners assessed the clarity and applicability of the guidelines across 20 AI products in a modified heuristic evaluation. This was not a randomized controlled experiment measuring comprehension or safety gains from following the guidelines.; General human-AI interaction guidelines do not directly demonstrate the effectiveness of a third-party agent-observation interface.

APP-HE-02 - Manipulating and Measuring Model Interpretability

Related original/source identifiers: S-HI07.

Author/institution: Forough Poursabzi-Sangdeh; Daniel G. Goldstein; Jake M. Hofman; Jennifer Wortman Vaughan; Hanna Wallach. **Date/version:** 2021. **Evidence:** E; Research-Supported Method.

Original source: Manipulating and Measuring Model Interpretability. **DOI:** 10.1145/3411764.3445315.

Inspection: A3; Preregistered task/manipulation definitions; detection/correction operationalization; differing outcomes across experiments including outlier-focus conditions.. Checked 2026-10-18.

Supports: Transparency can affect simulation and correction differently under the studied apartment-estimation conditions; no uniform interface ranking follows.

Conditions and limits: N=3,800 in preregistered apartment-price prediction experiments; the number of linear-model features and model transparency were manipulated. The controlled tasks involved apartment prices and linear models, not multistep agent traces.; Information overload is one author interpretation. Without direct process measurements, it must not be treated as an established mediating mechanism.

APP-HE-03 - To Trust or to Think: Cognitive Forcing Functions Can Reduce Overreliance on AI in AI-Assisted Decision-Making

Related original/source identifiers: S-HI08.

Author/institution: Zana Buçinca; Maja Barbara Malaya; Krzysztof Z. Gajos. **Date/version:** 2021. **Evidence:** E; Research-Supported Method.

Original source: To Trust or to Think: Cognitive Forcing Functions Can Reduce Overreliance on AI in AI-Assisted Decision-Making. **DOI:** 10.1145/3449287.

Inspection: A3; Methods, retained sample, simple-XAI/cognitive-forcing comparison, incorrect-AI outcomes, subjective measures and discussion limiting overall performance claims.. Checked 2026-10-18.

Supports: Cognitive forcing reduced certain overreliance outcomes but did not uniformly improve overall team performance; effort and preference tradeoffs remain.

Conditions and limits: N=199 retained participants; a nutrition-choice task with simulated AI errors compared simple explanations with several interactions encouraging independent thinking. Outcomes differ: identifying erroneous carbohydrate sources and making the complete decision must not be combined into one measure.; Overall effects across all questions were not uniformly significant; the intervention cannot be described as eliminating overreliance.

APP-HE-04 - Principles of Mixed-Initiative User Interfaces

Related original/source identifiers: S-HI16.

Author/institution: Eric Horvitz. **Date/version:** 1999. **Evidence:** K; M; P; Research-Supported Method.

Original source: Principles of Mixed-Initiative User Interfaces. **DOI:** 10.1145/302979.303030.

Inspection: A3; Expected-utility equations for action/inaction, dialogue option and timing discussion in LookOut.. Checked 2026-10-18.

Supports: Dialogue is an action with benefits and costs; uncertainty alone is not a universal asking rule.

Conditions and limits: The LookOut calendar-assistance prototype and decision theory; goal probabilities, actions, dialogue and time costs are modelled in advance. This is a prototype in the calendar-assistance domain. Utilities and probabilities must come from a model of that context; the original paper does not provide a universal threshold for agent intervention.

APP-HE-05 - Monitoring Distributed Systems

Related original/source identifiers: S-HI25.

Author/institution: Rob Ewaschuk; Betsy Beyer (editor). **Date/version:** 2016. **Evidence:** P; Documented engineering practice.

Original source: Monitoring Distributed Systems.

Inspection: A3; Official chapter: Why Monitor, reasonable expectations, symptoms/causes and actionable-paging criteria.. Checked 2026-10-18.

Supports: Human interruption is distinct from logging and investigation; simplicity and actionability are documented engineering practices.

Conditions and limits: Monitoring and on-call practices for production distributed services at Google distinguish ordinary recording, issues needing investigation and urgent alerts that page on-call personnel for immediate human action. Production engineering experience is not a universal intervention policy across industries or a criterion for judging agent errors.

APP-HE-06 - Evaluating Explainable AI: Which Algorithmic Explanations Help Users Predict Model Behavior?

Related original/source identifiers: S-GE26.

Author/institution: Peter Hase; Mohit Bansal. **Date/version:** 2020. **Evidence:** M; E; Research-Supported Method.

Original source: Evaluating Explainable AI: Which Algorithmic Explanations Help Users Predict Model Behavior?. **DOI:** 10.18653/v1/2020.acl-main.491.

Inspection: A3; Primary PDF Sections 4, 5.1 and 5.3: design, simulation tasks, participant limitations, subjective/objective analyses and uncertainty.. Checked 2026-10-18.

Supports: Subjective explanation ratings did not reliably predict objective performance in these tasks; absence of significant association is not proof of no relationship.

Conditions and limits: The study compared explanations in its text- and tabular-classification model-behavior prediction tasks and collected subjective ratings; participants and procedures followed its research design. The authors do not interpret a nonsignificant association as proving no relationship. These classification tasks are not direct tests of desktop-agent user comprehension.

APP-HE-07 - Precision and Recall for Time Series

Related original/source identifiers: S-GE27.

Author/institution: Nesime Tatbul; Tae Jun Lee; Stan Zdonik; Mejbah Alam; Justin Gottschlich. **Date/version:** 2018. **Evidence:** M; E; Research-Supported Method.

Original source: Precision and Recall for Time Series.

Inspection: A3; Primary PDF Section 4: existence, coverage, position and cardinality terms and application-selected weights.. Checked 2026-10-18.

Supports: Interval precision/recall may reward different detection properties; these weights require a use-case choice.

Conditions and limits: Ground truth and predictions are intervals. Existence, positional coverage, interval-length and fragmentation rewards are specified before interval precision and recall are calculated. Different weights encode different objectives; there is no unconditionally optimal formulation. Applying anomaly-interval methods to agent-state intervals requires redefining the evaluation target.

APP-HE-08 - The Cost of Interrupted Work: More Speed and Stress

Related original/source identifiers: S-HI29.

Author/institution: Gloria Mark; Daniela Gudith; Ulrich Klocke. **Date/version:** 2008. **Evidence:** E; Research-Supported Method.

Original source: The Cost of Interrupted Work: More Speed and Stress. **DOI:** 10.1145/1357054.1357072.

Inspection: A3; Primary 4-page paper: N=48 study, timing definition subtracting interruption-handling time, workload/stress outcomes and limitations.. Checked 2026-10-18.

Supports: Faster measured primary work can coexist with higher stress; the measure does not show shorter total elapsed work.

Conditions and limits: N=48 in a simulated office-email task with same- or different-context interruptions. Measured primary-task time excluded time spent handling interruptions. The result does not imply shorter total elapsed time; a nonsignificant quality difference does not establish general quality equivalence.; Compensatory acceleration is an author interpretation requiring finer process evidence. A short experiment cannot predict long-term adaptation.

APP-HE-09 - Effects of Workload, Work Complexity, and Repeated Alerts on Alert Fatigue in a Clinical Decision Support System

Related original/source identifiers: S-HI20.

Author/institution: Jessica S. Ancker; Alison Edwards; Sarah Nosal; Diane Hauser; Elizabeth Mauer; Rainu Kaushal; HITEC Investigators. **Date/version:** 2017. **Evidence:** E; Research-Supported Method.

Original source: Effects of Workload, Work Complexity, and Repeated Alerts on Alert Fatigue in a Clinical Decision Support System. **DOI:** 10.1186/s12911-017-0430-8.

Inspection: A3; Publisher full text: retrospective sampling, repeated-alert/acceptance definitions, results, time trends and missing appropriateness review.. Checked 2026-10-18.

Supports: Repeated exposure and acceptance need distinct interpretation; observed associations do not establish a universal habituation or correctness mechanism.

Conditions and limits: Retrospective alert logs from 112 outpatient primary-care clinicians during 2010-2013; acceptance was a system-recorded behavioral measure. The observational study does not establish causality; individual alerts were not manually assessed for clinical appropriateness.; Acceptance or ignoring is not actual clinical correctness and cannot be transferred directly to developer notifications.

APP-HE-10 - On Calibration of Modern Neural Networks

Related original/source identifiers: S-GE20.

Author/institution: Chuan Guo; Geoff Pleiss; Yu Sun; Kilian Q. Weinberger. **Date/version:** 2017. **Evidence:** K; M; E; Research-Supported Method.

Original source: On Calibration of Modern Neural Networks.

Inspection: A3; Primary PDF Section 2 calibration definition and empirical binning; Section 4 held-out distribution assumptions and limitations.. Checked 2026-10-18.

Supports: Calibration links specified predictions with empirical outcomes; an LLM confidence phrase is not independently calibrated by its format.

Conditions and limits: Confidence and empirical correctness frequencies were studied for classification predictions with defined reference labels and separate calibration data; temperature-scaling parameters were fitted on a designated calibration set. The experiments concern classification with defined labels. They do not guarantee calibration of LLM-reported percentages or continued calibration after distribution shift.

APP-HE-11 - Inter-Coder Agreement for Computational Linguistics

Related original/source identifiers: S-GE25.

Author/institution: Ron Artstein; Massimo Poesio. **Date/version:** 2008. **Evidence:** K; Research-Supported Method.

Original source: Inter-Coder Agreement for Computational Linguistics. **DOI:** 10.1162/coli.07-034-R2.

Inspection: A2; Primary PDF Section 2.1 agreement, reliability and validity; selected definition context, not all coefficient derivations reread.. Checked 2026-10-18.

Supports: Annotator agreement cannot exclude shared errors or establish the validity of a coding scheme.

Conditions and limits: Repeated judgments by multiple annotators require explicit annotation units, categories or distance definitions, with agreement coefficients suited to the task and label distribution. There is no universal threshold independent of category prevalence, task and coefficient assumptions. High agreement does not establish factual correctness or construct validity.

APP-HE-12 - LLM01:2025 Prompt Injection

Author/institution: OWASP Gen AI Security Project. **Date/version:** 2025. **Evidence:** P; Documented security guidance.

Original source: LLM01:2025 Prompt Injection.

Inspection: A3; Direct and indirect injection definitions; mitigation sections 1-7 and output/external-content example.. Checked 2026-10-18.

Supports: External content may steer model output; privilege separation, output validation, content boundaries and adversarial tests are mitigation practices rather than proof of perfect prevention.

Conditions and limits: General LLM applications with untrusted input; the Yukio implications are engineering synthesis. Guidance, not an experimental Yukio defense rate. The source does not establish immunity from prompt engineering, structured output or read-only role.

APP-HE-13 - Logging Cheat Sheet

Author/institution: OWASP Cheat Sheet Series contributors. **Date/version:** Undated maintained guidance; checked 2026-10-18. **Evidence:** P; Documented security guidance.

Original source: Logging Cheat Sheet.

Inspection: A3; Purpose, trust boundaries and source reliability; event attributes; data to exclude; protection and disposal headings. Relevant source/security paragraphs read, not all linked references.. Checked 2026-10-18.

Supports: Logging should be purpose-specific and account for untrusted sources, sensitive fields, access, protection and disposal.

Conditions and limits: General application/security logging guidance; Yukio code-evidence retention requires a purpose-specific adaptation. Dynamic undated guidance snapshot. It is not a claim that logging requirements mechanically forbid Yukio from retaining expressly selected code evidence. No legal-compliance conclusion or complete secret-detection guarantee is derived.

14.6 Inherited KB foundations and limitations

S-OT05 - Trace Context - W3C Recommendation, 23 November 2021

Author/institution: W3C Distributed Tracing Working Group. **Date/version:** 2021. **Evidence:** P; Research-Supported Method.

Original source: Trace Context - W3C Recommendation, 23 November 2021.

Inspection: A3; §3.2 traceparent; §3.2.2.5 trace-flags/Sampled; §4 Processing Model.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: Trace Context standardizes cross-component identity propagation. Its sampled flag is advisory and subject to implementation and load conditions.

Conditions and limits: Participating components must parse and propagate the selected Trace Context version; their sampling decisions may differ. The sampling flag is neither a persistence receipt nor proof of end-to-end completeness. The inspected text was the 2021 Recommendation.

S-OT09 - BPF ring buffer - Linux Kernel Documentation

Author/institution: Linux Kernel Documentation Contributors. **Date/version:** Continuously maintained; checked in 2026. **Evidence:** P; Research-Supported Method.

Original source: BPF ring buffer - Linux Kernel Documentation.

Inspection: A3; Motivation; Semantics and APIs; Design and Implementation; reservation and NMI failure conditions.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: A ring buffer can be shared across CPUs, but full capacity or certain lock-contention conditions can cause reservation failure.

Conditions and limits: The account concerns Linux BPF ring-buffer reservation/commit interfaces; probe coverage is not determined by the buffer. This describes buffered transport, not guaranteed instrumentation of every system event. No stress test was performed.

S-OT10 - ETW - About Event Tracing; EVENT_TRACE_PROPERTIES structure

Author/institution: Microsoft. **Date/version:** Continuously maintained; checked in 2026. **Evidence:** P; Research-Supported Method.

Original source: ETW - About Event Tracing; EVENT_TRACE_PROPERTIES structure.

Inspection: A3; About Event Tracing; EVENT_TRACE_PROPERTIES: EventsLost, LogBuffersLost, RealTimeBuffersLost and Remarks.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: ETW distinguishes events not recorded, logging buffers not written and real-time buffers not delivered to consumers.

Conditions and limits: Providers, sessions and consumers must be configured through the relevant Windows APIs; loss statistics require the correct scope of the monitored tracing session. API availability establishes neither the target client’s required provider configuration nor the current account’s permissions.

S-OT11 - Build an Endpoint Security app - WWDC20 Session 10159

Author/institution: Apple. **Date/version:** 2020. **Evidence:** P; Research-Supported Method.

Original source: Build an Endpoint Security app - WWDC20 Session 10159.

Inspection: A3; Official transcript, lines 164-206 and 248-304: subscriptions, queues, AUTH/NOTIFY, caching, muting and processing-related loss.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: Endpoint Security publishes subscribed system events; AUTH can block operations. Filtering and processing queues affect coverage.

Conditions and limits: An authorized client must subscribe to the relevant events; caching, muting and processing delays affect delivery. Historical API guidance requires separate 2026-version and entitlement checks. The separate seq_num documentation was available only as search excerpts.

S-OT12 - inotify(7) - Linux manual page

Author/institution: Linux man-pages project. **Date/version:** Continuously maintained; checked in 2026. **Evidence:** P; Research-Supported Method.

Original source: inotify(7) - Linux manual page.

Inspection: A3; /proc interfaces; Limitations and caveats; event coalescing, overflow and absent actor identity.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: inotify can merge identical unread events, discard overflowing events and omit the initiating process or user identity.

Conditions and limits: Claims concern watched objects and enabled notifications within inotify's filesystem and event semantics. Filesystem notifications alone cannot reliably count all activities or attribute them to an agent. No local reproduction was performed.

S-OT15 - AgentSight: System-Level Observability for AI Agents Using eBPF

Author/institution: Yusheng Zheng; Yanpeng Hu; Tong Yu; Andi Quinn. **Date/version:** 2025. **Evidence:** M; E; Research-Supported Method.

Original source: AgentSight: System-Level Observability for AI Agents Using eBPF. **DOI:** 10.48550/arXiv.2508.02736.

Inspection: A3; Pinned arXiv v1; §3 mechanism; §5 environment, Table 1 and three cases.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: Three repetitions of each of three task categories reported 2.9% average overhead and 4.9% maximum individual overhead.

Conditions and limits: Ubuntu 22.04, Linux 6.14.0, Claude Code 1.0.62 and fixed repositories; three workflows, each repeated three times. Small samples and fixed tasks/versions; cases provide no general error-detection precision/recall. Neither Yukio nor other platforms were tested.

S-OT16 - Semantic Conventions for GenAI agent and framework spans

Author/institution: OpenTelemetry GenAI Semantic Conventions Contributors. **Date/version:** Development-branch snapshot, 2026-10-17. **Evidence:** M; P; Research-Supported Method.

Original source: Semantic Conventions for GenAI agent and framework spans.

Inspection: A3; Development status; Agent/Workflow/Tool operations and fields; migration notice on the old site.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: The GenAI agent conventions were in Development status and defined Agent, Workflow and Tool operation semantics.

Conditions and limits: The inherited review used the dedicated repository's main-branch snapshot on the check date; adoption requires independent verification. Main is mutable and the old official page is no longer maintained. Conventions do not prove client field implementation or complete collection.

S-OT20 - Virtual Time and Global States of Distributed Systems

Author/institution: Friedemann Mattern. **Date/version:** 1989. **Evidence:** K; Research-Supported Method.

Original source: Virtual Time and Global States of Distributed Systems.

Inspection: A3; §6 Virtual time; §7 Vector time; PDF pages 5-8: definitions, protocol and componentwise order.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: With full participation and time propagation, vector clocks preserve causal partial order and concurrency relationships.

Conditions and limits: Participating processes update and propagate vectors according to the protocol; comparisons remain within that event/communication model. Vector dimensions and propagation have costs. Arbitrary logs lacking propagated information cannot retrospectively yield every causal relationship.

S-OT22 - Distributed Snapshots: Determining Global States of Distributed Systems

Author/institution: K. Mani Chandy; Leslie Lamport. **Date/version:** 1985. **Evidence:** K; Research-Supported Method.

Original source: Distributed Snapshots: Determining Global States of Distributed Systems. **DOI:** 10.1145/214451.214456.

Inspection: A3; §1 stable properties; §2 system model; marker algorithm and consistent global-state discussion.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: A consistent global state includes in-transit messages. The original snapshot algorithm requires participating processes and reliable FIFO channels.

Conditions and limits: Participants execute the marker protocol; channels are reliable and FIFO, delays finite, and in-transit messages included. Passive incomplete logs do not automatically meet these assumptions. A phase ending differs from termination of the entire computation.

S-OT23 - Unreliable Failure Detectors for Reliable Distributed Systems

Author/institution: Tushar Deepak Chandra; Sam Toueg. **Date/version:** 1996. **Evidence:** K; Research-Supported Method.

Original source: Unreliable Failure Detectors for Reliable Distributed Systems. **DOI:** 10.1145/226643.226647.

Inspection: A1; Cornell author-page abstract; JACM 43(2):225-267 metadata; 1995 technical-report link.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: Failure detectors are classified by completeness and accuracy; asynchronous detection can incorrectly suspect failures.

Conditions and limits: The abstract concerns failure detection and consensus with asynchronous, crash-prone processes. Exact theorem conditions still require full-text review. Readable full text was not obtained. The knowledge base does not reproduce complete proofs or precise weakest conditions for detector classes.

S-OT25 - The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing

Author/institution: Tyler Akidau; Robert Bradshaw; Craig Chambers; Slava Chernyak; Rafael J. Fernández-Moctezuma; Reuven Lax; Sam McVeety; Daniel Mills; Frances Perry; Eric Schmidt; Sam Whittle. **Date/version:** 2015. **Evidence:** M; P; Research-Supported Method.

Original source: The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. **DOI:** 10.14778/2824032.2824076.

Inspection: A3; §2 event/processing time; watermarks and offline-source footnote; windows, triggers and updates.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: Unbounded, out-of-order processing requires distinct event and processing times; watermark completeness depends on source and policy assumptions.

Conditions and limits: The model addresses unbounded or unordered input; any completeness guarantee depends on the particular source and watermark policy. A heuristic watermark is not a general completeness proof. The 2015 system comparison does not describe current tools.

S-OT26 - CloudEvents Specification - Version 1.0.2

Author/institution: CloudEvents Project. **Date/version:** v1.0.2, pinned version. **Evidence:** P; Research-Supported Method.

Original source: CloudEvents Specification - Version 1.0.2.

Inspection: A3; Occurrence/Event/Source terminology; required id, source and specversion attributes.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: One occurrence may produce multiple events. Source plus id distinguishes events and can identify repeated delivery.

Conditions and limits: Producers must follow the pinned CloudEvents 1.0.2 specification and correctly maintain source/id semantics. A format specification guarantees neither exactly-once execution, actual causation nor global order. Its publication date was not independently checked.

S-OT28 - OCEL (Object-Centric Event Log) 2.0 Specification

Author/institution: Alessandro Berti; István Koren; Jan Niklas Adams; Gyunam Park; Benedikt Knopp; Nina Graves; Majid Rafiei; Lukas Liß; Leah Tacke Genannt Unterberg; Yisong Zhang; Christopher Schwanen; Marco Pegoraro; Wil M. P. van der Aalst. **Date/version:** 2024. **Evidence:** P; M; Research-Supported Method.

Original source: OCEL (Object-Centric Event Log) 2.0 Specification. **DOI:** 10.48550/arXiv.2403.01975.

Inspection: A3; Pinned v1, submitted 2024-03-04; cover: 2.0/2023-10-16; §§2-4 and Definition 2.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: OCEL 2.0 represents events involving multiple objects, qualified relationships and time-varying object attributes.

Conditions and limits: Events, objects, types and relationships must already be represented according to the pinned arXiv-v1 OCEL 2.0 metamodel. Representing relationships does not discover actual causation. The old website's 2.0 URL returned 2.1.0pre5 during the inherited review, prompting use of pinned arXiv text.

S-OT29 - Workflow Mining: Discovering Process Models from Event Logs

Author/institution: Wil M. P. van der Aalst; Ton Weijters; Laura Maruster. **Date/version:** 2004. **Evidence:** K; M; Research-Supported Method.

Original source: Workflow Mining: Discovering Process Models from Event Logs. **DOI:** 10.1109/TKDE.2004.47.

Inspection: A3; Author manuscript, first seven pages; §2 Petri nets and α-algorithm model/log assumptions.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: Classical α-algorithm rediscovery guarantees apply only to specified workflow-net classes and event logs meeting the required conditions.

Conditions and limits: The relevant guarantees require the specified net class, complete noise-free logs and explicit case identities. They do not guarantee recovering true workflows from noise, missing rare paths or arbitrary unstructured agent activity.

S-OT31 - Conformance Checking over Uncertain Event Data

Author/institution: Marco Pegoraro; Merih Seran Uysal; Wil M. P. van der Aalst. **Date/version:** 2020 preprint; 2021 journal article. **Evidence:** M; E; Research-Supported Method.

Original source: Conformance Checking over Uncertain Event Data. **DOI:** 10.1016/j.is.2021.101810.

Inspection: A3; Uncertain activities, times and existence; behavior nets; alignment bounds; §6 synthetic experiments and complexity.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: Explicit uncertain event sets yield lower/upper conformance-cost bounds over compatible traces, with potentially rapid combinatorial growth.

Conditions and limits: Activity, time and existence ranges, the conformance model and cost function must already be specified. Bounds depend on encoded uncertainty, are not state probabilities and omit unmodelled collection gaps. No local reproduction was performed.

S-OT32 - Runtime Verification for LTL and TLTL

Author/institution: Andreas Bauer; Martin Leucker; Christian Schallhart. **Date/version:** 2011. **Evidence:** K; M; Research-Supported Method.

Original source: Runtime Verification for LTL and TLTL. **DOI:** 10.1145/2000799.2000800.

Inspection: A3; Author manuscript §§1.1-1.3 and §2 LTL3, first seven PDF pages; final metadata checked against the author's institution.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: LTL3 assigns true, false or inconclusive to a finite prefix by considering all future extensions.

Conditions and limits: Monitoring uses known propositions and explicit LTL/TLTL properties; unknown future events differ from missed past events. Monitoring a selected execution and verifying its explicit properties does not establish correctness of the overall user task.

S-OT33 - 16.30/31 Feedback Control Systems, Topic 9: State-space model features

Author/institution: MIT OpenCourseWare, Department of Aeronautics and Astronautics. **Date/version:** 2010. **Evidence:** K; Research-Supported Method.

Original source: 16.30/31 Feedback Control Systems, Topic 9: State-space model features.

Inspection: A3; Lecture dated 2010-09-30, pages 9-6 to 9-10: LTI definitions, unobservable directions and rank derivation.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: For a known LTI model, observability is equivalent to the matrix formed from C through CA^(n−1) having rank equal to the state dimension.

Conditions and limits: A known finite-dimensional LTI model and noiseless input/output observations are assumed. These original teaching materials are not Kalman's first paper. The conditions do not cover arbitrary unknown or nonlinear agents.

S-GE16 - Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena

Author/institution: Lianmin Zheng et al.. **Date/version:** 2023. **Evidence:** E; M; Research-Supported Method.

Original source: Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena.

Inspection: A3; §3.3 position, verbosity, self-preference and reasoning limitations; human-preference agreement experiments.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: The studied models and protocols showed position/verbosity biases. The paper explicitly retained uncertainty about self-enhancement because of insufficient samples.

Conditions and limits: Specified model outputs from MT-Bench and Chatbot Arena, human preference labels, prompting protocols and constructed response pairs with order transformations. Human-preference agreement is not factual correctness. Candidate biases are not all confirmed, and results cannot cover every 2026 judge.

S-GE19 - Does Context Matter? ContextualJudgeBench for Evaluating LLM-based Judges in Contextual Settings

Author/institution: Austin Xu; Srijan Bansal; Yifei Ming; Semih Yavuz; Shafiq Joty. **Date/version:** 2025. **Evidence:** M; E; Research-Supported Method.

Original source: Does Context Matter? ContextualJudgeBench for Evaluating LLM-based Judges in Contextual Settings. **DOI:** 10.18653/v1/2025.acl-long.470.

Inspection: A3; Eight contextual tasks, dataset construction, order-swapped consistent accuracy, results and appendix analyses.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: The benchmark examines support, completeness, concision, refusal and other contextual judgments. Consistent accuracy requires both orders correct, yielding a 25% random baseline.

Conditions and limits: Judges evaluate constructed response pairs in eight specified context-dependent task categories. This is not ordinary two-choice accuracy’s 50% baseline. Some analyses use model meta-judges; length associations alone are not causal proof. Task coverage is bounded.

S-GE21 - SelectiveNet: A Deep Neural Network with an Integrated Reject Option

Author/institution: Yonatan Geifman; Ran El-Yaniv. **Date/version:** 2019. **Evidence:** M; E; Research-Supported Method.

Original source: SelectiveNet: A Deep Neural Network with an Integrated Reject Option.

Inspection: A3; Selective-risk/coverage definitions, training/calibration procedure and experimental Table 1.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: Prediction and rejection selection are jointly trained; models with abstention are compared using coverage together with conditional risk.

Conditions and limits: Specified supervised tasks/distributions support joint learning, with selective risk and coverage measured on validation/test examples. Training-target coverage differs from realized test coverage. Bounded tasks and distributions do not establish abstention guarantees for arbitrary log states.

S-GE31 - Strictly Proper Scoring Rules, Prediction, and Estimation

Author/institution: Tilmann Gneiting; Adrian E. Raftery. **Date/version:** 2007. **Evidence:** K; Research-Supported Method.

Original source: Strictly Proper Scoring Rules, Prediction, and Estimation. **DOI:** 10.1198/016214506000001437.

Inspection: A3; Author full text: proper-scoring definitions, quadratic/logarithmic scores and probabilistic prediction.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: Scoring-rule theory identifies rules that reward truthful probability reporting in expectation and supports interpretation of Brier/log scores.

Conditions and limits: Prediction distributions, outcome space and scoring direction are explicit; expectation is under the true outcome distribution. The theory needs a defined prediction target and reference outcomes. A good probabilistic score alone does not prove one state judgment or explanation correct.

S-HI01 - Toward a Theory of Situation Awareness in Dynamic Systems

Author/institution: Mica R. Endsley. **Date/version:** 1995. **Evidence:** K; M; H; Research-Supported Method.

Original source: Toward a Theory of Situation Awareness in Dynamic Systems. **DOI:** 10.1518/001872095779049543.

Inspection: A2; Original text: SA levels, mental models and goals; author-uploaded PDF pages 5, 11-13, 26; browser numbering may differ by one.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: C-04-001: Perception, comprehension and projection are distinct analytical levels.; C-04-002: Mental models help interpret situations; some mechanisms are explicitly hypotheses.

Conditions and limits: Goal-directed operators in dynamic systems; the three levels are a theoretical decomposition. The framework spans dynamic-control domains, not desktop-agent experiments.; Not every cited experiment was verified; the framework’s predictions cannot all be treated as established.

S-HI18 - Effects of Intelligent Notification Management on Users and Their Tasks

Author/institution: Shamsi T. Iqbal; Brian P. Bailey. **Date/version:** 2008. **Evidence:** E; M; Research-Supported Method.

Original source: Effects of Intelligent Notification Management on Users and Their Tasks. **DOI:** 10.1145/1357054.1357070.

Inspection: A3; Breakpoint classification and methods; second study N=16, two primary-task types, delayed/immediate comparison and classifier limitations.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: C-06-009: Delivering notifications at task breakpoints reduced some interruption costs in the studied tasks.

Conditions and limits: The second experiment involved 16 participants, programming/graphics tasks and peripheral notifications; immediate and breakpoint delivery were compared. Application switches need not be cognitive task boundaries; abundant non-breakpoints can inflate overall classification accuracy.; Delay trades reduced disruption against timeliness; mean waiting times cannot be transferred as fixed rules.

S-HI19 - On the Need for Attention-Aware Systems: Measuring Effects of Interruption on Task Performance, Error Rate, and Affective State

Author/institution: Brian P. Bailey; Joseph A. Konstan. **Date/version:** 2006. **Evidence:** E; Research-Supported Method.

Original source: On the Need for Attention-Aware Systems: Measuring Effects of Interruption on Task Performance, Error Rate, and Affective State. **DOI:** 10.1016/j.chb.2005.12.009.

Inspection: A3; N=50, six primary-task types; peripheral tasks during/between primary tasks; errors, time and anxiety outcomes.. Inherited from the KB inspection dated 2026-10-17; not newly reverified for this application report.

Supports: C-06-009: Notification timing affects task-completion and emotional costs; interruption is not cost-free.

Conditions and limits: Fifty participants performed six controlled primary-task types. Peripheral-task presentation during execution was compared with presentation between tasks. Short controlled tasks cannot yield a universal number of recovery minutes for every work interruption.

14.7 Companion evidence and continuation files

The companion evidence index contains source records, aliases, input hashes, pinned code-file hashes, actual RUN results and planned validation IDs. The evidence package includes the portable probe script, its actual results, this report and the cited supplied research/textbook references. It excludes private session data and the public Git checkout. Reproducing a probe requires a separate checkout of the stated commit; the recorded results do not imply a new execution when the package is opened.